

关于本文档

文档名称：《Kubernetes 资源对象和 Pod 资源》

使用协议：《知识共享公共许可协议(CCPL)》

贡献者

贡献者名称	贡献度	文档变更记录	个人主页
马哥（马永亮）	主编		http://github.com/iKubernetes/
王晓春	作者	kubernetes-v1.32.3	http://www.wangxiaochun.com

文档协议

署名要求：使用本系列文档，您必须保留本页中的文档来源信息，具体请参考《知识共享 (Creative Commons) 署名4.0公共许可协议国际版》。

非商业化使用：遵循《知识共享公共许可协议(CCPL)》，并且您不能将本文档用于马哥教育相关业务之外的其他任何商业用途。

您的权利：遵循本协议后，在马哥教育相关业务之外的领域，您将有以下使用权限：

共享 — 允许以非商业性质复制本作品。

改编 — 在原基础上修改、转换或以本作品为基础进行重新编辑并用于个人非商业使用。

致谢

本文档中，部分素材参考了相关项目的文档，以及通过搜索引擎获得的内容，这里先一并向相关的贡献者表示感谢。

Kubernetes 资源对象和 Pod 资源

本章内容

- 资源对象
- 名称空间

• Pod 资源

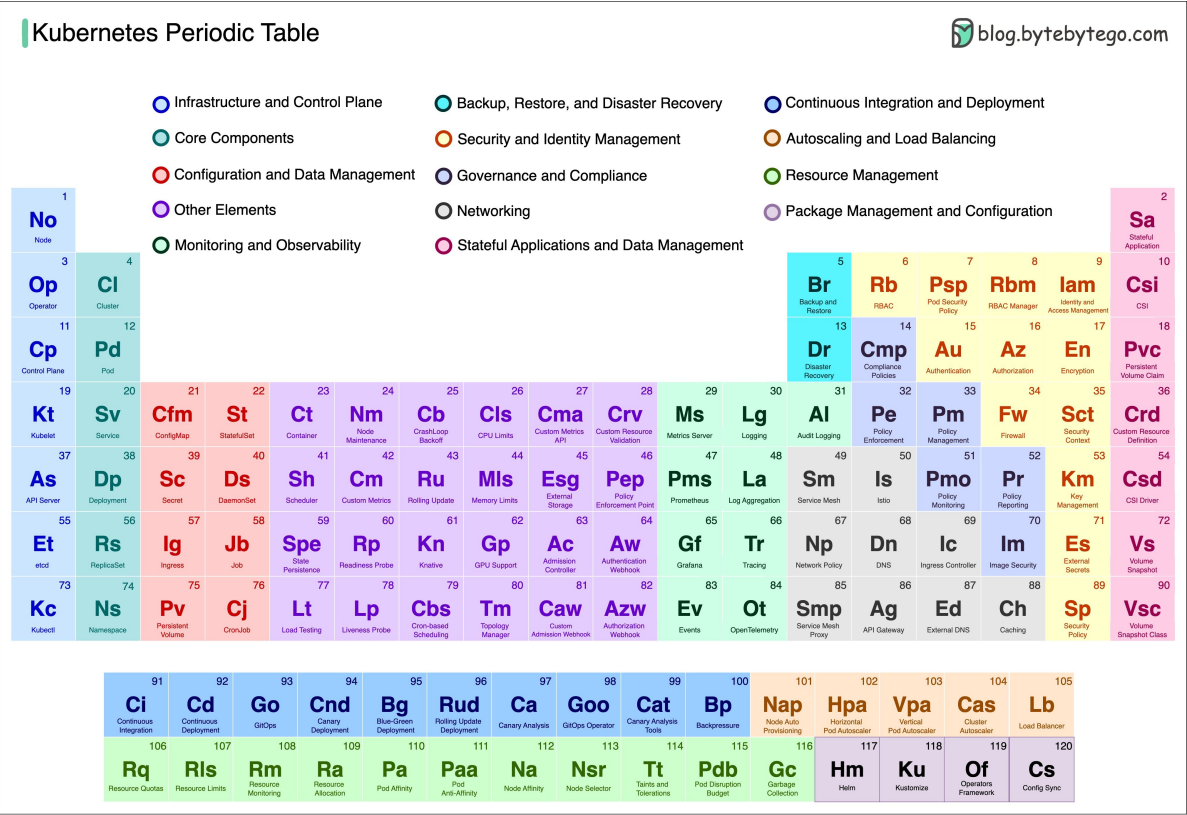
1 资源对象

1.1 资源对象及 API 群组

1.1.1 Kubernetes 的资源对象

Kubernetes 元素周期表

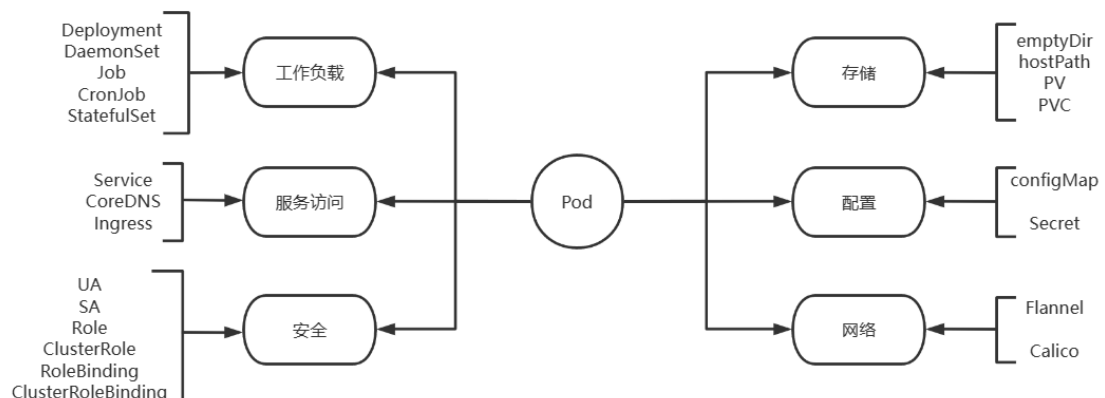
Kubernetes 是一个分布式系统平台,功能强大,所以涉及到的内容很多,下图为 Kubernetes 相关的组件和技术称为Kubernetes 元素周期表



Kubernetes 系统将一切事物都称为资源对象, 相当于面向对象的思想



Kubernetes 常见资源



有一些独立存在,即不依赖于其它对象存在的资源类型, Kubernetes 提供了单独的 API 资源, 其遵循 REST 风格组织并管理这些资源对象

componentstatuses	cs	v1
<code>false</code> ComponentStatus		
configmaps	cm	v1
<code>true</code> ConfigMap		
endpoints	ep	v1
<code>true</code> Endpoints		
events	ev	v1
<code>true</code> Event		
limitranges	limits	v1
<code>true</code> LimitRange		
namespaces	ns	v1
<code>false</code> Namespace		
nodes	no	v1
<code>false</code> Node		
persistentvolumeclaims	pvc	v1
<code>true</code> PersistentVolumeClaim		
persistentvolumes	pv	v1
<code>false</code> PersistentVolume		
Pods	po	v1
<code>true</code> Pod		
podtemplates		v1
<code>true</code> PodTemplate		
replicationcontrollers	rc	v1
<code>true</code> ReplicationController		
resourcequotas	quota	v1
<code>true</code> ResourceQuota		
secrets		v1
<code>true</code> Secret		
serviceaccounts	sa	v1
<code>true</code> ServiceAccount		
services	svc	v1
<code>true</code> Service		
mutatingwebhookconfigurations		admissionregistration.k8s.io/v1
<code>false</code> MutatingWebhookConfiguration		
validatingwebhookconfigurations		admissionregistration.k8s.io/v1
<code>false</code> ValidatingWebhookConfiguration		
customresourcedefinitions	crd,crds	apiextensions.k8s.io/v1
<code>false</code> CustomResourceDefinition		
apiservices		apiregistration.k8s.io/v1
<code>false</code> APIService		
controllerrevisions		apps/v1
<code>true</code> ControllerRevision		
daemonsets	ds	apps/v1
<code>true</code> DaemonSet		
deployments	deploy	apps/v1
<code>true</code> Deployment		
replicasets	rs	apps/v1
<code>true</code> ReplicaSet		
statefulsets	sts	apps/v1
<code>true</code> StatefulSet		
tokenreviews		authentication.k8s.io/v1
<code>false</code> TokenReview		
localsubjectaccessreviews		authorization.k8s.io/v1
<code>true</code> LocalSubjectAccessReview		
selfsubjectaccessreviews		authorization.k8s.io/v1
<code>false</code> SelfSubjectAccessReview		
selfsubjectrulesreviews		authorization.k8s.io/v1
<code>false</code> SelfSubjectRulesReview		

subjectaccessreviews			authorization.k8s.io/v1
false	SubjectAccessReview		
horizontalpodautoscalers	hpa		autoscaling/v2
true	HorizontalPodAutoscaler		
cronjobs	cj		batch/v1
true	CronJob		
jobs			batch/v1
true	Job		
certificatesigningrequests	csr		certificates.k8s.io/v1
false	CertificateSigningRequest		
leases			coordination.k8s.io/v1
true	Lease		
endpointslices			discovery.k8s.io/v1
true	EndpointSlice		
events	ev		events.k8s.io/v1
true	Event		
flowschemas			
flowcontrol.apiserver.k8s.io/v1beta3	false	FlowSchema	
prioritylevelconfigurations			
flowcontrol.apiserver.k8s.io/v1beta3	false	PriorityLevelConfiguration	
ingressclasses			networking.k8s.io/v1
false	IngressClass		
ingresses	ing		networking.k8s.io/v1
true	Ingress		
networkpolicies	netpol		networking.k8s.io/v1
true	NetworkPolicy		
runtimeclasses			node.k8s.io/v1
false	RuntimeClass		
poddisruptionbudgets	pdb		policy/v1
true	PodDisruptionBudget		
clusterrolebindings			rbac.authorization.k8s.io/v1
false	ClusterRoleBinding		
clusterroles			rbac.authorization.k8s.io/v1
false	ClusterRole		
rolebindings			rbac.authorization.k8s.io/v1
true	RoleBinding		
roles			rbac.authorization.k8s.io/v1
true	Role		
priorityclasses	pc		scheduling.k8s.io/v1
false	PriorityClass		
csidrivers			storage.k8s.io/v1
false	CSIDriver		
csinodes			storage.k8s.io/v1
false	CSINode		
csistoragecapacities			storage.k8s.io/v1
true	CSISStorageCapacity		
storageclasses	sc		storage.k8s.io/v1
false	StorageClass		
volumeattachments			storage.k8s.io/v1
false	VolumeAttachment		

```
[root@master1 ~]#kubectl api-versions
```

```
admissionregistration.k8s.io/v1
```

```
apiextensions.k8s.io/v1
```

```
apiregistration.k8s.io/v1
```

```
apps/v1
```

```
authentication.k8s.io/v1
```

```

authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1
events.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1beta3
networking.k8s.io/v1
node.k8s.io/v1
policy/v1
rbac.authorization.k8s.io/v1
scheduling.k8s.io/v1
storage.k8s.io/v1
v1

```

```
[root@master1 ~]#kubectl api-resources --api-group=apps
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
controllerrevisions		apps/v1	true	ControllerRevision
daemonsets	ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true	Deployment
replicasets	rs	apps/v1	true	ReplicaSet
statefulsets	sts	apps/v1	true	StatefulSet

```
[root@master1 ~]#kubectl api-resources | awk '{print $1}'
```

```

NAME
bindings
componentstatuses
configmaps
endpoints
events
limitranges
namespaces
nodes
persistentvolumeclaims
persistentvolumes
pods
podtemplates
replicationcontrollers
resourcequotas
secrets
serviceaccounts
services
mutatingwebhookconfigurations
validatingwebhookconfigurations
customresourcedefinitions
apiservices
controllerrevisions
daemonsets
deployments
replicasets
statefulsets
tokenreviews
localsubjectaccessreviews
selfsubjectaccessreviews

```

```
selfsubjectrulesreviews
subjectaccessreviews
horizontalpodautoscalers
cronjobs
jobs
certificatesigningrequests
leases
endpointslices
events
flowschemas
prioritylevelconfigurations
ingressclasses
ingresses
networkpolicies
runtimeclasses
poddisruptionbudgets
clusterrolebindings
clusterroles
rolebindings
roles
priorityclasses
csidrivers
csinodes
csistoragecapacities
storageclasses
volumeattachments
```

```
[root@master1 ~]#kubectl api-resources | awk '{print $1}'|wc -l
56
```

#查看所有资源

```
[root@ubuntu2204 ~]#kubectl get all -A
```

NAMESPACE	NAME	READY	STATUS
RESTARTS	AGE		
default	pod/myapp-6f9c7b4668-6vjv6	1/1	Running
1 (5h9m ago)	7h32m		
default	pod/myapp-6f9c7b4668-725r9	1/1	Running
1 (5h9m ago)	7h32m		
default	pod/myapp-6f9c7b4668-tcczj	1/1	Running
1 (5h9m ago)	7h32m		
kube-system	pod/calico-kube-controllers-5994f46b69-dq9pc	1/1	Running
1 (5h9m ago)	7h38m		
kube-system	pod/calico-node-bvmcw	1/1	Running
1 (5h9m ago)	7h38m		
kube-system	pod/calico-node-f46v4	1/1	Running
4 (5h8m ago)	7h38m		
kube-system	pod/calico-node-fkdvk	1/1	Running
2 (5h8m ago)	7h38m		
kube-system	pod/calico-node-g78b5	1/1	Running
2 (5h10m ago)	7h38m		
kube-system	pod/calico-node-twx6t	1/1	Running
3 (5h10m ago)	7h38m		
kube-system	pod/calico-node-xbt4b	1/1	Running
2 (5h10m ago)	7h38m		
kube-system	pod/coredns-6db9bd4cd7-t5b4v	1/1	Running
1 (5h9m ago)	7h38m		

kube-system	pod/dashboard-metrics-scraper-77b667b99d-dw4d9	1/1	Running		
1 (5h9m ago)	7h37m				
kube-system	pod/kubernetes-dashboard-74fb9f77fb-bhz2v	1/1	Running		
2 (5h8m ago)	7h37m				
kube-system	pod/metrics-server-dfb478476-lsdqx	1/1	Running		
2 (5h8m ago)	7h38m				
kube-system	pod/node-local-dns-5lqq2	1/1	Running		
1 (5h9m ago)	7h38m				
kube-system	pod/node-local-dns-6bjp4	1/1	Running		
1 (5h9m ago)	7h38m				
kube-system	pod/node-local-dns-hts5v	1/1	Running		
1 (5h10m ago)	7h38m				
kube-system	pod/node-local-dns-jxrhh	1/1	Running		
1 (5h10m ago)	7h38m				
kube-system	pod/node-local-dns-nmvwh	1/1	Running		
1 (5h9m ago)	7h38m				
kube-system	pod/node-local-dns-wv8mx	1/1	Running		
1 (5h10m ago)	7h38m				
NAMESPACE	NAME	TYPE	CLUSTER-IP		
EXTERNAL-IP	PORT(S)	AGE			
default	service/kubernetes	ClusterIP	10.68.0.1		
<none>	443/TCP				
default	service/myapp	NodePort	10.68.78.97		
<none>	80:31890/TCP				
kube-system	service/dashboard-metrics-scraper	ClusterIP	10.68.54.201		
<none>	8000/TCP				
kube-system	service/kube-dns	ClusterIP	10.68.0.2		
<none>	53/UDP, 53/TCP, 9153/TCP				
kube-system	service/kube-dns-upstream	ClusterIP	10.68.143.215		
<none>	53/UDP, 53/TCP				
kube-system	service/kubernetes-dashboard	NodePort	10.68.50.181		
<none>	443:30313/TCP				
kube-system	service/metrics-server	ClusterIP	10.68.196.91		
<none>	443/TCP				
kube-system	service/node-local-dns	ClusterIP	None		
<none>	9253/TCP				
NAMESPACE	NAME	DESIRED	CURRENT	READY	UP-TO-
DATE	AVAILABLE	NODE	SELECTOR	AGE	
kube-system	daemonset.apps/calico-node	6	6	6	6
6	kubernetes.io/os=linux	7h38m			
kube-system	daemonset.apps/node-local-dns	6	6	6	6
6	<none>	7h38m			
NAMESPACE	NAME	READY	UP-TO-DATE		
AVAILABLE	AGE				
default	deployment.apps/myapp	3/3	3		3
	7h32m				
kube-system	deployment.apps/calico-kube-controllers	1/1	1		1
	7h38m				
kube-system	deployment.apps/coredns	1/1	1		1
	7h38m				
kube-system	deployment.apps/dashboard-metrics-scraper	1/1	1		1
	7h37m				
kube-system	deployment.apps/kubernetes-dashboard	1/1	1		1
	7h37m				

```
kube-system deployment.apps/metrics-server 1/1 1 1
7h38m

NAMESPACE NAME DESIRED
CURRENT READY AGE
default replicaset.apps/myapp-6f9c7b4668 3 3
3 7h32m
kube-system replicaset.apps/calico-kube-controllers-5994f46b69 1 1
1 7h38m
kube-system replicaset.apps/coredns-6db9bd4cd7 1 1
1 7h38m
kube-system replicaset.apps/dashboard-metrics-scraper-77b667b99d 1 1
1 7h37m
kube-system replicaset.apps/kubernetes-dashboard-74fb9f77fb 1 1
1 7h37m
kube-system replicaset.apps/metrics-server-dfb478476 1 1
1 7h38m
[root@ubuntu2204 ~]#
```

范例：查看CRD

```
[root@master1 ~]#kubectl get crd
NAME CREATED AT
bgpconfigurations.crd.projectcalico.org 2023-07-22T12:10:37Z
bgpfilters.crd.projectcalico.org 2023-07-22T12:10:37Z
bgppeers.crd.projectcalico.org 2023-07-22T12:10:37Z
blockaffinities.crd.projectcalico.org 2023-07-22T12:10:37Z
caliconodestatuses.crd.projectcalico.org 2023-07-22T12:10:37Z
clusterinformations.crd.projectcalico.org 2023-07-22T12:10:37Z
felixconfigurations.crd.projectcalico.org 2023-07-22T12:10:37Z
globalnetworkpolicies.crd.projectcalico.org 2023-07-22T12:10:37Z
globalnetworksets.crd.projectcalico.org 2023-07-22T12:10:37Z
hostendpoints.crd.projectcalico.org 2023-07-22T12:10:37Z
ipamblocks.crd.projectcalico.org 2023-07-22T12:10:37Z
ipamconfigs.crd.projectcalico.org 2023-07-22T12:10:37Z
ipamhandles.crd.projectcalico.org 2023-07-22T12:10:37Z
ippools.crd.projectcalico.org 2023-07-22T12:10:37Z
ipreservations.crd.projectcalico.org 2023-07-22T12:10:37Z
kubecontrollersconfigurations.crd.projectcalico.org 2023-07-22T12:10:37Z
networkpolicies.crd.projectcalico.org 2023-07-22T12:10:37Z
networksets.crd.projectcalico.org 2023-07-22T12:10:37Z
redisfailovers.databases.spotahome.com 2024-04-29T14:26:08Z
```

1.1.1.1 名称空间级别资源

- 工作负载型资源(workload):
 - Pod：这是我们K8S 中最小的组成部分，特点就是和我们的 pause 共享我们的网络栈、共享存储卷。
 - replicaset：也就是 RS ， 这是我们的调度器控制器，负责管理 pod 的创建，通过我们标签的选择去控制pod 的副本数
 - Deployment：控制器，通过控制 RS 的创建去创建 pod
 - statefulset：主要是为了我们的有状态服务去建立的这么一个管理器。
 - Daemonset：可以在我们每个节点都运行一个 pod 的组件。
 - job：我们的工作，为了批处理任务。
 - cronjob：我们的轮询工作，为了批处理任务。

- **服务发现及负载均衡型资源(ServiceDiscovery LoadBalance):**
 - Service (简称为svc) : 为了把我们的服务暴露出去
 - Ingress: 为了把我们的服务暴露出去,由此接受外部的访问
- **配置与存储型资源:**
 - Volume: 存储卷, 为了给 pod 提供一个持久化的能力
 - CSI (容器存储接口) : ,可以扩展各种各样的第三方存储卷, 在K8S中很多资源需要灵活的提供给我们的容器去运行, 但是在K8S中定义了 CSI 接口只要存储资源符合这个接口的规范, 那我这个 K8S 集群中 pod 就能够去调用这个存储, 这种接口就叫做CSI, 基本上现在市面上能够见到的存储都是支持 CSI 的。
- **特殊类型的存储卷:**
 - ConfigMap (当配置中心来使用的资源来型) : 一般来说用来存储配置文件, 达到热更新的状态。
 - Secret: 加密的方案去存储数据的, 所以可以通过它来保存一些密码文件、密钥等等
 - DownwardAPI: 把外部环境中的信息输出给容器、这个和CSI也比较像, 也就意味着可以通过一个下载文件的接口, 通过这个接口可以去调用里面的数据, 存储和上传等等。

1.1.1.2 集群级资源

集群级资源:

- Namespace (名称空间)
- Node (工作节点) 、Role (角色)
- ClusterRole (集群角色)
- RoleBinding (重新链接)
- ClusterRoleBinding (集群重新链接)

1.1.1.3 元数据型资源

元数据型资源 (根据一些指标去进行对应的操作) :

- HPA
- PodTemplate (pod模板)
- LimitRange (资源限制)

1.1.2 资源及其在 API 中的组织形式

Kubernetes 利用标准的 RESTful 术语来描述其 API 概念。

- 资源类型: 是指在 URL 中使用的名称, 如 Pod、Namespace 和 Service 等, 其 URL 格式为"/GROUP/VERSION/RESOURCE", 示例: /apps/v1/deployment
- 所有资源类型都有一个对应的 JSON 表示格式: kind(种类), 在 K8s 中用户创建对象必须以 JSON格式提交对象的配置信息。
- 隶属于同一资源类型的对象组成的列表称为 collection(集合), 如 PodList
- 某种类型的单个实例称为"resource"(资源)或"object"(对象), 如运行的名为 pod-test 的 Pod 对象

kind 代表了资源对象所属的类型, 如 Namespace、Deployment、Service 等类型, 这些类型又可以分为三个类别:

- Object 类: 对象表示 K8s 系统上的持久化实体, Namespace、Deployment、Service 和 Pod 都属于该类别
- List 列表类: 列表是指同一类型资源的集合, 如 PodList、NodeList 等。
- Simple 简单类: 常用于子对象上执行某种特殊操作, 或者管理非持久的实体, 如/binding 或/status 等

在 Kubernetes 系统中，所有的对象型资源都有一个独有的名称表示以实现其幂等的创建及获取操作。有的资源隶属于集群范畴，如 Namespace 和 PersistentVolume，而多数资源类型则受限于名称空间，如 Pod、Deployment 和 Service 等。名称空间级别的资源的 URL 路径中含有其所属空间的名称，这些资源对象在名称空间被删除时会被一并删除，这些资源的访问也受控于其所属的名称空间级别的授权。

Kubernetes 将 API 分割为多个逻辑组合，称为**API 群组**，不同的群组支持单独启用或禁用，并可以再次分解。群组化管理的 API 使得其可以更轻松的进行扩展。当前 K8s 集群系统上的 API server 上的相关信息可以使用 `kubectl api-versions` 获取。配置资源清单时会使用 API 群组：

#显示API群组,结果格式为: **GROUP_NAME/VERSION**,同一个组可以有多个版本并存
#**GROUP_NAME**: API群组名, 如果省略表示属于**core**核心组
#**VERSION:v1**,经过验证的稳定版本, 可以生产环境使用,如:**apps/v1**
#**alpha**:内测,可能包含错误, 生产不建议使用
#**beta**: 公测, 存在变动的可能或者潜在的问题, 生产不建议使用,如:**autoscaling/v2beta2**

```
[root@master1 ~]#kubectl api-versions
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
auth.democrd.io/v1alpha1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1
events.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1beta2
flowcontrol.apiserver.k8s.io/v1beta3
networking.k8s.io/v1
node.k8s.io/v1
policy/v1
rbac.authorization.k8s.io/v1
scheduling.k8s.io/v1
storage.k8s.io/v1
storage.k8s.io/v1beta1
v1
```

```
[root@master1 ~]#kubectl api-versions
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2beta1
autoscaling/v2beta2
batch/v1
batch/v1beta1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1
discovery.k8s.io/v1beta1
```

```

events.k8s.io/v1
events.k8s.io/v1beta1
flowcontrol.apiserver.k8s.io/v1beta1
networking.k8s.io/v1
node.k8s.io/v1
node.k8s.io/v1beta1
policy/v1
policy/v1beta1
rbac.authorization.k8s.io/v1
scheduling.k8s.io/v1
storage.k8s.io/v1
storage.k8s.io/v1beta1
v1

```

#每个资源都有自己的APIVERSION和是否属于名称空间

```
[root@master1 ~]#kubectl api-resources
```

NAME		SHORTNAMES	APIVERSION
	NAMESPACED	KIND	
bindings			v1
true	Binding		
componentstatuses		cs	v1
false	ComponentStatus		
configmaps		cm	v1
true	ConfigMap		
endpoints		ep	v1
true	Endpoints		
events		ev	v1
true	Event		
limitranges		limits	v1
true	LimitRange		
namespaces		ns	v1
false	Namespace		
nodes		no	v1
false	Node		
persistentvolumeclaims		pvc	v1
true	PersistentVolumeClaim		
persistentvolumes		pv	v1
false	PersistentVolume		
Pods		po	v1
true	Pod		
podtemplates			v1
true	PodTemplate		
replicationcontrollers		rc	v1
true	ReplicationController		
resourcequotas		quota	v1
true	ResourceQuota		
secrets			v1
true	Secret		
serviceaccounts		sa	v1
true	ServiceAccount		
services		svc	v1
true	Service		
mutatingwebhookconfigurations			admissionregistration.k8s.io/v1
false	MutatingWebhookConfiguration		
validatingwebhookconfigurations			admissionregistration.k8s.io/v1
false	ValidatingWebhookConfiguration		
customresourcedefinitions		crd,crds	apiextensions.k8s.io/v1
false	CustomResourceDefinition		

apiservices			apiregistration.k8s.io/v1
false	APIService		
controllerrevisions			apps/v1
true	ControllerRevision		
daemonsets		ds	apps/v1
true	DaemonSet		
deployments		deploy	apps/v1
true	Deployment		
replicasets		rs	apps/v1
true	ReplicaSet		
statefulsets		sts	apps/v1
true	StatefulSet		
tokenreviews			authentication.k8s.io/v1
false	TokenReview		
localsubjectaccessreviews			authorization.k8s.io/v1
true	LocalSubjectAccessReview		
selfsubjectaccessreviews			authorization.k8s.io/v1
false	SelfSubjectAccessReview		
selfsubjectrulesreviews			authorization.k8s.io/v1
false	SelfSubjectRulesReview		
subjectaccessreviews			authorization.k8s.io/v1
false	SubjectAccessReview		
horizontalpodautoscalers		hpa	autoscaling/v2
true	HorizontalPodAutoscaler		
cronjobs		cj	batch/v1
true	CronJob		
jobs			batch/v1
true	Job		
certificatesigningrequests		csr	certificates.k8s.io/v1
false	CertificateSigningRequest		
leases			coordination.k8s.io/v1
true	Lease		
endpointslices			discovery.k8s.io/v1
true	EndpointSlice		
events		ev	events.k8s.io/v1
true	Event		
flowschemas			
flowcontrol.apiserver.k8s.io/v1beta2	false	FlowSchema	
prioritylevelconfigurations			
flowcontrol.apiserver.k8s.io/v1beta2	false	PriorityLevelConfiguration	
ingressclasses			networking.k8s.io/v1
false	IngressClass		
ingresses		ing	networking.k8s.io/v1
true	Ingress		
networkpolicies		netpol	networking.k8s.io/v1
true	NetworkPolicy		
runtimeclasses			node.k8s.io/v1
false	RuntimeClass		
poddisruptionbudgets		pdb	policy/v1
true	PodDisruptionBudget		
podsecuritypolicies		psp	policy/v1beta1
false	PodSecurityPolicy		
clusterrolebindings			rbac.authorization.k8s.io/v1
false	ClusterRoleBinding		
clusterroles			rbac.authorization.k8s.io/v1
false	ClusterRole		
rolebindings			rbac.authorization.k8s.io/v1
true	RoleBinding		

```

roles                                rbac.authorization.k8s.io/v1
  true                               Role
priorityclasses                      pc                                scheduling.k8s.io/v1
  false                             PriorityClass
csidrivers                           storage.k8s.io/v1
  false                             CSIDriver
csinodes                             storage.k8s.io/v1
  false                             CSINode
csistoragecapacities                storage.k8s.io/v1beta1
  true                              CSISStorageCapacity
storageclasses                      sc                                storage.k8s.io/v1
  false                             StorageClass
volumeattachments

```

#Kubernetes-v1.24.3有57种资源

```

[root@master1 ~]# kubectl api-resources |wc -l
57

```

#显示指定API群组的资源

```

[root@master1 ~]# kubectl api-resources --api-group=apps
NAME                SHORTNAMES  APIVERSION  NAMESPACED  KIND
controllerrevisions  ds          apps/v1     true        ControllerRevision
daemonsets           ds          apps/v1     true        DaemonSet
deployments          deploy     apps/v1     true        Deployment
replicasets          rs         apps/v1     true        ReplicaSet
statefulsets         sts        apps/v1     true        StatefulSet

```

Kubernetes 的 API 以层级结构组织在一起

Object: 资源型对象, 表现为 http url中path

非Object: 非资源型对象, kubernetes特有, 例如: /healthz

API Server上的每个资源对象, 都在RESTful API中有一个端点

Kubernetes的基本特性就是它的所有资源对象都是模型化的 API 对象, 可以基于api-server对各种资源进行增、删、改、查等操作, 但是这些操作涉及到的不仅仅是资源本身和动作, 而且还涉及到资源和动作之间的内容, 比如api的分组版本和资源andapi的关联即权限授权等。

Object资源型对象对应的 API 群组可以归为以下两类:

- **核心群组(core group):**

在资源的配置信息 `apiVersion` 字段中引用时可以用不用指定路径,如:"apiVersion: v1"

REST 路径为 `/api/v1`

```

https://API_SERVER:PORT/api/v1/namespaces/<NS_NAME>/<RESOURCE_NAME>/<OBJECT_NAME>

```

#default名称空间下的mypod的Pod资源, URL路径直接访问:

```

curl https://API_SERVER:PORT/api/v1/namespaces/default/pods/mypod

```

```

/api/v1/namespaces/<NAMESPACE>/pods/

```

```

/api/v1/namespaces/<NAMESPACE>/pods/<POD_NAME>

```

#示例:

```

kubectl get --raw="/api/v1/namespaces/default/pods/demoapp-78b49597cf-qkj4h"
| jq

```

- **命名群组(named group):**

即有名称的群组

其在 apiVersion 字段中引用的格式为:"apiVersion: \$GROUP_NAME/\$VERSION",如: "apiVersion: apps/v1"

REST 路径为 /apis/\$GROUP_NAME/\$VERSION, 如 /apis/apps/v1

```
https://API_SERVER:HOST/apis/GROUP_NAME/VERSION/namespaces/<NS_NAME>/<RESOURCE_NAME>/<OBJECT_NAME>
```

```
/apis/<GROUP_NAME>/<VERSION>/<NAMESPACE>/default/deployments/  
/apis/<GROUP_NAME>/<VERSION>/<NAMESPACE>/default/deployments/<PODNAME>
```

#示例:

```
kubectl get --raw="/apis/apps/v1/namespaces/kube-system/deployments/coredns"  
| jq
```

范例:

```
[root@master1 ~]#kubectl get --raw="/api/v1/namespaces/default" | jq  
{  
  "kind": "Namespace",  
  "apiVersion": "v1",  
  "metadata": {  
    "name": "default",  
    "uid": "c9cb9aa8-ca5c-4293-9892-c7de230943b4",  
    "resourceVersion": "195",  
    "creationTimestamp": "2021-08-21T01:07:21Z",  
    "labels": {  
      "kubernetes.io/metadata.name": "default"  
    },  
    "managedFields": [  
      {  
        "manager": "kube-apiserver",  
        "operation": "Update",  
        "apiVersion": "v1",  
        "time": "2021-08-21T01:07:21Z",  
        "fieldsType": "FieldsV1",  
        "fieldsV1": {  
          "f:metadata": {  
            "f:labels": {  
              ".": {},  
              "f:kubernetes.io/metadata.name": {}  
            }  
          }  
        }  
      }  
    ],  
  },  
  "spec": {  
    "finalizers": [  
      "kubernetes"  
    ]  
  },  
}
```



```
"status": {
  "phase": "Active"
}
}
```

范例: 获取名为 apps 的 API 群组信息:

```
[root@master1 ~]#kubectl get --raw /apis/apps | jq
{
  "kind": "APIGroup",
  "apiVersion": "v1",
  "name": "apps",
  "versions": [
    {
      "groupVersion": "apps/v1",
      "version": "v1"
    }
  ],
  "preferredVersion": {
    "groupVersion": "apps/v1",
    "version": "v1"
  }
}
```

范例: 获取所有的 Deployment 对象的列表

```
[root@master1 ~]#kubectl get --raw /apis/apps/v1/deployments | jq
{
  "kind": "DeploymentList",
  "apiVersion": "apps/v1",
  "metadata": {
    "resourceVersion": "192070"
  },
  "items": [
    {
      .....
    }
  ]
}
```

1.1.3 访问 Kubernetes REST API

可以使用 curl 作为 http 客户端直接通过 API server 在集群上操作资源对象模拟请求和响应过程, 只不过 kubeadm 部署的 k8s 集群默认只支持 HTTPS 协议访问接口, 但是用户可以使用 `kubectl proxy` 命令来在本地主机上为 API server 启动一个代理网关, 由其支持使用 HTTP 进行通信。

```
#需要提前获取TOKEN才能访问
#清单文件
[root@master1 ~]#cat securiy-sa-token-cluster-admin.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: wang
  namespace: kube-system

---
apiVersion: v1
```

```

kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: wang-secret
  namespace: kube-system
  annotations:
    kubernetes.io/service-account.name: "wang"

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: testsa-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: wang
  namespace: kube-system

#生成资源
[root@master1 ~]#kubectl apply -f securiy-sa-token-cluster-admin.yaml

#查看SA对应的TOKEN
[root@master1 ~]#kubectl get secrets -n kube-system wang-secret -o yaml

#对上面的TOKEN进行base64解码后生成TOKEN变量
[root@master1 ~]#TOKEN=$(echo ZXlKaGJHY2lwa==|base64 -d)

#利用上面生成的TOKEN才能访问
curl -s --cacert /etc/kubernetes/pki/ca.crt -H "Authorization: Bearer ${TOKEN}"
https://kubernetes.wang.org:6443

#返回所有的API列表
curl https://kubernetes.wang.org:6443

#分组API
curl https://kubernetes.wang.org:6443/apis

#API详细信息
curl https://kubernetes.wang.org:6443/apis/autoscaling/v1

#API版本信息
curl https://kubernetes.wang.org:6443/version

#指定具体版本号的API
curl https://kubernetes.wang.org:6443/api/v1

#对etcd的健康性检查, 正常返回OK
curl https://kubernetes.wang.org:6443/healthz/etcd

#获取指标数据
curl https://kubernetes.wang.org:6443/metrics

```

范例：kubernetes API 直接访问

#需要提前获取TOKEN才能访问，此方式比较繁琐

```
[root@master1 ~]#curl -sk https://kubepi.wang.org:6443
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "forbidden: User \"system:anonymous\" cannot get path \"/\"",
  "reason": "Forbidden",
  "details": {},
  "code": 403
}
```

#清单文件

```
[root@master1 ~]#cat securiy-sa-token-cluster-admin.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: wang
  namespace: kube-system
```

```
---
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: wang-secret
  namespace: kube-system
  annotations:
    kubernetes.io/service-account.name: "wang"
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: testsa-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: wang
  namespace: kube-system
```

#生成资源

```
[root@master1 ~]#kubectl apply -f securiy-sa-token-cluster-admin.yaml
```

#查看SA对应的TOKEN

```
[root@master1 ~]#kubectl get secrets -n kube-system wang-secret -o yaml
```

#对上面的TOKEN进行base64解码后生成TOKEN变量

```
[root@master1 ~]#TOKEN=$(echo ZX1kaGJHY2lw==|base64 -d)
```

```
[root@master1 ~]#curl -s --cacert /etc/kubernetes/pki/ca.crt -H "Authorization:
Bearer ${TOKEN}" https://kubepi.wang.org:6443
{
```

```

"paths": [
  "/.well-known/openid-configuration",
  "/api",
  "/api/v1",
  "/apis",
  "/apis/",
  "/apis/admissionregistration.k8s.io",
  "/apis/admissionregistration.k8s.io/v1",
  "/apis/apiextensions.k8s.io",
  .....

```

范例：用代理访问

```

#上面方式比较繁琐，启用代理可以更加方便访问API Server
#前台启动一个代理
[root@master1 ~]#kubectl proxy --port=8081
Starting to serve on 127.0.0.1:8081

#在另一个终端执行下面
#查看所有API列表
[root@master1 ~]#curl -s 127.0.0.1:8081|less
{
  "paths": [
    "/.well-known/openid-configuration",
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/apis/admissionregistration.k8s.io",
    "/apis/admissionregistration.k8s.io/v1",
    "/apis/apiextensions.k8s.io",
    "/apis/apiextensions.k8s.io/v1",
    "/apis/apiregistration.k8s.io",
    "/apis/apiregistration.k8s.io/v1",
    "/apis/apps",
    "/apis/apps/v1",
    "/apis/auth.democrd.io",
    "/apis/auth.democrd.io/v1alpha1",
    .....

{
  "kind": "APIGroupList",
  "apiVersion": "v1",
  "groups": [
    {
      "name": "apiregistration.k8s.io",
      "versions": [
        {
          "groupVersion": "apiregistration.k8s.io/v1",
          "version": "v1"
        }
      ],
      "preferredVersion": {
        "groupVersion": "apiregistration.k8s.io/v1",
        "version": "v1"
      }
    }
  ]
}

```

```

    },
    {
      "name": "apps",
      "versions": [
        {
          "groupVersion": "apps/v1",
          "version": "v1"
        }
      ],
      "preferredVersion": {
        "groupVersion": "apps/v1",
        "version": "v1"
      }
    }
  ]
}
.....
#分组API
[root@master1 ~]#curl 127.0.0.1:8081/api/
{
  "kind": "APIVersions",
  "versions": [
    "v1"
  ],
  "serverAddressByClientCIDRs": [
    {
      "clientCIDR": "0.0.0.0/0",
      "serverAddress": "10.0.0.101:6443"
    }
  ]
}

#使用 jq 命令(json 数据处理的命令行工具)处理结果:
[root@master1 ~]#curl -s 127.0.0.1:8081/api/ | jq .kind
"APIVersions"

[root@master1 ~]#curl -s 127.0.0.1:8081/api/ | jq .serverAddressByClientCIDRs
[
  {
    "clientCIDR": "0.0.0.0/0",
    "serverAddress": "10.0.0.103:6443"
  }
]

#查看某个 Namespace 资源情况
[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq
{
  "kind": "Namespace",
  "apiVersion": "v1",
  "metadata": {
    "name": "default",
    "uid": "7a644f94-8b3c-400d-bc9c-e202f8560e04",
    "resourceVersion": "200",
    "creationTimestamp": "2022-02-22T12:26:26Z",
    "labels": {
      "kubernetes.io/metadata.name": "default"
    }
  },
  "managedFields": [
    {
      "manager": "kube-apiserver",
      "operation": "Update",
      "apiVersion": "v1",

```

```

        "time": "2022-02-22T12:26:26Z",
        "fieldsType": "Fieldsv1",
        "fieldsv1": {
            "f:metadata": {
                "f:labels": {
                    ".": {},
                    "f:kubernetes.io/metadata.name": {}
                }
            }
        }
    ],
    },
    "spec": {
        "finalizers": [
            "kubernetes"
        ]
    },
    "status": {
        "phase": "Active"
    }
}

```

kind字段

```

[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq .kind
"Namespace"

```

apiVersion字段

```

[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq
.apiVersion
"v1"

```

metadata字段

```

[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq
.metadata
{
  "name": "default",
  "uid": "7a644f94-8b3c-400d-bc9c-e202f8560e04",
  "resourceVersion": "200",
  "creationTimestamp": "2022-02-22T12:26:26Z",
  "labels": {
    "kubernetes.io/metadata.name": "default"
  },
  "managedFields": [
    {
      "manager": "kube-apiserver",
      "operation": "Update",
      "apiVersion": "v1",
      "time": "2022-02-22T12:26:26Z",
      "fieldsType": "Fieldsv1",
      "fieldsv1": {
        "f:metadata": {
          "f:labels": {
            ".": {},
            "f:kubernetes.io/metadata.name": {}
          }
        }
      }
    }
  ]
}

```

```

    ]
  }

  # spec字段
[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq .spec
{
  "finalizers": [
    "kubernetes"
  ]
}

# status字段
[root@master1 ~]#curl -s 127.0.0.1:8081/api/v1/namespaces/default | jq .status
{
  "phase": "Active"
}

#如上面的属于 Namespace 类型的名为 default 的资源信息，其表现为一个 JSON 序列化的数据，
kubernetes 大多数资源对象的配置格式都为 JSON 格式，具有 apiVersion、kind、metadata、spec
和status 五个一级字段。

```

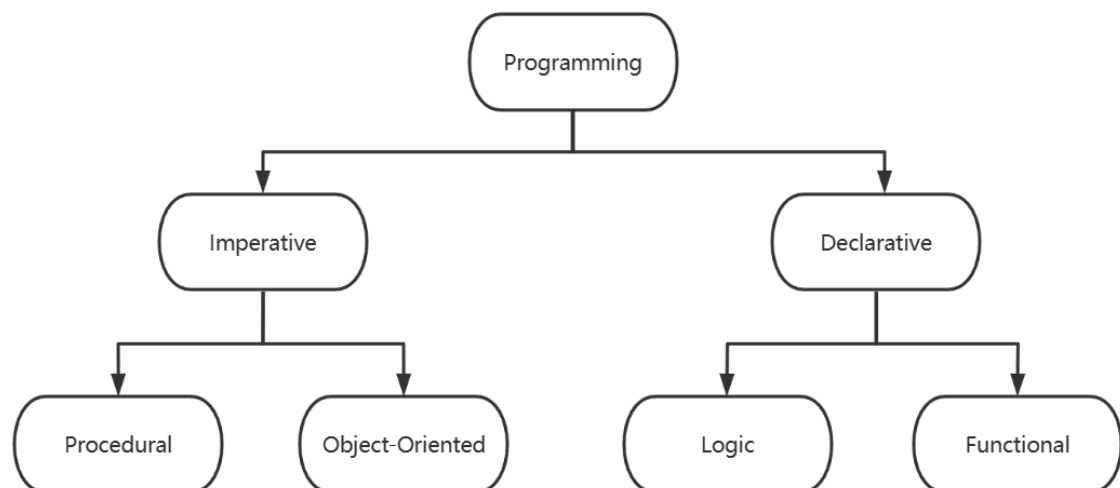
1.1.4 Kubernetes API 设计理念

<https://www.kubernetes.org.cn/kubernetes设计理念>

Kubernetes设计理念与分布式系统

分析和理解Kubernetes的设计理念可以使我们更深入地了解Kubernetes系统，更好地利用它管理分布式部署的云原生应用，另一方面也可以让我们借鉴其在分布式系统设计方面的经验。

API 设计方法



- 命令式API

也称为指令式API，用户需要一步步地告诉机器该如何做(How)，机器自身不具有任何“智能”，只被动接受指令

高度依赖用户自身理解和达成目标的能力和各类异常问题的经验，实现的是“命令式编程 Imperative Programming

- 声明式API

也称为申明式API，用户只需要告诉机器想要的结果(What)，机器自身需要确定如何达成该目标 机器需要一定的“智能”，但通常只能支持事先预设和可被其理解的特定任务

实现的是“声明式编程 Declarative Programming

相较于命令式编程，声明式编程是一个更高的层次上的编程

声明式API允许用户以给出最终期望目标的方式编写代码，但具体的执行过程（即机器智能的那部分代码），最终仍然需要以命令式编程实现，只不过，它们可由不同的人群完成

类比来说，声明式编程的用户类似于企业的高管，只关心心和交待最终目标;而命令式编程的用户类似于企业部门经理，他需要理解目标的达成路径，并组织人力完成目标

API 设计原则

对于云计算系统，系统API实际上处于系统设计的统领地位，K8s集群系统每支持一项新功能，引入一项新技术，一定会新引入对应的API对象，支持对该功能的管理操作，K8s系统API的设计有以下几条原则：

1. **所有API应该是声明式的。**正如前文所说，声明式的操作，相对于命令式操作，对于重复操作的效果是稳定的，这对于容易出现数据丢失或重复的分布式环境来说是很重要的。另外，声明式操作更容易被用户使用，可以使系统向用户隐藏实现的细节，隐藏实现的细节的同时，也就保留了系统未来持续优化的可能性。此外，声明式的API，同时隐含了所有的API对象都是名词性质的，例如 Service、Volume 这些API都是名词，这些名词描述了用户所期望得到的一个目标分布式对象。
2. **API对象是彼此互补而且可组合的。**这里面实际是鼓励API对象尽量实现面向对象设计时的要求，即“高内聚，松耦合”，对业务相关的概念有一个合适的分解，提高分解出来的对象的可重用性。事实上，K8s这种分布式系统管理平台，也是一种业务系统，只不过它的业务就是调度和管理容器服务。
3. **高层API以操作意图为基础设计。**如何能够设计好API，跟如何能用面向对象的方法设计好应用系统有相通的地方，高层设计一定是从业务出发，而不是过早的从技术实现出发。因此，针对K8s的高层API设计，一定是以K8s的业务为基础出发，也就是以系统调度管理容器的操作意图为基础设计。
4. **低层API根据高层API的控制需要设计。**设计实现低层API的目的，是为了被高层API使用，考虑减少冗余、提高重用性的目的，低层API的设计也要以需求为基础，要尽量抵抗受技术实现影响的诱惑。
5. **尽量避免简单封装，不要有在外部API无法显式知道的内部隐藏的机制。**简单的封装，实际没有提供新的功能，反而增加了对所封装API的依赖性。内部隐藏的机制也是非常不利于系统维护的设计方式，例如PetSet和ReplicaSet，本来就是两种Pod集合，那么K8s就用不同API对象来定义它们，而不会说只用同一个ReplicaSet，内部通过特殊的算法再来区分这个ReplicaSet是有状态的还是无状态。
6. **API操作复杂度与对象数量成正比。**这一条主要是从系统性能角度考虑，要保证整个系统随着系统规模的扩大，性能不会迅速变慢到无法使用，那么最低的限定就是API的操作复杂度不能超过 $O(N)$ ，N是对象的数量，否则系统就不具备水平伸缩性了。
7. **API对象状态不能依赖于网络连接状态。**由于众所周知，在分布式环境下，网络连接断开是经常发生的事情，因此要保证API对象状态能应对网络的不稳定，API对象的状态就不能依赖于网络连接状态。
8. **尽量避免让操作机制依赖于全局状态，因为在分布式系统中要保证全局状态的同步是非常困难的。**

1.2 资源清单格式

Kubernetes API 只接受及响应 JSON 格式的数据对象。同时，为了使用方便，它允许用户提供 YAML 格式的POST 对象，由 API server 事先自行将其转换为 JSON 格式后提交。API server 接受和返回的所有 JSON对象都遵循同一个模式，它们都具有 kind 和 apiVersion 字段，表示了对象所属的资源类型和 API 群组及版本。

大部分的对象或类别类型的资源还需要有三个嵌套的字段：metadata、spec 和 status 字段。

- metadata为资源提供元数据，如名称、隶属的名称空间和标签等；
- spec 字段则用于定义用户期望的状态，不同的资源类型，其状态的意义也各有不同，如 Pod 资源最为核心的功能在于运行容器；

- status 字段则记录着活动对象的当前状态信息，它由 Kubernetes 系统自行维护，对用户来说为只读字段。

在Kubernetes集群中，资源对象产生流程

- 根据业务应用架构的分析，确定我们要使用的资源对象(Kubernetes中的)
- 使用描述性语言，编写资源对象的定义文件
- 基于资源对象定义文件进行对象初始化，形成资源对象。

资源定义文件：

- 用来初始化kubernetes资源对象的
- 定义文件的格式根据其编写语言的种类，一般分为两种：YAML和json，
- 工作中我们常用的就是YAML。

1.2.1 YAML格式说明

YAML样式格式特点

- 大小写敏感
- 使用缩进表示层级关系
- 缩进时不允许使用Tab键，只允许使用空格。
- 缩进的空格数目不重要，只要相同层级的元素左侧对齐即可
- 以#开始的内容表示注释，从这个字符一直到行尾，都会被解析器忽略

数据格式

YAML是一种对象描述性语言，它有两种基本样式：Maps和Lists。

- 基本样式一：YAML Maps

```
#就是一个键值对，键和值中间使用"冒号+空格"隔开。
#格式1:
key: value

#示例:
app: mysql

#这种基本样式，还有一种变种形式，就是值的本身就是一个键值对，两层之间使用两个空格表示层级关系
#格式2:
key:
  key: value

#示例:
metadata:
  name: mysql
```

- 基本样式二：YAML Lists

其实这也是一种键值对，只不过它的值是一个列表形式。

特点：

- 1、键和冒号为一行，值为下一行
- 2、键和值有层级关系，使用两个空格表示层级关系
- 3、列表多元素之间是同级关系，使用 - 表示

格式：

```
key:
- value1
- value2
```

```
- value3
```

#示例:

args:

```
- sleep
- "1000"
- message
```

Kubernetes的资源定义文件就是在这两个基本样式的基础上组合起来的

基本上无论你想要什么样的文件结构，你都可以通过Maps和Lists去组合实现。

如果不确定yaml文件的语法是否正确，可以到<http://www.yamllint.com/>或其它网站上检测一下。

1.2.2 资源配置清单介绍

资源配置清单的格式采用 Yaml 格式

第一级字段名一般包括: apiVersion、kind、metadata、spec、status 五个字段

字段名采有小驼峰命名法,而值一般采用大驼峰命令法

每一个字段的值一般为如下类型

- string
- integer
- boolean
- Object: 对象
- []Object: 对象数组

K8s 系统上的绝大多数资源都是由其使用者所创建的。创建时先要使用 yaml 或者 json格式定义资源相关的配置数据，即用户期望的目标状态，而后在有 K8s 的底层组件确保活动对象的运行时状态与用户提供的配置清单中定义的状态无限接近。

在配置资源清单时一般使用 yaml 格式

几乎所有的资源，apiVersion、kind 和 metadata 字段的功能基本相同。但是 spec字段则可能有很大的不同

spec 用于规定资源的期望状态，而资源的嵌套属性是有很大差别的。

status字段则记录活动对象的当前状态，其要与 spec 中定义的状态相同，或者处于正转换为与其相同的过程中。

用户可以使用 `kubectl get TYPE/NAME -o yaml/json` 命令来获取任何一个对象的yaml 或者 json 格式的配置清单

如获取名称空间类型 kube-system 的配置清单：

```
[root@master1 ~]#kubectl get namespace kube-system -o yaml
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: "2020-02-22T07:56:11Z"
  labels:
    kubernetes.io/metadata.name: kube-system
  name: kube-system
  resourceVersion: "7"
  uid: a176ab46-ab7b-4737-ab52-2e53ca1d1d46
spec:
```

```

finalizers:
- kubernetes
status:
phase: Active

[root@master1 ~]#kubectl get namespace kube-system -o json
{
  "apiVersion": "v1",
  "kind": "Namespace",
  "metadata": {
    "creationTimestamp": "2020-02-22T07:56:11Z",
    "labels": {
      "kubernetes.io/metadata.name": "kube-system"
    },
    "name": "kube-system",
    "resourceVersion": "7",
    "uid": "a176ab46-ab7b-4737-ab52-2e53ca1d1d46"
  },
  "spec": {
    "finalizers": [
      "kubernetes"
    ]
  },
  "status": {
    "phase": "Active"
  }
}

```

1.2.3 apiVersion和kind

apiVersion和kind 描述类型的元数据

- apiVersion: API版本,用于对同一资源对象的不同版本进行并行管理,主要有 alpha、beta、stable
格式: 组名/版本
查看命令: **kubectl api-versions**, 可以看到当前共有27+个分组和版本
- kind: 资源类型,kubernetes的专用资源对象
查看命令: **kubectl api-resources [--api-group=]**, 可以看到当前共有50+种资源对象和对应的APIVERSION 版本信息

范例: 查看API版本

```

[root@master1 ~]#kubectl api-versions
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1

```

```

events.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1beta2
flowcontrol.apiserver.k8s.io/v1beta3
networking.k8s.io/v1
node.k8s.io/v1
policy/v1
rbac.authorization.k8s.io/v1
scheduling.k8s.io/v1
storage.k8s.io/v1
storage.k8s.io/v1beta1
v1

```

范例: 查看资源类型

```

[root@master1 ~]#kubectl api-resources
NAME                                SHORTNAMES  APIVERSION
NAMESPACEED  KIND
bindings                                           v1
  true      Binding
componentstatuses  cs          v1
  false     ComponentStatus
configmaps        cm          v1
  true      ConfigMap
endpoints         ep          v1
  true      Endpoints
events           ev          v1
  true      Event
limitranges      limits     v1
  true      LimitRange
namespaces       ns          v1
  false     Namespace
nodes            no          v1
  false     Node
persistentvolumeclaims  pvc        v1
  true      PersistentVolumeClaim
persistentvolumes  pv         v1
  false     PersistentVolume
pods             po          v1
  true      Pod
podtemplates     v1
  true      PodTemplate
replicationcontrollers  rc          v1
  true      ReplicationController
resourcequotas      quota      v1
  true      ResourceQuota
secrets           v1
  true      Secret
serviceaccounts    sa          v1
  true      ServiceAccount
services          svc         v1
  true      Service
mutatingwebhookconfigurations  admissionregistration.k8s.io/v1
  false     MutatingWebhookConfiguration
validatingwebhookconfigurations  admissionregistration.k8s.io/v1
  false     ValidatingWebhookConfiguration
customresourcedefinitions  crd,crds   apiextensions.k8s.io/v1
  false     CustomResourceDefinition

```

apiservices			apiregistration.k8s.io/v1
false	APIService		
controllerrevisions			apps/v1
true	ControllerRevision		
daemonsets		ds	apps/v1
true	DaemonSet		
deployments		deploy	apps/v1
true	Deployment		
replicasets		rs	apps/v1
true	ReplicaSet		
statefulsets		sts	apps/v1
true	StatefulSet		
tokenreviews			authentication.k8s.io/v1
false	TokenReview		
localsubjectaccessreviews			authorization.k8s.io/v1
true	LocalSubjectAccessReview		
selfsubjectaccessreviews			authorization.k8s.io/v1
false	SelfSubjectAccessReview		
selfsubjectrulesreviews			authorization.k8s.io/v1
false	SelfSubjectRulesReview		
subjectaccessreviews			authorization.k8s.io/v1
false	SubjectAccessReview		
horizontalpodautoscalers		hpa	autoscaling/v2
true	HorizontalPodAutoscaler		
cronjobs		cj	batch/v1
true	CronJob		
jobs			batch/v1
true	Job		
certificatesigningrequests		csr	certificates.k8s.io/v1
false	CertificateSigningRequest		
leases			coordination.k8s.io/v1
true	Lease		
endpointslices			discovery.k8s.io/v1
true	EndpointSlice		
events		ev	events.k8s.io/v1
true	Event		
flowschemas			
flowcontrol.apiserver.k8s.io/v1beta3	false	FlowSchema	
prioritylevelconfigurations			
flowcontrol.apiserver.k8s.io/v1beta3	false	PriorityLevelConfiguration	
ingressclasses			networking.k8s.io/v1
false	IngressClass		
ingresses		ing	networking.k8s.io/v1
true	Ingress		
networkpolicies		netpol	networking.k8s.io/v1
true	NetworkPolicy		
runtimeclasses			node.k8s.io/v1
false	RuntimeClass		
poddisruptionbudgets		pdb	policy/v1
true	PodDisruptionBudget		
clusterrolebindings			rbac.authorization.k8s.io/v1
false	ClusterRoleBinding		
clusterroles			rbac.authorization.k8s.io/v1
false	ClusterRole		
rolebindings			rbac.authorization.k8s.io/v1
true	RoleBinding		
roles			rbac.authorization.k8s.io/v1
true	Role		

```

priorityclasses          pc          scheduling.k8s.io/v1
  false          PriorityClass
csidrivers              storage.k8s.io/v1
  false          CSIDriver
csinodes              storage.k8s.io/v1
  false          CSINode
csistoragecapacities    storage.k8s.io/v1
  true          CSISStorageCapacity
storageclasses          sc          storage.k8s.io/v1
  false          StorageClass
volumeattachments       storage.k8s.io/v1
  false          VolumeAttachment

#查看指定API Group的资源
[root@ubuntu2204 ~]#kubectl api-resources --api-group apps
NAME                SHORTNAMES  APIVERSION  NAMESPACED  KIND
controllerrevisions  apps/v1    apps/v1     true        ControllerRevision
daemonsets           ds          apps/v1     true        DaemonSet
deployments          deploy     apps/v1     true        Deployment
replicasets          rs          apps/v1     true        ReplicaSet
statefulsets         sts        apps/v1     true        StatefulSet

[root@master1 ~]#kubectl api-resources --api-group=storage.k8s.io
NAME                SHORTNAMES  APIVERSION  NAMESPACED  KIND
csidrivers          storage.k8s.io/v1  false      CSIDriver
csinodes            storage.k8s.io/v1  false      CSINode
csistoragecapacities  storage.k8s.io/v1  true
CSISStorageCapacity
storageclasses      sc          storage.k8s.io/v1  false
StorageClass
volumeattachments   storage.k8s.io/v1  false
VolumeAttachment

```

1.2.4 metadata 嵌套字段

metadata 字段用于描述对象的元数据,即属性信息, 其内嵌多个用于定义资源的元数据, 如 name 和 labels 等。这些字段可以分为必选字段和可选字段

必选字段:

- name: 设定当前对象的名称, 名称空间级资源在其所属的名称空间的同一类型中必须唯一
- namespace: 指定当前对象隶属的名称空间, 默认值为 default, 实现资源隔离
- uid: 当前对象的唯一标识符, 用于区别"已删除"和"重新创建"的同一个名称的对象,系统可以自动生成

可选字段通常是指有 Kubernetes 系统自行维护和设置, 或者存在默认, 或允许使用空值的字段。

有以下可选字段:

- labels: 设定用于标识当前对象的标签, 键值数据, 格式: key1: value1 ,常用作标签选择器的挑选条件
- annotation: 非标识型键值数据, 格式: key1: value1,用来作为挑选条件, 用于 labels 的补充, 不支持标签选择器的选择
- resourceVersion: 当前对象的内部版本标识, 用来让客户端确定对象的变动与否
- generation: 标识当前对象目标状态的代别
- creationTimestamp: 当前对象创建日期的时间戳

- deletionTimestamp: 当前对象删除日期的时间戳

1.2.5 spec 和 status 字段

定义资源配置清单时，spec 是必须的字段。用于描述对象的目标状态，也就是用户期望对象所表现出来的特征。

- spec 字段

Specification 规格字段

此字段对于不同的对象类型来说各不相同，具体字段含义及所接受的数据类型需要参照 Kubernetes API 手册中的说明进行获取。可通过命令 `kubectl explain KIND.spec` 获取具体帮助

- status 字段

此字段记录对象的当前实际运行的状态，由 Kubernetes 系统负责更新，用户不能手动定义。

Master 节点的 controller manager 通过相应的控制器组件动态管理并确保对象的实际状态匹配用户所期望的状态。比如:Deployment 是一种描述集群中运行应用的资源对象，因此，创建 Deployment 类型对象时，需要为目标 Deployment 对象设定 spec，指定期望需要运行的 Pod 副本数量、使用的标签选择器以及 Pod 模板等。在创建时，Kubernetes 相关组件读取待创建的 Deployment 对象的 spec 以及系统上相应的活动对象的当前状态，必要时对活动的对象更新以确保 status 字段吻合 spec 字段中期望的状态。

注意：数据类的资源对象无 spec, Status 字段，比如：configmaps, secrets, endpoints 等

注意：字段名一般为小驼峰命名法，字段值为大驼峰命名法

```
[root@master1 ~]#kubectl explain pod.spec.containers.imagePullPolicy
KIND:      Pod
VERSION:   v1

FIELD:     imagePullPolicy <string>

DESCRIPTION:
  Image pull policy. One of Always, Never, IfNotPresent. Defaults to Always
  if :latest tag is specified, or IfNotPresent otherwise. Cannot be updated.
  More info:
  https://kubernetes.io/docs/concepts/containers/images#updating-images
```

1.2.6 资源配置清单格式文档帮助

官方帮助

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.22/>

通过 kubectl explain 查看帮助生成新的清单文件

定义资源配置清单时，apiVersion、kind 和 metadata 都大同小异，但是 spec 字段对不同的资源来说却是差别迥异。因此，用户在编写资源清单时要参考 Kubernetes API 的参考文档来了解各种可用属性字段。在 K8s 系统中内建了清单各个属性字段含义及所接受的数据类型等相关文档，用户可以使用 `kubectl explain` 命令来随时获取使用帮助

可以使用 `kubectl api-resources` 查看哪些资源可以使用 explain 来获取文档帮助

格式

```
kubectl explain TYPE_NAME [--api-version=GROUP_NAME/VERSION]
```

范例:

```
[root@master1 ~]# kubectl api-resources | awk '{print $1}'
NAME
bindings
componentstatuses
configmaps
endpoints
events
limitranges
namespaces
nodes
persistentvolumeclaims
persistentvolumes
pods
podtemplates
replicationcontrollers
resourcequotas
secrets
serviceaccounts
services
mutatingwebhookconfigurations
validatingwebhookconfigurations
customresourcedefinitions
apiservices
controllerrevisions
daemonsets
deployments
replicasets
statefulsets
tokenreviews
localsubjectaccessreviews
selfsubjectaccessreviews
selfsubjectrulesreviews
subjectaccessreviews
horizontalpodautoscalers
cronjobs
jobs
certificatesigningrequests
leases
endpointslices
events
flowschemas
prioritylevelconfigurations
ingressclasses
ingresses
networkpolicies
runtimeclasses
poddisruptionbudgets
podsecuritypolicies
clusterrolebindings
clusterroles
rolebindings
roles
```



```
priorityclasses
csidrivers
csinodes
csistoragecapacities
storageclasses
volumeattachments
```

kubectl explain 可以根据给出的对象类型或者相应的嵌套字段来显示该字段所接受的下一级字段相关文档。

```
#查看 Pod 资源的一级字段文档帮助,下面两种方式功能相同
[root@master1 ~]#kubectl explain pods
[root@master1 ~]#kubectl explain pods.
KIND:      Pod
VERSION:   v1

DESCRIPTION:
    Pod is a collection of containers that can run on a host. This resource is
    created by clients and scheduled onto hosts.

FIELDS:   # pods可以接受的二级字段
    apiVersion    <string>
    ...
    kind <string>
    ...
    metadata      <Object>
    ...
    spec <Object>
    ...
    status        <Object>
    ...
```

需要了解某一级字段表示的对象之下级对象字段,则只需要使用一个英文的句点指定其下级字段的对象名称即可

```
#查看pods的指定的字段的说明
[root@master1 ~]#kubectl explain pod.apiVersion
KIND:      Pod
VERSION:   v1

FIELD:     apiVersion <string>

DESCRIPTION:
    APIVersion defines the versioned schema of this representation of an
    object. Servers should convert recognized schemas to the latest internal
    value, and may reject unrecognized values. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-
    conventions.md#resources

# pods.spec字段下可接受下面的字段
[root@master1 ~]#kubectl explain pods.spec | grep -E "^\{,3\}[a-z]"
    activeDeadlineSeconds    <integer>
    affinity <Object>
    automountServiceAccountToken <boolean>
    containers <[]Object> -required-
```

```

    dnsConfig      <Object>
    dnsPolicy      <string>
    enableServiceLinks <boolean>
    ephemeralContainers <[]Object>
    .....

#依次查看编写资源清单所需的字段及其所需的数据类型
[root@master1 ~]# kubectl explain pods.spec.containers.ports.containerPort
KIND:      Pod
VERSION:   v1

FIELD:     containerPort <integer>

DESCRIPTION:
    Number of port to expose on the pod's IP address. This must be a valid port
    number, 0 < x < 65536.

```

范例：查看不同版本的资源

```

[root@ubuntu2204 ~]# kubectl api-versions
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1
events.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1
flowcontrol.apiserver.k8s.io/v1beta3
networking.k8s.io/v1
node.k8s.io/v1
policy/v1
rbac.authorization.k8s.io/v1
scheduling.k8s.io/v1
storage.k8s.io/v1
v1

[root@ubuntu2204 ~]# kubectl api-resources |grep hpa
horizontalpodautoscalers      hpa      autoscaling/v2
true      HorizontalPodAutoscaler

[root@ubuntu2204 ~]# kubectl explain hpa --api-version=autoscaling/v1
GROUP:      autoscaling
KIND:       HorizontalPodAutoscaler
VERSION:    v1

DESCRIPTION:
    configuration of a horizontal pod autoscaler.

FIELDS:
    apiVersion      <string>

```

APIVersion defines the versioned schema of this representation of an object. Servers should convert recognized schemas to the latest internal value, and may reject unrecognized values. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources>

kind <string>
kind is a string value representing the REST resource this object represents. Servers may infer this from the endpoint the client submits requests to. Cannot be updated. In CamelCase. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds>

metadata <ObjectMeta>
Standard object metadata. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata>

spec <HorizontalPodAutoscalerSpec>
spec defines the behaviour of autoscaler. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>.

status <HorizontalPodAutoscalerStatus>
status is the current information about the autoscaler.

```
[root@ubuntu2204 ~]#kubectl explain hpa --api-version=autoscaling/v2
```

```
GROUP:      autoscaling
KIND:       HorizontalPodAutoscaler
VERSION:    v2
```

DESCRIPTION:

HorizontalPodAutoscaler is the configuration for a horizontal pod autoscaler, which automatically manages the replica count of any resource implementing the scale subresource based on the metrics specified.

FIELDS:

apiVersion <string>
APIVersion defines the versioned schema of this representation of an object. Servers should convert recognized schemas to the latest internal value, and may reject unrecognized values. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources>

kind <string>
kind is a string value representing the REST resource this object represents. Servers may infer this from the endpoint the client submits requests to. Cannot be updated. In CamelCase. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds>

metadata <ObjectMeta>
metadata is the standard object metadata. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata>

spec <HorizontalPodAutoscalerSpec>

```
spec is the specification for the behaviour of the autoscaler. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#spec-and-status.
```

```
status      <HorizontalPodAutoscalerStatus>
status is the current information about the autoscaler.
```

1.3 创建资源清单文件

1.3.1 利用dry-run模拟运行指令式命令生成清单文件

```
[root@master1 ~]#kubectl create namespace namespace-test --dry-run=client -o
yaml
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: null
  name: namespace-test
spec: {}
status: {}

#生成文件
[root@master1 ~]#kubectl create namespace namespace-test --dry-run=client -o yaml
> namespace-test.yaml
[root@master1 ~]#vim namespace-test.yaml
```

1.3.2 利用现有资源生成清单文件再修改为新的清单文件

每次kubectl explain查看帮助过于繁琐,可以根据当前系统已有的活动资源来导出资源清单模板, 基于此模板再做修改可以更快速的使用资源。

格式

```
kubectl get TYPE NAME -o yaml > /path/export-resourcelist.yaml
kubectl get TYPE/NAME -o yaml > /path/export-resourcelist.yaml
```

范例: 导出指定资源的模版文件

```
[root@master1 ~]#kubectl get pods pod-test1 -o yaml > pod-test1.yaml
```

1.3.3 将 Docker Compose 文件转换为 Kubernetes 资源

```
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/translate-compose-
kubernetes/
```

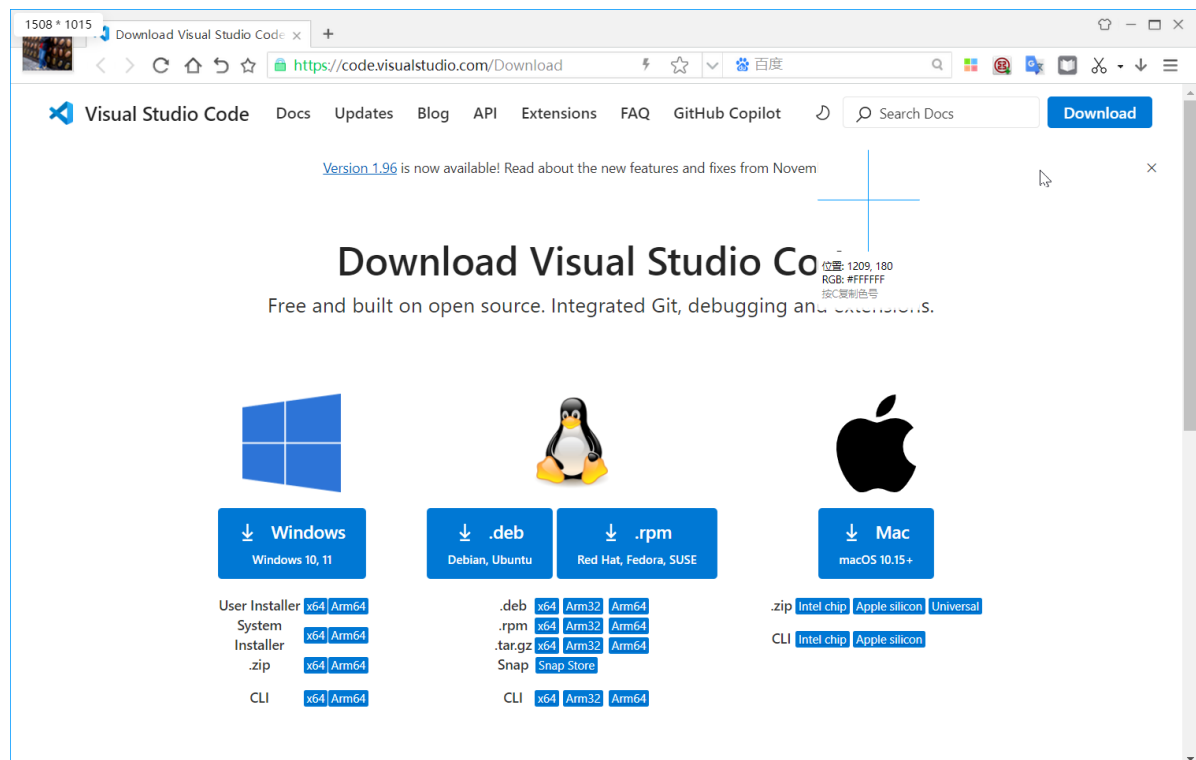
Kompose 是一个转换工具, 可将 compose (即 Docker Compose) 所组装的所有内容转换成容器编排器 (Kubernetes 或 OpenShift) 可识别的形式。

Kompose 官网 <http://kompose.io>。

1.3.4 通过 Visual Studio Code 自动生成 YAML文件

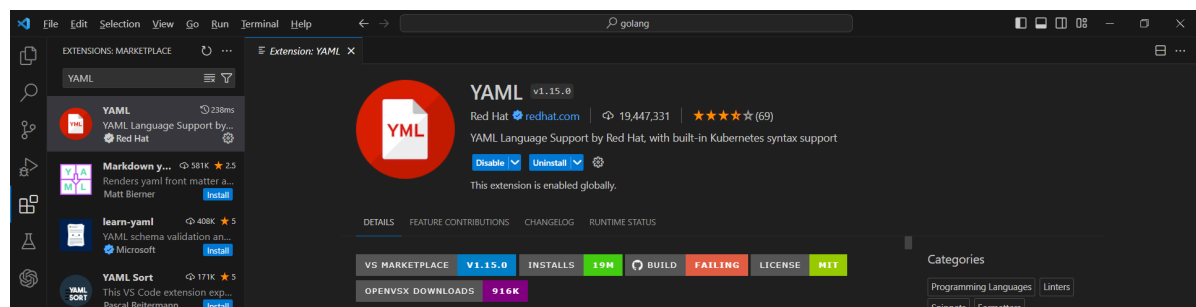
1.3.4.1 安装 Visual Studio Code

<https://code.visualstudio.com/download>
<https://visualstudio.microsoft.com/zh-hans/>



1.3.4.2 安装扩展 YAML (redhat)

YAML
#版本: v1.15.0



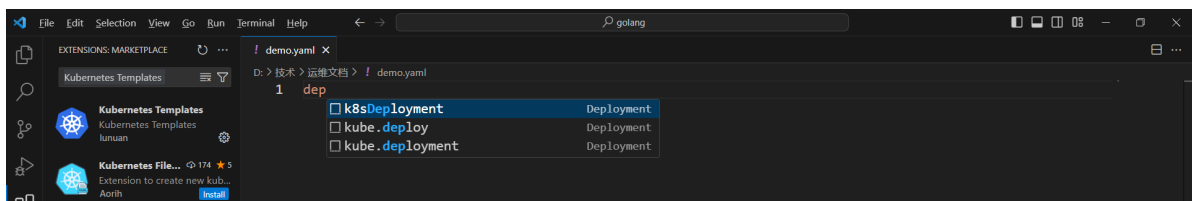
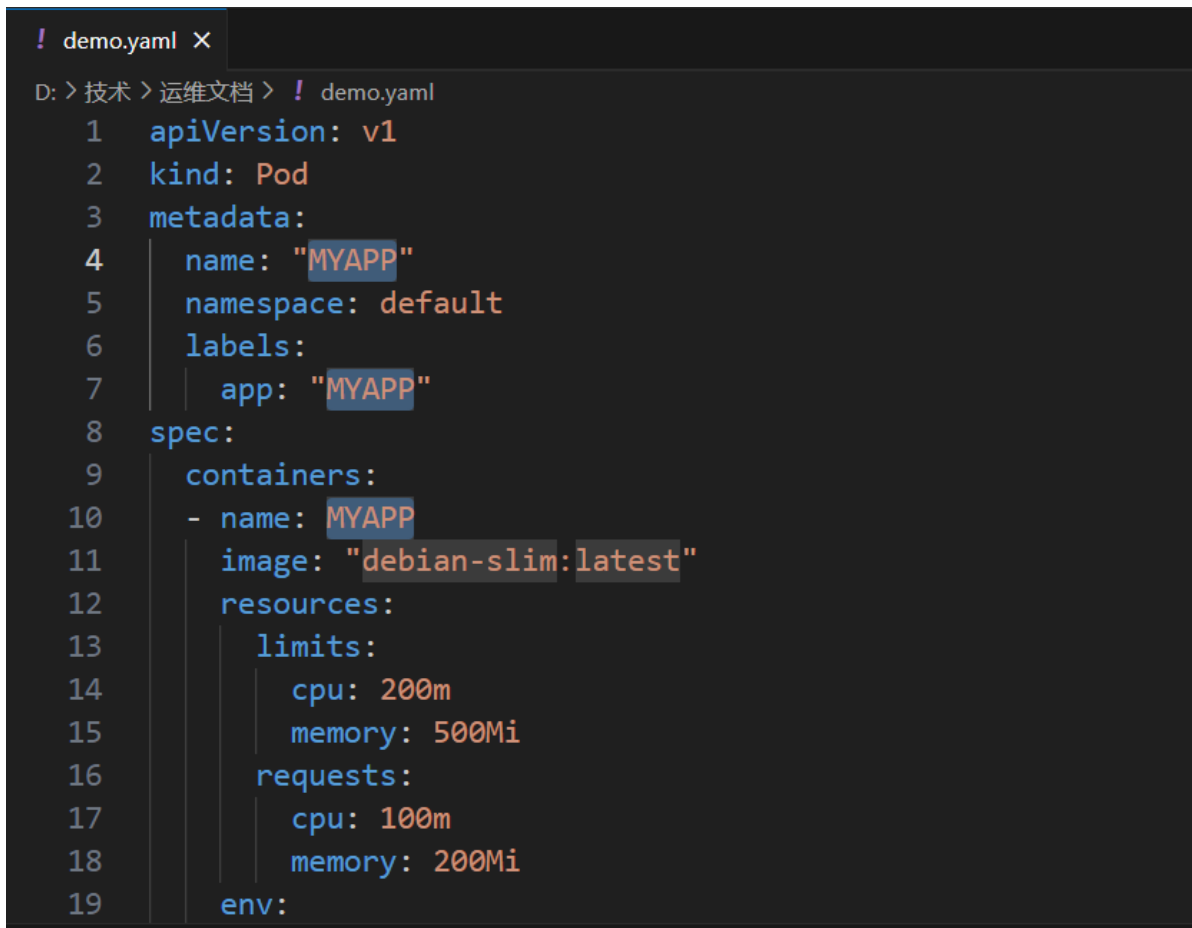
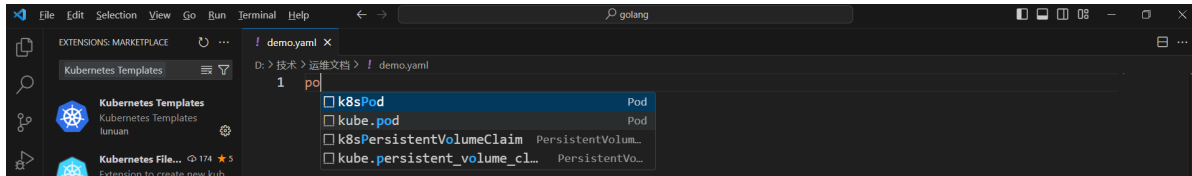
1.3.4.3 安装扩展 Kubernetes Templates

Kubernetes Templates
#版本: v1.3.1



1.3.4.4 创建 YAML后缀的文件

创建文件后,在内容器输入相创建的资源类型,选中后自动生成YAML模板



```
terminal Help ← → golang
! demo.yaml x
D: > 技术 > 运维文档 > ! demo.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: MYAPP
5    namespace: default
6    labels:
7      app: MYAPP
8  spec:
9    selector:
10     matchLabels:
11       app: MYAPP
12     replicas: 1
13     strategy:
14       rollingUpdate:
15         maxSurge: 25%
16         maxUnavailable: 25%
17       type: RollingUpdate
18     template:
19       metadata:
```

1.4 资源对象的管理方式

Kubernetes 的 API server 遵循声明式编程范式(declarative programming)而设计，侧重于构建程序逻辑而无需用户描述实现其的步骤和流程，用户只需要设定期望的状态，系统就会自行确定需要执行的操作以确保达到用户期望的状态。

例如，期望 Deployment 控制器管理三个 Pod 资源对象时，而系统观察到的当前数量却是两个，于是系统就会自行创建一个新的 Pod 来满足此预期。Kubernetes 的自愈、自治等功能都依赖于其声明式机制。

另外，Kubernetes 的 API server 也支持指令式范式(imperative programming)，直接通过命令及其选项完成对象的管理操作，如 `kubectl run/expose/delete/get` 等命令都属于此类命令。例如，使用 `kubectl run` 创建一个有着 3 个 pod 副本的 Deployment 对象，或者通过 `kubectl delete` 命令删除一个名为 `nginx-service` 的 Service 对象。

Kubernetes 系统通过控制器监控着系统对象，由其负责让系统当前的状态无限接近用户所期望的状态

kubectl 命令可分为三类命令：

- 指令式命令(imperative command):

指令式命令包括 `kubectl run/expose/delete/get` 等命令

适合完成一次性的操作任务

- 指令式对象配置(imperative object configuration):

指令式对象配置管理包括 `kubectl create/delete/get/replace/edit` 等

基于资源配置文件执行对象管理操作，但只能独立引用每个配置清单文件

指令式管理配置与指令式命令的区别是，其需要用户提供资源清单文件，通过资源清单来读取需要管理的目标资源对象。指令式对象配置管理的管理操作直接作用于活动对象，会导致用户丢失活动对象的当前状态。

此方式没有幂等性,重复执行可能会出错,生产不推荐使用

- 声明式对象配置(declarative object configuration):

基于资源配置文件执行对象管理操作

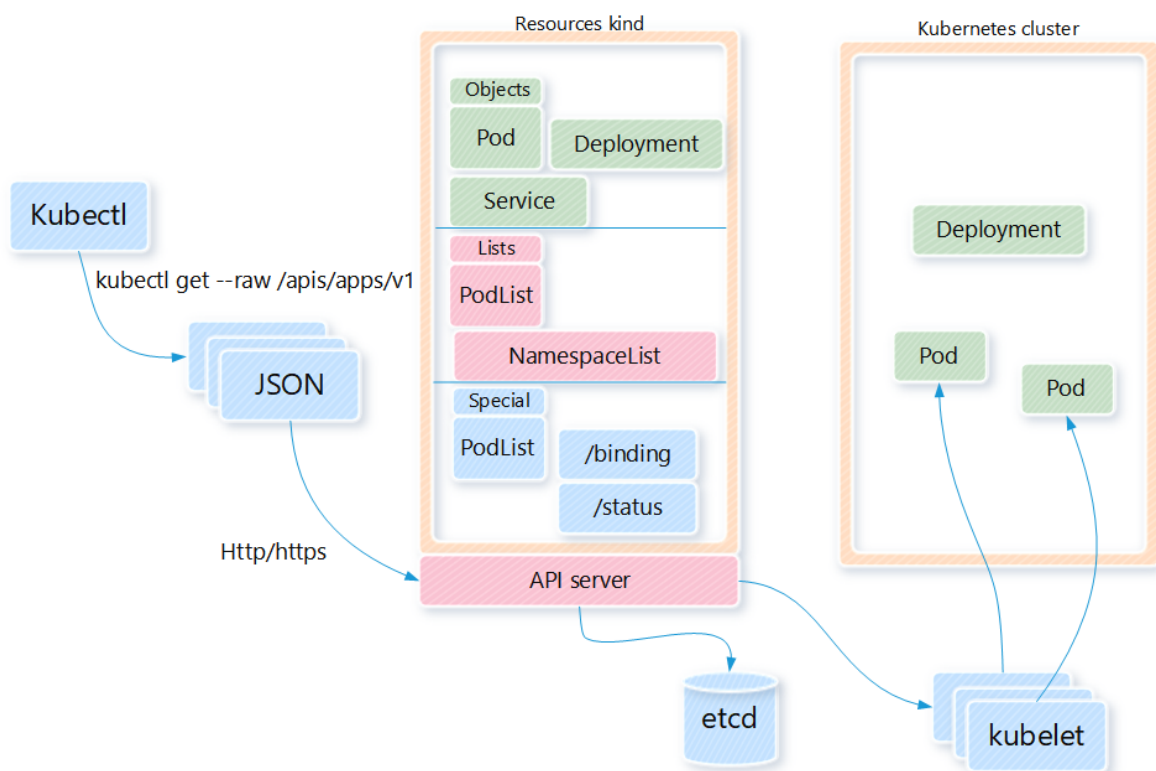
可直接引用目录下的所有配置清单文件，也可直接作用于单个配置文件

声明式对象配置并不直接指明要进行的对象管理操作，而是提供配置清单文件给 Kubernetes 系统，并委托系统跟踪活动对象的状态变动。

资源对象的创建、删除及修改操作全部通过命令**kubectl apply/patch**等来完成，并且每次操作时，提供给命令的配置信息都将保存于对象的注释信息(kubectl.kubernetes.io/last-applied-configuration)中，并且通过对比检查活动对象的当前状态、注解中的配置信息和资源清单中的配置三方的变更来实现仅修改变动字段的高级补丁机制。

```
kubectl apply -f /path/file -f ..... #加载指定文件
kubectl apply -f /path #加载指定目录下的所有以.yaml, .yml, .json后缀的文件
kubectl apply -f URL #加载URL的文件
```

此方式有幂等性,生产推荐使用此方式



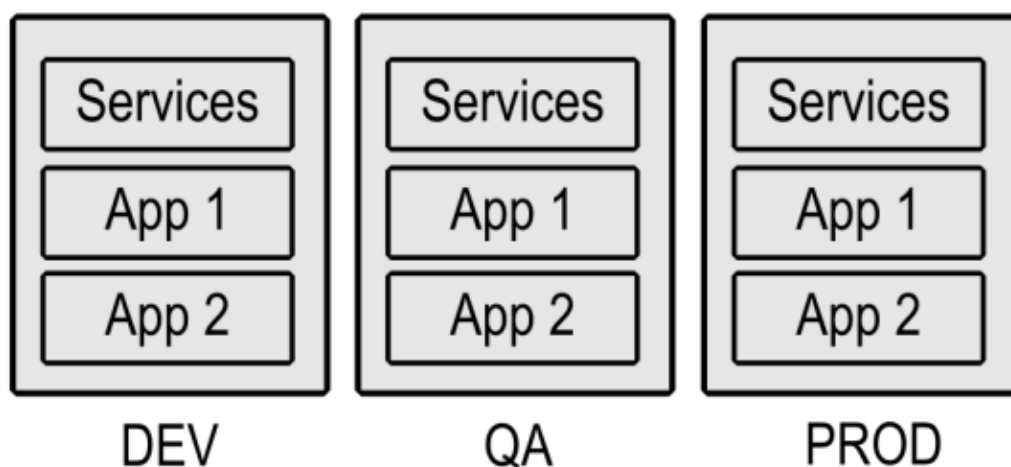
2 名称空间

2.1 名称空间说明

<https://kubernetes.io/docs/tasks/administer-cluster/namespaces/>

KUBERNETES CLUSTER

NAMESPACES



Kubernetes 的资源工作的有效范围分成两种级别:

- 集群级别: 针对整个Kubernetes集群内都有效
- 名称空间级别: 只针对指定名称空间内有效,而不属于任务名称空间

名称空间 Namespace 用于将集群分隔为多个隔离的逻辑分区以配置给不同的用户、租户、环境或者项目使用。

名称空间限定了资源对象工作在指定的名称范围内的作用域

例如, 可以为 development、Quality Assurance和 production 应用环境分别创建各自的名称空间。

注意: 名称空间本身是 Kubernetes 集群级别的资源

Kubernetes 的大多数资源都属于名称空间级别,除此还有全局性的集群级别的资源。

也因此这些集群级别的资源对象的名称必须在集群中全局唯一

比如: Node、Namespace、clusterrole 和 PersistentVolume 等资源不属于任何名称空间

需要使用名称空间的场景

- 环境管理:
需要在同一Kubernetes集群上隔离研发、预发和生产等一类的环境时, 可以通过名称空间进行
- 隔离:
多个项目团队的不同产品线需要部署于同一Kubernetes集群时, 可以使用名称空间进行隔离
- 资源控制:
名称空间可用作资源配额的承载单位, 从而限制其内部所有应用可以使用的CPU/Memory/PV各自的资源总和
需要在产品线或团队等隔离目标上分配各自总体可用的系统资源时, 可通过名称空间实现
- 权限控制:
基于RBAC鉴权体系, 能够在名称空间级别进行权限配置
- 提高集群性能:

进行资源搜索时，名称空间有利于Kubernetes API缩小查找范围，从而对减少搜索延迟和提升性能有一定的帮助

Kubernetes的名称空间可以划分为两种类型

- 系统级名称空间:
由Kubernetes集群默认创建，主要用来隔离系统级的资源对象
所有的系统级名称空间均不能进行删除操作（即使删除也会自动重建）
除default外，其它三个系统级名称空间不应该用作业务应用的部署目标
- 自定义名称空间
由用户按需创建
比如: 根据项目和场景, 分别创建对应不同的名称空间

系统级名称空间

- default: 为任何名称空间级别的资源提供的默认的名称空间
- kube-system: Kubernetes集群自身组件及其它系统级组件使用的名称空间，Kubernetes自身的关键组件均部署在该名称空间中
- kube-public: 公众开放的名称空间，所有用户（包括Anonymous）都可以读取内部的资源,通常为
空
- kube-node-lease: 节点租约资源所用的名称空间
分布式系统通常使用“租约(Lease)”机制来锁定共享资源并协调集群成员之间的活动
Kubernetes上的租约概念由API群组coordination.k8s.io群组下的Lease资源所承载，以支撑系统级别的功能需求，例如节点心跳(node heartbeats)和组件级的领导选举等
Kubernetes集群的每个管理组件在该名称空间下都有一个与同名的lease资源对象
~# kubectl -n kube-node-lease get lease

kubernetes 和Docker的名称空间的区别

- kubernetes的名称空间, 用户根据需要可以自行创建删除,属于用户级别
- Docker的名称空间是内核内置,用户不能创建删除,属于内核级别

2.2 查看名称空间及其资源对象

使用命令 `kubectl get namespaces` 可以查看当前系统的 Namespace 类型资源

查看特定的名称空间详细信息 `kubectl describe namespace <namespace>`

查看某资源时要指明其所属的名称空间，使用 `-n` 或 `--namespace` 选项指定名称空间,如果不指定默认查看default名称空间中资源

范例:

```
[root@master1 ~]# kubectl get namespaces
NAME                STATUS    AGE
default             Active   6d18h
kube-node-lease     Active   6d18h
kube-public         Active   6d18h
kube-system         Active   6d18h
kubernetes-dashboard Active   5d1h

[root@master1 ~]# kubectl get ns -o name
namespace/default
```

namespace/kube-node-lease
namespace/kube-public
namespace/kube-system

[root@master1 ~]#kubectl get ns default -o yaml

apiVersion: v1
kind: Namespace
metadata:
 creationTimestamp: "2024-04-11T02:15:56Z"
 labels:
 kubernetes.io/metadata.name: default
 name: default
 resourceVersion: "37"
 uid: f93ff442-9482-4938-a4fb-2853e51ff4a0
spec:
 finalizers:
 - kubernetes
status:
 phase: Active

[root@master1 ~]#kubectl get ns default -o json

```
{
  "apiVersion": "v1",
  "kind": "Namespace",
  "metadata": {
    "creationTimestamp": "2024-04-11T02:15:56Z",
    "labels": {
      "kubernetes.io/metadata.name": "default"
    },
    "name": "default",
    "resourceVersion": "37",
    "uid": "f93ff442-9482-4938-a4fb-2853e51ff4a0"
  },
  "spec": {
    "finalizers": [
      "kubernetes"
    ]
  },
  "status": {
    "phase": "Active"
  }
}
```

#查看指定信息

[root@master1 ~]#kubectl get ns default -o jsonpath={.metadata.name}
default

[root@master1 ~]#kubectl get ns default -o jsonpath={.apiVersion}
v1

#查看当前名称空间的资源

[root@ubuntu2204 ~]#kubectl get all

NAME	READY	STATUS	RESTARTS	AGE
pod/myapp-6f9c7b4668-6vjv6	1/1	Running	1 (5h10m ago)	7h34m
pod/myapp-6f9c7b4668-725r9	1/1	Running	1 (5h10m ago)	7h34m
pod/myapp-6f9c7b4668-tcczj	1/1	Running	1 (5h10m ago)	7h34m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
------	------	------------	-------------	---------	-----

service/kubernetes	ClusterIP	10.68.0.1	<none>	443/TCP
7h42m				
service/myapp	NodePort	10.68.78.97	<none>	80:31890/TCP
5h18m				

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/myapp	3/3	3	3	7h34m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/myapp-6f9c7b4668	3	3	3	7h34m

#查看指定名称空间的资源

[root@ubuntu2204 ~]#kubectl get all -n kube-system

NAME	READY	STATUS	RESTARTS
AGE			
pod/calico-kube-controllers-5994f46b69-dq9pc	1/1	Running	1 (5h11m ago)
7h41m			
pod/calico-node-bvmcw	1/1	Running	1 (5h11m ago)
7h41m			
pod/calico-node-f46v4	1/1	Running	4 (5h11m ago)
7h41m			
pod/calico-node-fkdvk	1/1	Running	2 (5h11m ago)
7h41m			
pod/calico-node-g78b5	1/1	Running	2 (5h13m ago)
7h41m			
pod/calico-node-twx6t	1/1	Running	3 (5h13m ago)
7h41m			
pod/calico-node-xbt4b	1/1	Running	2 (5h12m ago)
7h41m			
pod/coredns-6db9bd4cd7-t5b4v	1/1	Running	1 (5h11m ago)
7h40m			
pod/dashboard-metrics-scraper-77b667b99d-dw4d9	1/1	Running	1 (5h11m ago)
7h40m			
pod/kubernetes-dashboard-74fb9f77fb-bhz2v	1/1	Running	2 (5h11m ago)
7h40m			
pod/metrics-server-dfb478476-lsdqx	1/1	Running	2 (5h11m ago)
7h40m			
pod/node-local-dns-5lqq2	1/1	Running	1 (5h11m ago)
7h40m			
pod/node-local-dns-6bjp4	1/1	Running	1 (5h11m ago)
7h40m			
pod/node-local-dns-hts5v	1/1	Running	1 (5h12m ago)
7h40m			
pod/node-local-dns-jxrhh	1/1	Running	1 (5h13m ago)
7h40m			
pod/node-local-dns-nmvwh	1/1	Running	1 (5h11m ago)
7h40m			
pod/node-local-dns-wv8mx	1/1	Running	1 (5h13m ago)
7h40m			

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S)			
AGE			
service/dashboard-metrics-scraper	ClusterIP	10.68.54.201	<none>
8000/TCP			
7h40m			
service/kube-dns	ClusterIP	10.68.0.2	<none>
53/UDP,53/TCP,9153/TCP			
7h40m			
service/kube-dns-upstream	ClusterIP	10.68.143.215	<none>
53/UDP,53/TCP			
7h40m			

service/kubernetes-dashboard	NodePort	10.68.50.181	<none>
443:30313/TCP	7h40m		
service/metrics-server	ClusterIP	10.68.196.91	<none>
443/TCP	7h40m		
service/node-local-dns	ClusterIP	None	<none>
9253/TCP	7h40m		

NAME	DESIRED	CURRENT	READY	UP-TO-DATE
AVAILABLE				
NODE SELECTOR	AGE			
daemonset.apps/calico-node	6	6	6	6
kubernetes.io/os=linux	7h41m			
daemonset.apps/node-local-dns	6	6	6	6
<none>	7h40m			

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/calico-kube-controllers	1/1	1	1	
7h41m				
deployment.apps/coredns	1/1	1	1	
7h40m				
deployment.apps/dashboard-metrics-scraper	1/1	1	1	
7h40m				
deployment.apps/kubernetes-dashboard	1/1	1	1	
7h40m				
deployment.apps/metrics-server	1/1	1	1	
7h40m				

NAME	DESIRED	CURRENT	READY
AGE			
replicaset.apps/calico-kube-controllers-5994f46b69	1	1	1
7h41m			
replicaset.apps/coredns-6db9bd4cd7	1	1	1
7h40m			
replicaset.apps/dashboard-metrics-scraper-77b667b99d	1	1	1
7h40m			
replicaset.apps/kubernetes-dashboard-74fb9f77fb	1	1	1
7h40m			
replicaset.apps/metrics-server-dfb478476	1	1	1
7h40m			

范例:

```
[root@master1 ~]#kubectl get lease -n kube-node-lease
```

NAME	HOLDER	AGE
master1.wang.org	master1.wang.org	69d
master2.wang.org	master2.wang.org	69d
master3.wang.org	master3.wang.org	69d
node1.wang.org	node1.wang.org	69d
node2.wang.org	node2.wang.org	69d
node3.wang.org	node3.wang.org	69d


```
#查看所有名称空间
```

```
[root@master1 ~]#kubectl get lease -A
```

NAMESPACE	NAME	HOLDER	AGE
ingress-nginx	ingress-nginx-leader	ingress-nginx-	
controller-56f8b4f667-kvrff		41d	

kube-node-lease	master1.wang.org	master1.wang.org
		69d
kube-node-lease	node1.wang.org	node1.wang.org
		69d
kube-node-lease	node2.wang.org	node2.wang.org
		69d
kube-node-lease	node3.wang.org	node3.wang.org
		69d
kube-system	kube-apiserver-wcrnq4ikkkdbtpjm6szoyymu3lm	kube-apiserver-wcrnq4ikkkdbtpjm6szoyymu3lm_1bead4d0-d911-44ae-ac5d-7d3180325ae9
		14d
kube-system	kube-controller-manager	
	master1.wang.org_9d4ec997-cc1d-415f-a60e-ea8c7a16f86b	
		69d
kube-system	kube-scheduler	
	master1.wang.org_be63fc1b-ce8c-462f-b9f9-a59f5eeccaa7	
		69d

范例:

```
[root@master1 ~]#kubectl describe namespace default
Name:          default
Labels:        kubernetes.io/metadata.name=default
Annotations:   <none>
Status:        Active

No resource quota.

No LimitRange resource.
```

范例:

```
#只查看指定kube-system名称空间的pod
[root@master1 ~]#kubectl get pod -n kube-system

#查看所有名称空间的pod
[root@master1 ~]#kubectl get pod -A
[root@master1 ~]#kubectl get pod --all-namespaces
```

2.3 创建 Namespace 资源

2.3.1 指令式命令创建

使用指令式命令 `kubectl create` 可以直接创建名称空间，只需指定名称空间名称

```
[root@master1 ~]#kubectl create namespace namespace-test1
namespace/namespace-test1 created

[root@master1 ~]#kubectl get namespace namespace-test1
NAME          STATUS    AGE
namespace-test1  Active   4s
```

注意：实际生产中不建议使用指令式命令和配置，直接使用声明式就可以。

2.3.2 指令式配置创建

`kubectl create -f </path/to/namespace-obj.yaml` 实现创建

此方式生产不建议使用,执行此命令需要指定的namespace不存在

```
[root@master1 ~]#cat namespace-test2.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: namespace-test2

[root@master1 ~]#kubectl create -f namespace-test2.yaml
namespace/namespace-test1 created
```

2.3.1 声明式配置创建

Namespace 是 Kubernetes API 的标准资源类型之一, 其配置主要有 kind、apiVersion、metadata 和 spec 等一级字段组成。使用 `kubectl create/apply -f /path/to/namespace-obj.yaml` 命令就可以创建名称空间资源

范例: 方法1

```
[root@master1 ~]#cat namespace-test2.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: namespace-test2

[root@master1 ~]#kubectl apply -f namespace-test2.yaml
namespace/namespace-test1 created

[root@master1 ~]#kubectl get namespace namespace-test2
NAME              STATUS    AGE
namespace-test2   Active    4m12s
```

范例: 方法2

```
[root@ubuntu2204 ~]#kubectl get ns default -o yaml > default-ns.yaml
[root@ubuntu2204 ~]#vim default-ns.yaml
[root@ubuntu2204 ~]#cat default-ns.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: demo

[root@ubuntu2204 ~]#kubectl apply -f default-ns.yaml
namespace/demo created
[root@ubuntu2204 ~]#kubectl get ns demo
NAME    STATUS    AGE
demo    Active    7s
```

范例: 方法3

```
[root@ubuntu2204 ~]#kubectl create ns stage --dry-run=client -o yaml > stage-ns.yaml
```

```
[root@ubuntu2204 ~]#cat stage-ns.yaml
```

```
apiVersion: v1
```

```
kind: Namespace
```

```
metadata:
```

```
  creationTimestamp: null
```

```
  name: stage
```

```
spec: {}
```

```
status: {}
```

```
[root@ubuntu2204 ~]#kubectl apply -f stage-ns.yaml
```

```
namespace/stage created
```

```
[root@ubuntu2204 ~]#kubectl get ns stage
```

```
NAME      STATUS   AGE
```

```
stage     Active   6s
```

2.4 删除 Namespace 资源

注意: 删除 Namespace 会级联删除此名称空间的所有资源,非常危险

#方法1

```
[root@master1 ~]#kubectl delete namespaces namespace-test1
```

```
namespace "namespace-test1" deleted
```

#方法2

```
[root@master1 ~]#kubectl delete -f namespace-test2.yaml
```

```
namespace "namespace-test2" deleted
```

范例:

#无法删除内置名称空间

```
[root@master1 ~]#kubectl delete ns default kube-public kube-system
```

```
Error from server (Forbidden): namespaces "default" is forbidden: this namespace may not be deleted
```

```
Error from server (Forbidden): namespaces "kube-public" is forbidden: this namespace may not be deleted
```

```
Error from server (Forbidden): namespaces "kube-system" is forbidden: this namespace may not be deleted
```

范例: 删除名称空间会级联删除此名称空间下面的所有资源

```
[root@master1 ~]#kubectl create ns test
```

```
[root@master1 ~]#kubectl create -n test deployment myapp --image registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas 3
```

```
[root@master1 ~]#kubectl get -n test all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/myapp-7dbd45d496-7hk7b	1/1	Running	0	11s
pod/myapp-7dbd45d496-c92hq	1/1	Running	0	11s
pod/myapp-7dbd45d496-vrcm6	1/1	Running	0	11s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
------	-------	------------	-----------	-----


```

deployment.apps/myapp    3/3      3          3          11s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/myapp-7dbd45d496    3          3          3        11s

#删除名称空间
[root@master1 ~]#kubectl delete ns test

#所有此名称空间下的资源都被删除
[root@master1 ~]#kubectl get -n test all
No resources found in test namespace.

```

2.5 删除指定名称空间的资源

使用 kubectl 管理资源时，如果提供了名称空间选项，就表示此管理操作仅针对指定名称空间进行，而删除 Namespace 资源则会级联删除其包含的所有其他资源对象。

命令格式	功能
kubectl delete TYPE RESOURCE -n	删除指定名称空间内的指定资源
kubectl delete TYPE --all -n	删除指定名称空间内的指定类型的所有资源
kubectl delete all -n	删除指定名称空间内的所有资源
kubectl delete all --all	删除所有名称空间中的所有资源

范例:

```
[root@master1 ~]#kubectl delete svc,deployment --all -n demo
```

3 Pod 资源

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/>

3.1 Pod 资源基础

3.1.1 什么是 Pod

Pod 是 Kubernetes API 中最常见最核心资源类型

Pod 是一个或多个容器的集合，因而也可称为容器集，但却是Kubernetes调度、部署和运行应用的原子单元

Pod 的容器运行时只要符合CRI标准即可,而非必须为Docker

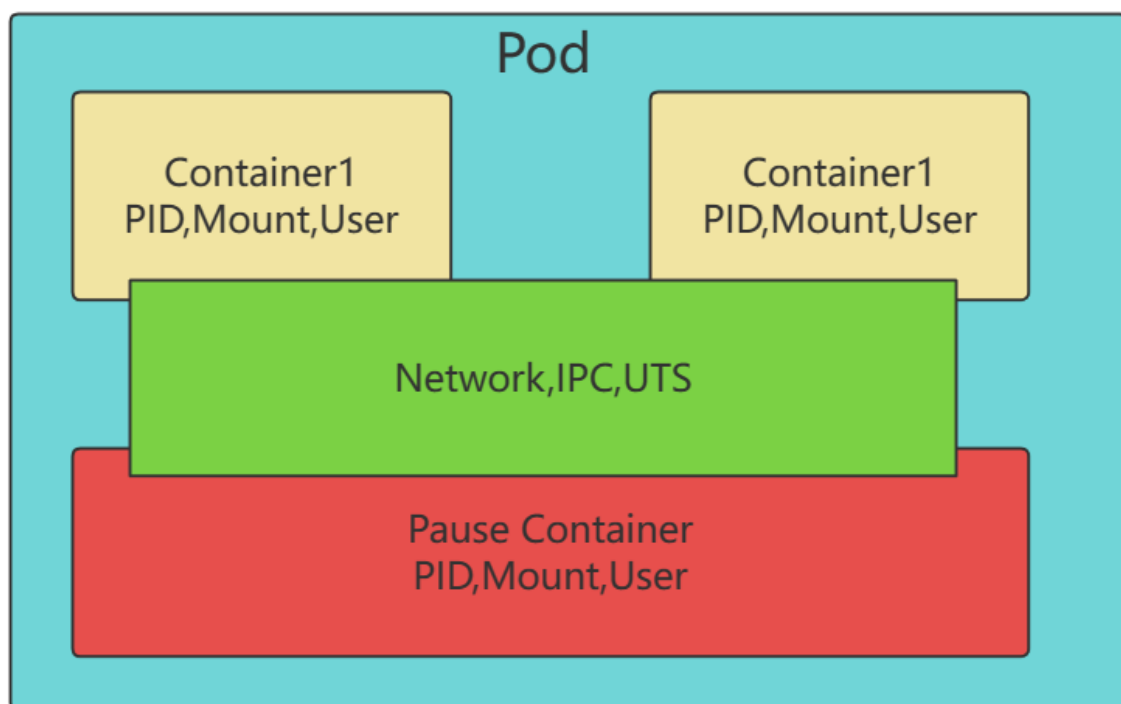
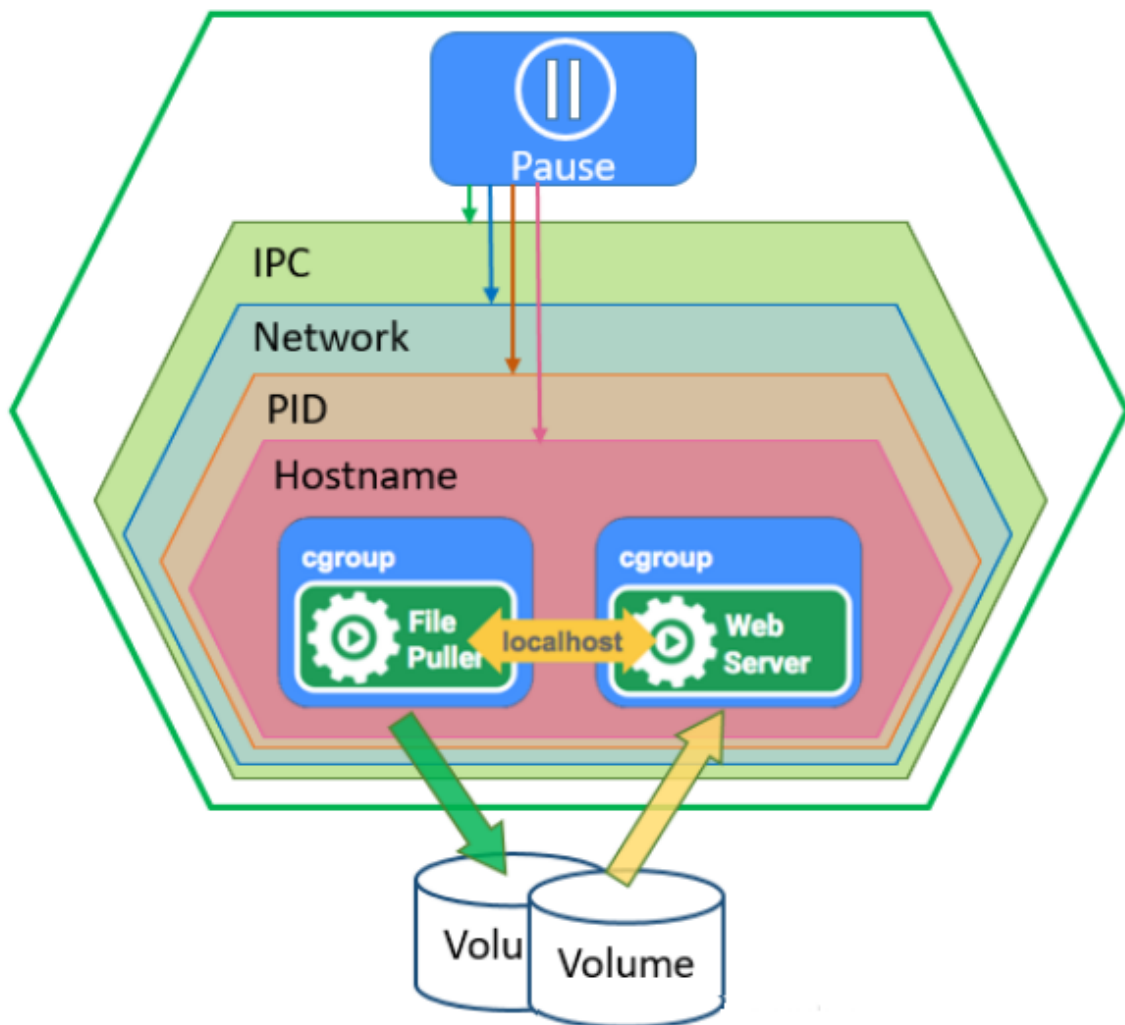
<https://kubernetes.io/docs/setup/production-environment/container-runtimes/>

同一Pod内的所有容器都将运行于由Scheduler选定的同一个worker节点上

在同一个pod内的容器共享的存储资源、网络协议栈及容器的运行控制策略等

每个Pod中的容器依赖于一个特殊名为pause容器事先创建出可被各应用容器共享的基础环境，包括Network、IPC和UTS名称空间共享给Pod中各个容器，PID名称空间也可以共享，但需要用户显式定义,Mount和User是不共享的,每个容器有独立的Mount,User的名称空间

Docker 容器主要有六个名称空间实现容器间的资源隔离: User,Mount,Pid,IPC,Network,UTS



Pod的组成形式有两种

- 单容器Pod：除Pause容器外,仅含有一个容器
- 多容器Pod：除Pause容器外，含有多个具有“超亲密”关系的容器，一般由主容器和辅助容器（比如：sidecar容器）构成

Pod 资源分类：

- 自主式 Pod

由用户直接定义并提交给API Server创建的Pods

节点故障后不具有自动调度至其它节点的故障自愈能力

- 由Workload Controller 管控的 Pod

比如: 由Deployment控制器管理的Pod

节点故障后具有自动调度至其它节点的故障自愈能力

- 静态 Pod

由kubelet加载配置信息后，自动在对应的节点上创建的Pod

用于实现Master节点上的系统组件API Server、Controller-Manager、Scheduler 和Etcd功能的Pod

相关配置存放在控制节点的 /etc/kubernetes/manifests 目录下

3.1.2 自主式 Pod

自主式Pod是由Pod本身去控制本身，不受其它资源如控制器控制的 Pod

这种Pod本身不具有自我修复功能，当自主式Pod被创建后，都会被Kubernetes调度到集群的Node上。

不论是否故障,都会在此Node节点上存在,直到Pod的进程终止和被删掉

中间如果Pod即使出了故障, Pod也不会被调度至其它节点实现自愈。

可以支持三种方法创建自主式pod

- 指令式命令
- 指令式对象配置文件
- 声明式对象配置文件

3.1.2.1 指令式命令创建 Pod

通过kubectl 命令行工具指定选项创建 Pod，适合临时性工作

基本语法

```
kubectl run NAME --image=image [--port=port] [--replicas=replicas]
kubectl run NAME --image=image [--env="key=value"] [--port=port] [--dry-run=server|client] [--overrides=inline-json] [--command] -- [COMMAND] [args...]
[options]
```

#参数详解

<code>--image=''</code>	#指定容器要运行的镜像
<code>--port=''</code>	#设定容器暴露的端口
<code>--dry-run=true</code>	#以模拟的方式来进行执行命令
<code>--env=[]</code>	#执行的时候，向对象中传入一些变量
<code>--labels=''</code>	#设定pod对象的标签
<code>--limits='cpu=200m,memory=512Mi'</code>	#设定容器启动后的资源配置
<code>--replicas=n</code>	#设定pod的副本数量,新版不再支持

```
--command=false      #设为true, 将 -- 后面的字符串做为命令代替容器默认的启动命令, 而非做为默认启动命令的参数
-it                  #打开交互终端
--rm                 #即出即删除容器
--restart=Never      #不会重启
```

范例:

```
#初始化一个Pod对象, 包含一个nginx容器
[root@master1 ~]#kubectl run nginx01 --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
pod/nginx01 created

#查看pod状态
[root@master1 ~]#kubectl get pod nginx01
NAME          READY   STATUS             RESTARTS   AGE
nginx01       0/1     ContainerCreating   0          19s

#创建Busybox的Pod, 默认busybox没有前台进程, 需要指定前台程序才能持续运行
[root@master1 ~]#kubectl run busybox --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0 -- sleep 3600
[root@master1 ~]#kubectl get pod busybox
NAME          READY   STATUS    RESTARTS   AGE
busybox       1/1     Running   0          9m37s
```

范例: 在命令行创建pod 引用环境变量, 之后可以通过\$(VAR_NAME)在容器中获取变量

```
#运行pod时定义一个变量
[root@master1 ~]#kubectl run pod-conf --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 --env="NAME=wangxiaochun"

#验证变量
[root@master1 ~]#kubectl exec -it pod-conf -- sh
# echo $NAME
wangxiaochun

[root@master1 ~]#kubectl delete pod pod-conf
```

范例: 启动一个Pod, 并交互

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
If you don't see a command prompt, try pressing enter.
root@pod-4372 /#
```

3.1.2.2 Pod 资源清单说明

使用以上的命令行方式不利于重复执行

一般会将pod的信息定义在 JSON 或者 YAML 格式的资源清单文件中, 由资源管理命令kubectl 进行指令式或者声明式管理。

创建时用户通过 `kubectl apply/create` 命令将创建请求提交到 API server 并将其保存至集群状态存储系统 etcd 中, 而后由调度器将创建请求调度到最佳的目标 node, 并被运行于该 node 的 kubelet 借助于容器引擎创建并启动 pod。

这种由用户直接通过 API 创建的 Pod 对象也称为自主式 Pod。

3.1.2.2.1 Pod 资源清单必须存在的属性

必须存在的属性，也就是必须包含在创建pod 书写 yaml 文件中，如果没有这些属性的存在它是不允许被执行的

只要指定下面这些属性，就可以实现一个 pod 了

参数名	字段类型	说明
apiVersion	string(键值对)	这里指的是K8S API 的版本，目前是基于 V1 的，可以通过 <code>kubectl api-versions</code> 命令查询
kind	string(键值对)	这里指的是 yaml 文件定义的资源类型和角色，比如我们创建的是一个 pod，它就是 Pod 类型，如果创建的是一个 deployment 就是deployment 类型
metadata	object(对象类型)	元数据对象，固定值就写 metadata，也就意味着 metadata 这个字段对象里面写的是他的一些元数据类型，这个对象类型主要有两个字段，第一个 metadata.name、第二个metadata.namespace
metadata.name	string(键值对)	元数据对象的名字，指定元数据对象的名称，比如命名 pod 的名字
spec	object(对象类型)	详细定义的对象，固定值写 spec
spec.containers[]	list(列表类型)	这里写的是 spec 对象的容器列表定义，是一个列表，第一个主要元素就是 name、第二主要元素是 image
spec.containers[].name	string(键值对)	定义容器的名称,在当前pod中唯一
spec.containers[].image	string(键值对)	这里定义的是镜像名称

3.1.2.2.2 主要对象类型

主要对象类型的含义是这里的值是比较重要的，但是在 yaml 文件中不写也可以，会自动补充一些默认值

参数名	字段类型	说明
metadata.namespace	string(键值对)	指定命名空间，由自己定义，因为 pod 的级别是一个名称空间级别的对象资源，如果没有定义metadata.namespace，默认使用的是 default 名称空间。如果想把它放到一些特殊的名称空间下，就可以加一个 metadata下的namespace 进编写
spec.containers[].imagePullPolicy	string(键值对)	镜像的下载策略，Always、Never、ifnotpresent有三个值可选，（1）Always：意思在使用镜像的时候，都会尝试从远程的仓库去进行下载而不是用本地已经存在的镜像（2）Never：就是永远都是用本地的镜像，没有该镜像就不用了，也不去远程仓库拉取（3）ifnotpresent：如果本地有就使用本地镜像，如果本地没有就从远程仓库去下载。 如果这个值没有设置的话默认用 Always，建议ifnotpresent
spec.containers[].command[]	list(列表)	指定我们容器启动的命令，也就是一个 pod 里面会有很多的容器，每一个容器再去封装镜像的时候都会写一个 CMD 容器运行时指令，那如果用该参数对它做了替换以后就相当于，容器原来的指令不运行了，而运行的是我们这个参数指定的命令
spec.containers[].args[]	list(列表)	指定容器启动运行的时候一些参数
spec.containers[].workingDir	string(键值对)	指定容器的工作目录，工作目录的含义就是一旦我们进入到这个容器的内部时，他就会处于这么一个目录下
spec.containers[].volumeMounts[]	list(列表)	指定容器内部的存储卷配置
spec.containers[].volumeMounts[].name	string(键值对)	指定可以被容器挂载的存储卷名称
spec.containers[].volumeMounts[].mountPath	string(键值对)	指定可以被容器挂载的存储卷路径

参数名	字段类型	说明
spec.containers[].volumeMounts[].readOnly	string(键值对)	设置存储卷路径的读写模式，ture 或者 false，默认为读写模式
spec.containers[].ports[]	list(列表)	指定容器需要用到的端口列表
spec.containers[].ports[].name	string(键值对)	指定端口名称
spec.containers[].ports[].containerPort	string(键值对)	指定容器需要监听的端口号
spec.containers[].ports[].hostPort	string(键值对)	指定容器所在主机需要监听的端口号，默认跟上面的 containerPort 相同，注意设置了 hostPort 同一个台主机无法启动该容器的相同副本（因为主机的端口不能相同，这样会冲突）
spec.containers[].ports[].protocol	string(键值对)	指定端口协议，支持 TCP 或者 UDP，默认值为 TCP
spec.containers[].env[]	list(列表)	指定容器运行前需要设置的环境变量列表，可以赋予一些新的环境变量
spec.containers[].env[].name	string(键值对)	指定给这个环境变量起一个名称
spec.containers[].env[].value	string(键值对)	指定给这个环境变量起一个值
spec.containers[].resources	object(对象类型)	指定资源限制类型，和资源请求的值（这里开始就是这是容器的资源上限）
spec.containers[].resources.limits	object(对象类型)	指定设置容器运行时资源的运行上限
spec.containers[].resources.limits.cpu	string(键值对)	指定 CPU 的限制，单位为 core（核心）数，将用于 docker run --cpu-shares 参数
spec.containers[].resources.limits.memory	string(键值对)	指定 MEM 内存的限制，单位为 MIB,GIB
spec.containers[].resources.requests	object(对象类型)	指定容器启动和调度时的限制设置
spec.containers[].resources.requests.cpu	string(键值对)	CPU 请求，单位为 core 核心数，容器启动时初始化可用数量
spec.containers[].resources.requests.memory	string(键值对)	内存请求，单位为 MIB、GIB，容器启动的初始化可用数量

3.1.2.2.3 额外的参数选项

还有一些额外的参数选项

参数名	字段类型	说明
spec.restartPolicy	string(键值对)	定义 Pod 的重启策略， 默认值always 可选值为 Always、OnFailure、Never，1、Always：Pod 只要退出了，因为在 pod 中运行的是容器，那如果容器退出了，pod 就会出现一个退出状态，但是我们的 kubelet 去负责整个容器的生命周期，所以这时候就会将它重启。但是重启多次还是不行就会表示意外退出不在重启。2、OnFailure：如果你的 pod 里面的容器退出码是非 0 的话就会将该容器重启，因为他没有正常退出。如果是 0 的话表示这个容器是正常退出，就不会再次重启它。3、Never：Pod 终止后，kubelet 将退出码报告给 Master，都不会不会重启该 Pod。不管你是意外退出还是正常退出，都不会重启。
spec.nodeselector	object(对象类型)	定义 Node 节点的 Label 过滤标签，以 key:value 格式指定，可以去选择我们的 node 节点去运行。
spec.imagePullSecrets	object(对象类型)	定义 pull 镜像时使用 secret 名称，以 name:secretkey 格式指定，代表我们在下载这个镜像的时候，要使用什么样的密钥
spec.hostNetwork	Boolean	定义是否使用主机网络模式，默认值为 false 不使用主机网络。设置 true 表示使用宿主机网络，不使用 docker 网桥，同时设置了 true 将无法在同一台宿主机上启动第二个副本。

3.1.2.2.4 yaml格式的pod清单文件示例的完整说明

```
# yaml格式的pod定义文件完整内容：
apiVersion: v1          #必选，版本号，例如v1
kind: Pod               #必选，Pod
metadata:               #必选，元数据
  name: string          #必选，Pod名称要求符合RFC1035规范：
                        http://mirrors.nju.edu.cn/rfc/beta/errata/rfc1035.html
  namespace: string     #Pod所属的命名空间，生产一般不在此处指定死，而通过kubectl -n
                        选项指定比较灵活
  labels:               #自定义标签
    - name: string      #自定义标签名字
  annotations:          #自定义注释列表，可以自由定义，格式比较灵活
    - name: string
spec:                   #必选，Pod中容器的详细定义
  initContainers:        #可选，初始化容器，在容器启动前执行一些初始化操作
    - name: string       #必选，容器名称
      image: string       #必选，容器的镜像名称
      imagePullPolicy: [Always | Never | IfNotPresent] #获取镜像的策略 Always表示下载镜像
                        IfnotPresent表示优先使用本地镜像，否则下载镜像，Nerver表示仅使用本地镜像
      command: [string]   #可选，容器的启动命令列表，如不指定，使用打包时使用的启动命令
      args: [string]      #可选，容器的启动命令参数列表
  containers:           #必选，Pod中容器列表
    - name: string        #必选，容器名称
```



```

image: string                #必选, 容器的镜像名称
imagePullPolicy: [Always | Never | IfNotPresent] #获取镜像的策略 Always表示下载镜像 IfnotPresent表示优先使用本地镜像, 否则下载镜像, Nerver表示仅使用本地镜像
command: [string]            #可选, 容器的启动命令列表, 如不指定, 使用打包时使用的启动命令
args: [string]               #可选, 容器的启动命令参数列表
workingDir: string           #可选, 容器的工作目录
volumeMounts:                #可选, 挂载到容器内部的存储卷配置
- name: string                #引用pod定义的共享存储卷的名称, 需用volumes[]部分定义的卷名
  mountPath: string           #存储卷在容器内mount的绝对路径, 应少于512字符
  readOnly: boolean           #是否为只读模式
ports:                        #需要暴露的端口库号列表
- name: string                #端口号名称
  containerPort: int          #容器需要监听的端口号
  hostPort: int               #容器所在主机需要监听的端口号, 默认与Container相同
  protocol: string            #端口协议, 支持TCP和UDP, 默认TCP
env:                           #可选, 容器运行前需设置的环境变量列表
- name: string                #环境变量名称
  value: string               #环境变量的值
resources:                    #可选, 资源限制和请求的设置
  limits:                     #资源限制的设置
    cpu: string               #Cpu的限制, 单位为core数, 将用于docker run --cpu-shares参数
    memory: string            #内存限制, 单位可以为Mib/Gib, 将用于docker run --memory参数
  requests:                   #资源请求的设置
    cpu: string               #Cpu请求, 容器启动的初始可用数量
    memory: string            #内存清楚, 容器启动的初始可用数量
StartupProbe:                 #可选, 检测容器内进程是否完成启动。注意三种检查方式同时只能使用一种。
  httpGet:                    #对Pod内个容器健康检查方法设置为HttpGet, 需要制定Path、port
    path: string
    port: number
  readinessProbe:              #可选, 健康检测
    httpGet:                  #对Pod内个容器健康检查方法设置为HttpGet, 需要制定Path、port
    path: string
    port: number
  livenessProbe:               #对Pod内个容器健康检查的设置, 当探测无响应几次后将自动重启该容器, 检查方法有exec、HttpGet和tcpSocket, 对一个容器只需设置其中一种方法即可
    exec:                     #对Pod容器内检查方式设置为exec方式
      command: [string]       #exec方式需要制定的命令或脚本
    httpGet:                  #对Pod内个容器健康检查方法设置为HttpGet, 需要制定Path、port
    path: string
    port: number
    host: string
    scheme: string
    HttpHeaders:
      - name: string
        value: string
    tcpSocket:                 #对Pod内个容器健康检查方式设置为tcpSocket方式
      port: number
    initialDelaySeconds: 0     #容器启动完成后首次探测的时间, 单位为秒
    timeoutSeconds: 0          #对容器健康检查探测等待响应的超时时间, 单位秒, 默认1秒

```

```

    periodSeconds: 0 #对容器监控检查的定期探测时间设置，单位秒，默认
10秒一次
    successThreshold: 0
    failureThreshold: 0
    securityContext:
      privileged: false
    restartPolicy: [Always | Never | OnFailure] #Pod的重启策略，Always表示一旦不管以何
种方式终止运行，kublet都将重启，OnFailure表示只有Pod以非0退出码退出才重启，Nerver表示不再重
启该Pod
    nodeSelector: oobject #设置NodeSelector表示将该Pod调度到包含这个label的node
上，以key: value的格式指定
    nodeName: node节点 #指定Pod 运行的节点，默认随机调度至合适的节点
    imagePullSecrets: #Pull镜像时使用的secret名称，以key: secretkey格式指定
      - name: string
    hostNetwork: false #是否使用主机网络模式，默认false，如果设置true，表示使用
宿主网络
    volumes: #在该pod上定义共享存储卷列表
      - name: string #共享存储卷名称 （volumes类型有很多种）
        emptyDir: {} #类型为emptyDir的存储卷，与Pod同生命周期的一个临时目
录。为空值
        hostPath: string #类型为hostPath的存储卷，表示挂载Pod所在宿主机的
的目录
        path: string #Pod所在宿主机的目录，将被用于同期中mount
的目录
        secret: #类型为secret的存储卷，挂载集群与定义的secre对象到
容器内部
          scretname: string
          items:
            - key: string
              path: string
        configMap: #类型为configMap的存储卷，挂载预定义的configMap对象到容
器内部
          name: string
          items:
            - key: string
              path: string

```

3.1.2.2.5 创建pod资源清单文件

3.1.2.2.5.1 手动创建pod资源清单文件

```

[root@master1 ~]#vim /data/kubernetes/yaml/pod-test1.yaml
[root@master1 ~]#cat /data/kubernetes/yaml/pod-test1.yaml
apiVersion: v1 #指定API版本，此为固定值
kind: Pod #指定资源类型 pod
metadata: #元数据类型
  name: pod-test1 # 指定pod名称
  namespace: default #指定名称空间，如果不指定为default
spec: #表述信息
  containers: #容器信息
    - name: nginx-web01 #容器名称为APP
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 #运行的是
nginx 镜像，可以同时运行多个容器

```

3.1.2.2.5.2 利用已有pod生成资源清单文件

自己编写资源定义文件有些繁琐，可以利用现有的pod生成yaml清单文件,再修改即可

```
[root@master1 ~]#kubectl get pod pod-test1 -o yaml --export > pod-test1-export.yaml
#命令解析
-o yaml      #以yaml的格式显示当前资源的所有属性信息。
--export     #在导出信息的时候，会忽略掉有系统生成的信息,这个功能在1.19版本之后被移除了

#修改生成的文件,只保留需要的内容
[root@master1 ~]#vim pod-test1-export.yaml
[root@master1 ~]#cat pod-test1-export.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-test2
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    name: nginx-web02
```

3.1.2.3 指令式对象配置管理方式

3.1.2.3.1 创建 Pod 资源

```
[root@master1 ~]#cat pod-test1.yaml
apiVersion: v1      #指定API版本,此为固定值
kind: Pod           #指定资源类型 pod
metadata:           #元数据类型
  name: pod-test1   #指定pod名称
  namespace: default #指定名称空间,如果不指定为default
  annotations:      #指定注解信息
    wang.org/author: "wangxiaochun"
spec:               #表述信息
  containers:       #容器信息
  - name: nginx-web01 #容器名称为APP
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 #运行的是
nginx 镜像,可以同时运行多个容器
    #command: ["/bin/sh","c","sleep 100"] #可以指定运行的命令,而非镜像内的命令
    #command: ["/bin/sh"]                 #也可以用下面格式
    #args: ["-c","sleep 100"]
#注意: 一个资源清单文件中可以同时写多个Pod对象,彼此间使用"---"隔开就可以

[root@master1 ~]#kubectl create -f /data/kubernetes/yaml/pod-test1.yaml
pod/pod-test1 created
```

3.1.2.3.2 更新 Pod 资源

```
#方法1
#导出配置
[root@master1 ~]#kubectl get pods pod-test1 -o yaml > pod-test1-update.yaml

# 编辑导出的配置清单
[root@master1 ~]#vim pod-test1-update.yaml
image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.18.0 #修改版本
```

```
#更新
[root@master1 ~]#kubectl replace -f pod-test1-update.yaml
pod/pod-test1 replaced

#验证是否修改
[root@master1 ~]#kubectl describe -f pod-test1.yaml

#方法2
#此方法在线直接编辑,生产不建议使用
[root@master1 ~]#kubectl edit pod pod-test1
[root@master1 ~]#kubectl edit pod/pod-test1
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.22.0 #修改镜像版本

#验证立即生效
[root@master1 ~]#kubectl get pod pod-test1 -o wide
NAME             READY   STATUS    RESTARTS   AGE   IP             NODE
NOMINATED NODE   READINESS GATES
pod-test1        1/1     Running   1 (2m34s ago)   3m50s   172.16.3.15    node1.wang.org
node1.wang.org    <none>   <none>
[root@master1 ~]#curl -I 172.16.3.15
HTTP/1.1 200 OK
Server: nginx/1.16.0
Date: Tue, 01 Mar 2022 09:28:32 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Tue, 23 Apr 2019 10:18:21 GMT
Connection: keep-alive
ETag: "5cbee66d-264"
Accept-Ranges: bytes
```

3.1.2.4 声明式对象管理方式

在指令式对象配置管理方式中,同时指定的资源必须进行同一种操作,而且 `kubectl replace` 命令是通过完全替换现有的活动对象来进行资源的更新,对于生产环境并非理想的做法。

而声明式对象配置操作在管理资源对象时将配置信息保存于目标对象的注解中,并通过比较活动对象的当前配置、前一次管理操作时保存于注解中的配置及当前命令提供的配置生成更新补丁,从而完成活动对象的补丁式更新操作。常用的命令有 `kubectl apply` 和 `kubectl patch` 等。

范例: 创建 Pod 资源对象

```
[root@master1 ~]#kubectl apply -f pod-test1.yaml
pod/pod-test1 created

[root@master1 ~]#kubectl get -f pod-test1.yaml -o wide
NAME             READY   STATUS    RESTARTS   AGE   IP             NODE
NOMINATED NODE   READINESS GATES
pod-test1        1/1     Running   0           5m8s   172.16.3.13    node1.wang.org
<none>           <none>
[root@master1 ~]#curl 172.16.3.13 -I
HTTP/1.1 200 OK
Server: nginx/1.21.5
```

```
Date: Tue, 01 Mar 2022 04:32:32 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Tue, 28 Dec 2021 15:28:38 GMT
Connection: keep-alive
ETag: "61cb2d26-267"
Accept-Ranges: bytes
```

范例: 更新应用版本

```
#更新对象的操作, 可以直接在原有的资源清单文件上修改后再执行kubectl apply
[root@master1 ~]#vim pod-test1.yaml
[root@master1 ~]#cat pod-test1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-test1
  labels:
    app: test1
    version: v1.0
spec:
  containers:
  - name: nginx-web01
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 #修改此行

#应用更新
[root@master1 ~]#kubectl apply -f pod-test1.yaml
pod/pod-test1 configured

#验证
[root@master1 ~]#kubectl get -f pod-test1.yaml -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE
NOMINATED NODE READINESS GATES
pod-test1     1/1     Running   0           5m33s  172.16.3.13   node1.wang.org
<none>        <none>
[root@master1 ~]#curl 172.16.3.13 -I
HTTP/1.1 200 OK
Server: nginx/1.20.0
Date: Tue, 01 Mar 2022 04:33:03 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Tue, 20 Apr 2021 13:35:47 GMT
Connection: keep-alive
ETag: "607ed8b3-264"
Accept-Ranges: bytes
```

3.1.3 查看 Pod 信息

可以通过以下命令查看Pod 状态

```
kubectl get pod <pod_name> [(-o|--output=)json|yaml|jsonpath] [-n namespace]
kubectl describe pod <pod_name> [-n namespace]
```

范例

#查看当前名称空间的所有pod

```
[root@master1 ~]#kubectl get pod
```

#查看所有空间的所有pod

```
[root@master1 ~]#kubectl get pod -A
```

#查看指定空间的所有pod

```
[root@master1 ~]#kubectl get pod -n kube-system
```

#查看指定pod当前状态信息

```
[root@master1 ~]#kubectl get -f pod-test1.yaml
```

```
[root@master1 ~]#kubectl get pod pod-test1
```

```
[root@master1 ~]#kubectl get po pod-test1 #支持缩写po
```

NAME	READY	STATUS	RESTARTS	AGE
pod-test1	1/1	Running	0	67s

#持续查看pod状态

```
[root@master1 ~]#kubectl get pods pod-test1 -w
```

#查看扩展信息

```
[root@master1 ~]#kubectl get pod pod-test1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod-test1	1/1	Running	0	88s	172.16.3.12	node1.wang.org
<none>		<none>				

#通过 -o yaml 的方式来进行查看yaml格式的详细信息

```
[root@master1 ~]#kubectl get pod pod-test -o yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  annotations:
```

```
    kubectl.kubernetes.io/last-applied-configuration: |
```

```
.....
```

#查看pod上面的label

```
[root@master1 ~]#kubectl get pod pod-test1 --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
pod-test1	1/1	Running	7 (7h4m ago)	14d	app=test1,version=v2.0

#定制化显示

#定制格式的语法: 自定义字段名:属性获取命令

#多个自定义字段显示的时候, 可以使用逗号","隔开

```
[root@master1 ~]#kubectl get pod pod-test1 -o custom-
```

```
columns=NAME:metadata.name,STATUS:status.phase
```

NAME	STATUS
pod-test1	Running

#查看指定pod的详细信息

```
[root@master1 ~]#kubectl describe pod pod-test1
```

```
Name: pod-test1
```

```
Namespace: default
```

```
Priority: 0
```

```
Node: node1.wang.org/10.0.0.104
```

```
.....
```

范例: 按Json格式查看Pod 信息

```
[root@master1 ~]#kubectl get pod pod-test-5558f75445-9mhrt -o jsonpath=
{.metadata.name}
pod-test-5558f75445-9mhrt

[root@master1 ~]#kubectl get pod pod-test-5558f75445-9mhrt -o jsonpath=
{.spec.containers[0].name}
pod-test

[root@master1 ~]#kubectl get pod mynginx -o yaml -o jsonpath=
{.spec.containers[0].image}
registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.26.0

[root@master1 ~]#kubectl get pod myapp -o jsonpath={.status.podIP}
10.244.4.6
```

3.1.4 查看 Pod 中指定容器应用的日志

查看 Pod 日志格式如下

```
kubectl logs [-f] (POD | TYPE/NAME) [-c CONTAINER] [options]

# 选项
-f, --follow=true      #跟踪日志
-p, --previous=true   #默认只能查看到当前容器此次重启后生成的日志，此选项可以查看前一个已退出
                        的容器的日志
--all-containers=true #所有容器
--tail=N              #最后N个日志
```

范例: 容器无法启动错误查看

```
[root@master1 ~]#kubectl run mysql --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle

[root@master1 ~]#kubectl logs mysql
2024-12-27 03:32:06+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL
Server 8.0.29-1.el8 started.
2024-12-27 03:32:07+00:00 [Note] [Entrypoint]: Switching to dedicated user
'mysql'
2024-12-27 03:32:07+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL
Server 8.0.29-1.el8 started.
2024-12-27 03:32:07+00:00 [ERROR] [Entrypoint]: Database is uninitialized and
password option is not specified
    You need to specify one of the following:
    - MYSQL_ROOT_PASSWORD
    - MYSQL_ALLOW_EMPTY_PASSWORD
    - MYSQL_RANDOM_ROOT_PASSWORD

[root@master1 ~]#kubectl logs --tail=3 mysql
    - MYSQL_ROOT_PASSWORD
    - MYSQL_ALLOW_EMPTY_PASSWORD
    - MYSQL_RANDOM_ROOT_PASSWORD

[root@master1 ~]#kubectl run mysql --env="MYSQL_ALLOW_EMPTY_PASSWORD=yes" --
image registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
```

pod/mysql created

```
[root@master1 ~]#kubectl get pod mysql -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES			
mysql	1/1	Running	0	2m16s	10.244.4.9	node3.wang.org
<none>		<none>				

#

```
[root@master1 ~]#apt update && apt -y install mysql-client
```

```
[root@master1 ~]#mysql -uroot -h10.244.4.9 -e status
```

```
-----  
mysql Ver 8.0.40-0ubuntu0.22.04.1 for Linux on x86_64 ((Ubuntu))
```

```
Connection id:          9  
Current database:         
Current user:           root@10.244.0.0  
SSL:                    Cipher in use is TLS_AES_256_GCM_SHA384  
Current pager:          stdout  
Using outfile:          ''  
Using delimiter:        ;  
Server version:         8.0.29 MySQL Community Server - GPL  
Protocol version:       10  
Connection:             10.244.4.9 via TCP/IP  
Server characterset:     utf8mb4  
Db characterset:         utf8mb4  
Client characterset:     utf8mb4  
Conn. characterset:     utf8mb4  
TCP port:               3306  
Binary data as:         Hexadecimal  
Uptime:                 1 min 47 sec  
  
Threads: 2 Questions: 8 Slow queries: 0 Opens: 118 Flush tables: 3 Open  
tables: 37 Queries per second avg: 0.074  
-----
```

范例:

#查看名为pod-test1的pod中nginx-web01的容器的日志

```
[root@master1 ~]#kubectl logs pod-test1 -c nginx-web01
```

```
172.16.0.0 - - [03/Mar/2021:02:01:52 +0000] "GET / HTTP/1.1" 200 612 "-"  
"curl/7.68.0" "-"
```

#进入容器内执行操作

```
[root@master1 ~]#kubectl exec pod-test -- ps aux
```

PID	USER	TIME	COMMAND
1	root	0:03	python3 /usr/local/bin/demo.py
50	root	0:00	ps aux

3.1.5 进入Pod 执行命令

```
kubectl exec (POD | TYPE/NAME) [-c CONTAINER] [flags] -- COMMAND [args...]  
[options]
```

范例


```
[root@ubuntu2204 ~]#kubectl exec -h
Execute a command in a container.
```

Examples:

```
# Get output from running the 'date' command from pod mypod, using the first
container by default
```

```
kubectl exec mypod -- date
```

```
# Get output from running the 'date' command in ruby-container from pod mypod
```

```
kubectl exec mypod -c ruby-container -- date
```

```
# Switch to raw terminal mode; sends stdin to 'bash' in ruby-container from
pod mypod
```

```
# and sends stdout/stderr from 'bash' back to the client
```

```
kubectl exec mypod -c ruby-container -i -t -- bash -il
```

```
# List contents of /usr from the first container of pod mypod and sort by
modification time
```

```
# If the command you want to execute in the pod has any flags in common (e.g.
-i),
```

```
# you must use two dashes (--) to separate your command's flags/arguments
```

```
# Also note, do not surround your command and its flags/arguments with quotes
```

```
# unless that is how you would execute it normally (i.e., do ls -t /usr, not
"ls -t /usr")
```

```
kubectl exec mypod -i -t -- ls -t /usr
```

```
# Get output from running 'date' command from the first pod of the deployment
mydeployment, using the first container
```

```
by default
```

```
kubectl exec deploy/mydeployment -- date
```

```
# Get output from running 'date' command from the first pod of the service
myservice, using the first container by
```

```
default
```

```
kubectl exec svc/myservice -- date
```

Options:

```
-c, --container='':
```

Container name. If omitted, use the `kubectl.kubernetes.io/default-container` annotation **for** selecting the

container to be attached or the first container **in** the pod will be chosen

```
-f, --filename=[]:
```

to use to exec into the resource

```
--pod-running-timeout=1m0s:
```

The length of time (like 5s, 2m, or 3h, higher than zero) to wait **until** at least one pod is running

```
-q, --quiet=false:
```

Only print output from the remote session

```
-i, --stdin=false:
```

Pass stdin to the container

```
-t, --tty=false:
```

Stdin is a TTY

```
Usage:
  kubectl exec (POD | TYPE/NAME) [-c CONTAINER] [flags] -- COMMAND [args...]
[options]
```

范例:

```
[root@master1 ~]#kubectl run mysql --env="MYSQL_ALLOW_EMPTY_PASSWORD=yes" --
image registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
pod/mysql created
[root@master1 ~]#kubectl exec mysql -- date
Mon Mar 24 01:08:13 UTC 2025

[root@master1 ~]#kubectl run mysql --env="TZ=Asia/Shanghai" --
env="MYSQL_ALLOW_EMPTY_PASSWORD=yes" --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
pod/mysql created

[root@master1 ~]#kubectl exec mysql -- date
Fri Dec 27 11:47:13 CST 2024

[root@master1 ~]#kubectl exec -it mysql -- sh
sh-4.4# mysql -uroot -e status
-----
mysql Ver 8.0.29 for Linux on x86_64 (MySQL Community Server - GPL)

Connection id:      8
Current database:
Current user:       root@localhost
SSL:               Not in use
Current pager:      stdout
Using outfile:      ''
Using delimiter:    ;
Server version:     8.0.29 MySQL Community Server - GPL
Protocol version:   10
Connection:         Localhost via UNIX socket
Server characterset: utf8mb4
Db characterset:    utf8mb4
Client characterset: latin1
Conn. characterset: latin1
UNIX socket:        /var/run/mysqld/mysqld.sock
Binary data as:     Hexadecimal
Uptime:             48 sec

Threads: 2 Questions: 5 Slow queries: 0 Opens: 117 Flush tables: 3 Open
tables: 36 Queries per second avg: 0.104
-----
```

3.1.6 删除 Pod

删除资源对象推荐使用指令式管理命令 `kubectl delete`

格式:

```
kubectl delete pod <pod_name> ... [--force --grace-period=0]
```

```
#优雅删除
[root@master1 ~]#kubectl delete pod pod-test1

#立即删除
[root@master1 ~]#kubectl delete pod pod-test1 --force --grace-period=0
warning: Immediate deletion does not wait for confirmation that the running
resource has been terminated. The resource may continue to run on the cluster
indefinitely.
pod "pod-test1" force deleted

#删除多个资源
[root@master1 ~]#kubectl delete -f pod-test1.yaml -f pod-test2.yaml

#删除指定目录下的资源
[root@master1 ~]#kubectl delete -f /data/kubernetes/yaml/

#删除demo名称空间的所有Pod
[root@master1 ~]#kubectl delete pod --all -n demo

#删除所有pod
[root@master1 ~]#kubectl get po|cut -d" " -f1 | tail -n +2 |xargs kubectl delete
pod

#快速删除所有pod
[root@master1 ~]#kubectl get pod|awk 'NR!=1{print $1}'|xargs -i kubectl delete
pod {} --force --grace-period=0

[root@master1 ~]#vim .bashrc
alias rmpod="kubectl get pod|awk 'NR!=1{print \$1}'|xargs -i kubectl delete pod
{} --force --grace-period=0"
[root@master1 ~]#. .bashrc
```

注意:

- 不要使用 apply 命令结合 `--prune` 选项进行删除, 即 `kubectl apply -f /path/to/yaml-file/ --prune -l labels`
- 该命令会删除文件夹下不存在相应定义(不属于基于标签过滤出的符合条件的对象)的资源对象。

3.1.7 创建定制的 Pod

kubernetes 支持多种定制 Pod 的实现方法

- 对于不同应用的Pod,重新定制对应的镜像
- 启动容器时指定env环境变量
- 启动容器的指定command和args
- 将配置信息基于卷资源对象, 再将其加载到容器, 比如: configMap和secret等

3.1.7.1 利用环境变量实现容器传参

在容器上嵌套使用env字段

- 每个环境变量需要通过pod.spec.containers.env.name给出指定的名称
- 传递的值则定义在pod.spec.containers.env.value字段上

范例: 实现 LAMP的应用wordpress

```
#mysql
[root@master1 ~]#cat pod-mysql.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mydb
  namespace: default
spec:
  containers:
  - name: mysql
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
    env:
      - name: MYSQL_ROOT_PASSWORD
        value: "654321" #注意如果是数字必须加""引起来
      - name: MYSQL_DATABASE
        value: wordpress
      - name: MYSQL_USER
        value: wpuser
      - name: MYSQL_PASSWORD
        value: "123456" #注意如果是数字必须加""引起来
```

```
[root@master1 ~]#kubectl apply -f pod-mysql.yaml
```

#查看MySQL对应Pod的IP

```
[root@master1 ~]#kubectl get pod mydb -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
mydb	1/1	Running	0	5m46s	10.244.1.6	node3.wang.org

<none>

#wordpress

```
[root@master1 ~]#cat pod-wordpress.yaml
apiVersion: v1
kind: Pod
metadata:
  name: wordpress
  namespace: default
spec:
  hostNetwork: true #使用容器所在宿主机网络的IP和Port
  containers:
  - name: wordpress
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
    #ports: #或者使用宿主机的指定端口
    #- name: http #不支持大写字母
    # containerPort: 80
    # hostPort: 8888
    env:
      - name: TZ
        value: Asia/Shanghai
      - name: WORDPRESS_DB_HOST
        value: 10.244.1.6 #指向前面的pod的mysql当前的IP地址
      - name: WORDPRESS_DB_NAME
        value: wordpress
      - name: WORDPRESS_DB_USER
        value: wpuser
      - name: WORDPRESS_DB_PASSWORD
        value: "123456"
```

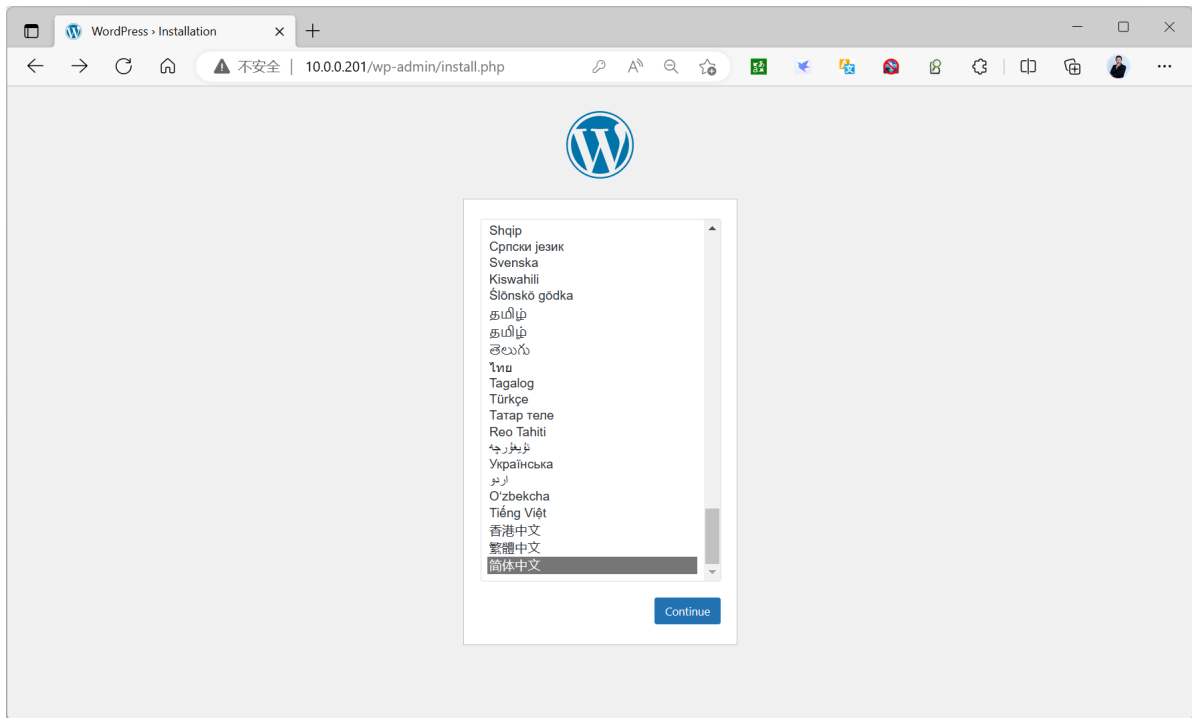
```
[root@master1 ~]#kubectl apply -f pod-wordpress.yaml
```

```
[root@master1 ~]#kubectl get pod wordpress -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES			
wordpress	1/1	Running	0	5m12s	10.0.0.201	node1.wang.org
<none>		<none>				

#打开浏览器访问

http://10.0.0.201



范例

```
[root@master1 ~]#kubectl run mypod --image registry.cn-  
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
```

pod/mypod created

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
mypod	1/1	Running	0	4s
nfs-client-provisioner-695dc47789-h4hf1	1/1	Running	16 (29m ago)	259d

```
[root@master1 ~]#kubectl exec mypod -- env
```

PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

HOSTNAME=mypod

KUBERNETES_SERVICE_PORT_HTTPS=443

KUBERNETES_PORT=tcp://10.96.0.1:443

KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443

KUBERNETES_PORT_443_TCP_PROTO=tcp

KUBERNETES_PORT_443_TCP_PORT=443

KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1

KUBERNETES_SERVICE_HOST=10.96.0.1

KUBERNETES_SERVICE_PORT=443

DEPLOYENV=Production

RELEASE=Stable

PS1=[\u@\h \w]\\$

HOME=/root

```
[root@master1 ~]#kubectl exec mypod -- ss -ntl
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:80	0.0.0.0:*	

#准备清单文件

```
[root@master1 ~]#cat pod-with-env.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-env-demo
spec:
  containers:
  - name: demo-env
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    env:
    - name: HOST
      value: "127.0.0.1"
    - name: PORT
      value: "9999"
```

```
[root@master1 ~]#kubectl apply -f pod-env.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
mypod	1/1	Running	0	75s
nfs-client-provisioner-695dc47789-h4hfl	1/1	Running	16 (31m ago)	259d
pod-env-demo	1/1	Running	0	2s

```
[root@master1 ~]#kubectl exec pod-env-demo -- ss -ntl
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	127.0.0.1:9999	0.0.0.0:*	

```
[root@master1 ~]#kubectl exec pod-env-demo -- env
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=pod-env-demo
HOST=127.0.0.1
PORT=9999
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
DEPLOYENV=Production
RELEASE=Stable
PS1=[\u@\h \w]\$
HOME=/root
```

3.1.7.2 利用command和args字段传递容器的启动命令和参数

Pod配置中，spec.containers[].command字段能够在容器上指定替代镜像默认运行的应用程序，且可同时使用spec.containers[].args字段进行参数传递，它们将覆盖镜像中的默认定义的参数。

若仅定义了command字段时，其值将覆盖镜像中定义的程序及参数。

若仅是定义了args字段，该字段值将作为参数传递给镜像中默认指定运行的应用程序

注意: args中使用环境变量,需要使用格式: \$(环境变量名)

范例: 修改默认进程

```
#默认busybox无法启动
[root@master1 ~]#docker history registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
IMAGE                CREATED              CREATED BY          SIZE
COMMENT
219ee5171f80         4 years ago        /bin/sh -c #(nop)  CMD ["sh"]          0B

<missing>            4 years ago        /bin/sh -c #(nop)  ADD file:85e5bcb27fcc24f6a... 1.23MB
[root@master1 ~]#kubectl run mybusybox --image registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
#k8s自动重启
[root@master1 ~]#kubectl get pod
NAME          READY   STATUS             RESTARTS   AGE
mybusybox     0/1     CrashLoopBackOff   5 (50s ago) 3m53s

#用自定义的命令替换sh
[root@master1 ~]#cat pod-with-cmd-admin-box.yaml
apiVersion: v1
kind: Pod
metadata:
  name: admin-box
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    #image: registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1
    name: admin-box
    command: ['sleep', '36000']
    #command: ['sleep']
    #args: ['36000']

[root@master1 ~]#kubectl apply -f pod-with-cmd-admin-box.yaml

#查看
[root@master1 ~]#kubectl get pod admin-box
NAME          READY   STATUS    RESTARTS   AGE
admin-box     1/1     Running   0           20s

[root@master1 ~]#kubectl exec admin-box -- pstree -p
sleep(1)
[root@master1 ~]#kubectl exec admin-box -- ps aux
PID   USER     TIME   COMMAND
    1  root      0:00   sleep 36000
    20  root      0:00   ps aux
```

范例: 添加运行参数

```
#原镜像的启动命令
[root@node1 ~]#docker pull registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
[root@node1 ~]#docker history --no-trunc registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 |grep ENTRYPOINT
<missing>
years ago /bin/sh -c #(nop) ENTRYPOINT ["/bin/sh" "-c" "python3 /usr/local/bin/demo.py"]
0B

[root@master1 ~]#cat pod-with-cmd-and-args.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-cmd-and-args
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ['/bin/sh', '-c']
    args: ['python3 /usr/local/bin/demo.py -p 8080']

[root@master1 ~]#kubectl get pod pod-with-cmd-and-args -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
pod-with-cmd-and-args               1/1     Running   0           70s   172.16.4.39    node2.wang.org
node2.wang.org                     <none>   <none>

[root@master1 ~]#kubectl exec pod-with-cmd-and-args -- ps aux
PID    USER    TIME  COMMAND
   1   root      0:00 python3 /usr/local/bin/demo.py -p 8080
   9   root      0:00 ps aux

[root@master1 ~]#kubectl exec pod-with-cmd-and-args -- hostname -i
172.16.4.39

[root@master1 ~]#curl 172.16.4.39:8080
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-with-cmd-and-args, ServerIP: 172.16.4.39!
```

范例: 添加运行命令, 参数和环境变量

```
[root@master1 ~]#cat pod-with-cmd-and-args.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-cmd-and-args
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ['/bin/sh', '-c']
    #args: ['python3 /usr/local/bin/demo.py -p $port']
```



```
args: ['python3 /usr/local/bin/demo.py -p ${port}']
env: #和上面args同时启用, args优先生效, PORT优先级低
  - name: PORT
    value: "8888"
```

3.1.8 使用宿主机网络实现容器的外部访问

默认容器使用私有的独立网段, 无法从集群外直接访问, 可以通过下面两种方式实现外部访问

- 让容器直接使用宿主机的网络地址, 即容器使用host的网络模型
- 让容器通过宿主机的端口映射实现, 即DNAT

注意: 都要避免端口冲突

范例: 直接使用容器所在宿主机网络

```
#清单文件
[root@master1 ~]#cat pod-hostnetwork.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-hostnetwork-demo
spec:
  #nodeName: node2.wang.org
  hostNetwork: true
  containers:
  - name: demo-env
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    env:
    - name: PORT
      value: "9999" #要确保此端口在宿主机不能被占用

[root@master1 ~]#kubectl apply -f pod-hostnetwork.yaml

[root@master1 ~]#kubectl get pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP        NODE           NOMINATED NODE
pod-hostnetwork-demo 1/1     Running   0          9s    10.0.0.203 node3.wang.org <none>

[root@master1 ~]#curl 10.0.0.203:9999
kubernetes pod-test v0.1!! ClientIP: 10.0.0.200, ServerName: node3.wang.org,
ServerIP: 10.0.0.203!
```

范例: 使用容器所在宿主机指定的端口

```
[root@master1 ~]#cat pod-hostport.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-hostport-demo
spec:
  containers:
  - name: demo-env
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    env:
    - name: PORT
      #镜像的环境变量可以指定容器运行时使用的端口
```

```
value: "9999"
ports:
- name: http
  containerPort: 9999
  hostPort: 8888
```

```
[root@master1 ~]#kubectl apply -f pod-hostport.yaml
```

```
pod/pod-hostport-demo created
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
pod-hostport-demo	1/1	Running	0	5s	10.244.168.202	node3.wang.org	<none>

```
#访问容器所在宿主机IP和指定的端口
```

```
[root@master1 ~]#curl 10.0.0.203:8888
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.200, ServerName: pod-hostport-demo,
ServerIP: 10.244.168.202!
```

```
#本质上就是通过宿主机的DNAT策略实现
```

```
[root@node3 ~]#iptables -vnL -t nat|grep DNAT|grep 8888
```

```
1 60 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0
tcp dpt:8888 to:10.244.168.202:9999
```

3.1.9 临时容器 Ephemeral Containers

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/ephemeral-containers/>

3.1.9.1 了解临时容器

Pod 是 Kubernetes 应用程序的基本构建块。由于 Pod 是一次性且可替换的，因此一旦 Pod 创建，就无法将容器加入到 Pod 中。取而代之的是，通常使用 Deployment 以受控的方式来删除并替换 Pod。

有时有必要检查现有 Pod 的状态。例如，对于难以复现的故障进行排查。在这些场景中，可以在现有 Pod 中运行临时容器来检查其状态并运行任意命令。

Kubernetes v1.16推出临时容器, Kubernetes v1.25稳定可用,就是在原有的Pod上,添加一个临时的Container,这个Container可以包含我们排查问题所有的工具,比如: ip、ss、ps、pstree、top、kill、top、jstat、jmap等

什么是临时容器?

临时容器与其他容器的不同之处在于，它们缺少对资源或执行的保证，并且永远不会自动重启，因此不适用于构建应用程序。临时容器使用与常规容器相同的 ContainerSpec 节来描述，但许多字段是不兼容和不允许的。

- 临时容器没有端口配置，因此像 ports, livenessProbe, readinessProbe 这样的字段是不允许的。
- Pod 资源分配是不可变的，因此 resources 配置是不允许的。
- 有关允许字段的完整列表，请参见 EphemeralContainer 参考文档。

临时容器是使用 API 中的一种特殊的 ephemeralcontainers 处理器进行创建的，而不是直接添加到 pod.spec 段，因此无法使用 kubectl edit 来添加一个临时容器。

与常规容器一样，将临时容器添加到 Pod 后，将不能更改或删除临时容器。

临时容器的用途

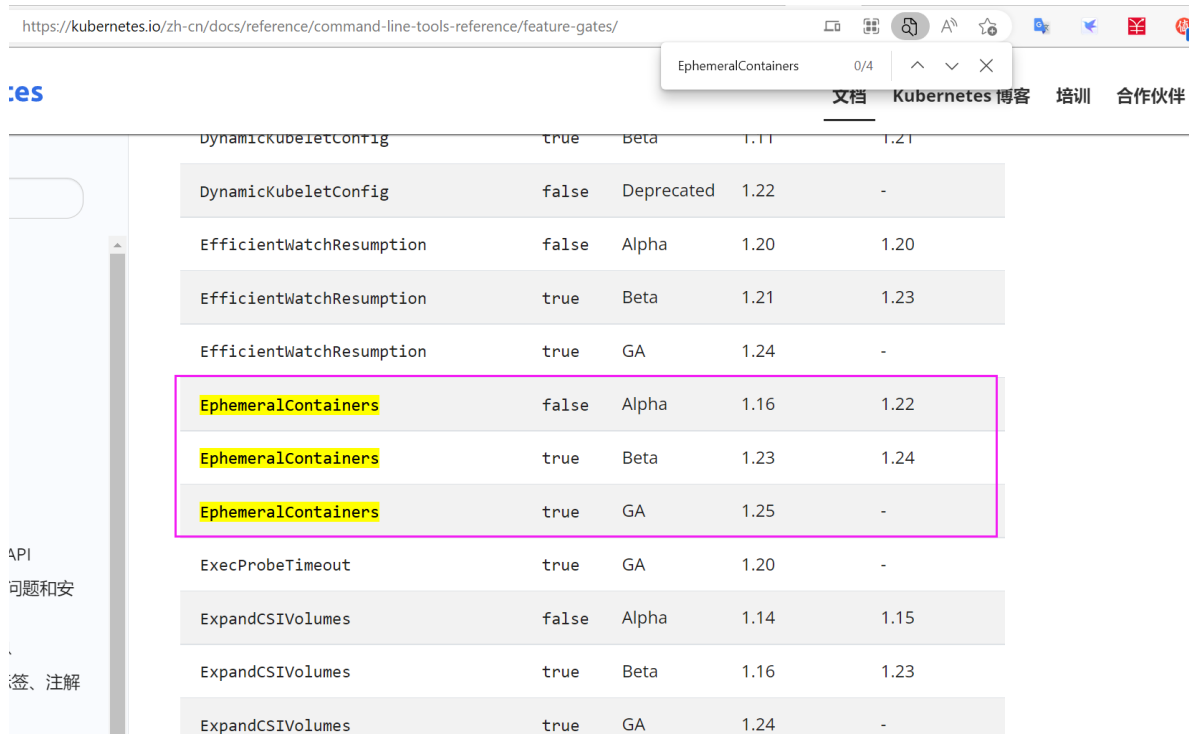
当由于容器崩溃或容器镜像不包含调试工具而导致 kubectl exec 无用时，临时容器对于交互式故障排查很有用。

尤其是，Distroless 镜像允许用户部署最小的容器镜像，从而减少攻击面并减少故障和漏洞的暴露。由于 distroless 镜像不包含 Shell 或任何的调试工具，因此很难单独使用 kubectl exec 命令进行故障排查。

使用临时容器时，启用 进程名字空间共享 很有帮助，可以查看其他容器中的进程。

开启EphemeralContainers特性

<https://kubernetes.io/zh-cn/docs/reference/command-line-tools-reference/feature-gates/>



Feature Gate	Default	Stable	Alpha	Beta	GA
DynamicKubeletConfig	true	Beta	1.11	1.21	
DynamicKubeletConfig	false	Deprecated	1.22	-	
EfficientWatchResumption	false	Alpha	1.20	1.20	
EfficientWatchResumption	true	Beta	1.21	1.23	
EfficientWatchResumption	true	GA	1.24	-	
EphemeralContainers	false	Alpha	1.16	1.22	
EphemeralContainers	true	Beta	1.23	1.24	
EphemeralContainers	true	GA	1.25	-	
ExecProbeTimeout	true	GA	1.20	-	
ExpandCSIVolumes	false	Alpha	1.14	1.15	
ExpandCSIVolumes	true	Beta	1.16	1.23	
ExpandCSIVolumes	true	GA	1.24	-	

范例: 如果是v1.22版前需要手动开启EphemeralContainers

```
#在所有节点修改相关的service文件,包括,api-server,controller-manger,scheduler等
[root@master1 ~]#vim /lib/systemd/system/kubelet.service
[Service]
ExecStart=/usr/bin/kubelet --feature-gates="EphemeralContainers=true" #修改此行
[root@master1 ~]#vim /lib/systemd/system/kubelet.service
[root@master1 ~]#systemctl daemon-reload
[root@master1 ~]#systemctl restart kubelet.service
```

范例: 查看Pod配置

```
[root@master1 ~]#kubectl explain pod.spec
shareProcessNamespace <boolean>
  Share a single process namespace between all of the containers in a pod.
  when this is set containers will be able to view and signal processes from
  other containers in the same pod, and the first process in each container
  will not be assigned PID 1. HostPID and ShareProcessNamespace cannot both
  be set. Optional: Default to false.
```

3.1.9.2 使用临时容器

<https://kubernetes.io/zh-cn/docs/tasks/debug/debug-application/debug-running-pod/#ephemeral-container>

kubectl debug 命令用法

```
[root@master1 ~]# kubectl debug --help
```

Debug cluster resources using interactive debugging containers.

'debug' provides automation for common debugging tasks for cluster objects identified by resource and name. Pods will be used by default if no resource is specified.

The action taken by 'debug' varies depending on what resource is specified. Supported actions include:

- * workload: Create a copy of an existing pod with certain attributes changed, for example changing the image tag to a new version.
- * workload: Add an ephemeral container to an already running pod, for example to add debugging utilities without restarting the pod.
- * Node: Create a new pod that runs in the node's host namespaces and can access the node's filesystem.

Examples:

```
# Create an interactive debugging session in pod mypod and immediately attach to it.
```

```
# (requires the EphemeralContainers feature to be enabled in the cluster)
kubectl debug mypod -it --image=busybox
```

```
# Create a debug container named debugger using a custom automated debugging image.
```

```
# (requires the EphemeralContainers feature to be enabled in the cluster)
kubectl debug --image=myproj/debug-tools -c debugger mypod
```

```
# Create a copy of mypod adding a debug container and attach to it
kubectl debug mypod -it --image=busybox --copy-to=my-debugger
```

```
# Create a copy of mypod changing the command of mycontainer
kubectl debug mypod -it --copy-to=my-debugger --container=mycontainer -- sh
```

```
# Create a copy of mypod changing all container images to busybox
kubectl debug mypod --copy-to=my-debugger --set-image=*=busybox
```

```
# Create a copy of mypod adding a debug container and changing container images
```

```
kubectl debug mypod -it --copy-to=my-debugger --image=debian --set-image=app=app:debug,sidecar=sidecar:debug
```

```
# Create an interactive debugging session on a node and immediately attach to it.
```

```
# The container will run in the host namespaces and the host's filesystem will be mounted at /host
```

```
kubectl debug node/mynode -it --image=busybox
```

Options:

`--arguments-only=false:`

If specified, everything after `--` will be passed to the new container as Args instead of Command.

`--attach=false:`

If `true`, wait `for` the container to `start` running, and `then` attach as if `'kubect1 attach ...'` were called.

Default `false`, unless `'-i/--stdin'` is `set`, in which case the default is `true`.

`-c, --container='':`

Container name to use `for` debug container.

`--copy-to='':`

Create a copy of the target Pod with this name.

`--env=[]:`

Environment variables to `set in` the container.

`--image='':`

Container image to use `for` debug container.

`--image-pull-policy='':`

The image pull policy `for` the container. If left empty, this value will not be specified by the client and defaulted by the server.

`-q, --quiet=false:`

If `true`, suppress informational messages.

`--replace=false:`

When used with `'--copy-to'`, delete the original Pod.

`--same-node=false:`

When used with `'--copy-to'`, schedule the copy of target Pod on the same `node`.

`--set-image=[]:`

When used with `'--copy-to'`, a list of `name=image` pairs `for` changing container images, similar to how `'kubect1 set image'` works.

`--share-processes=true:`

When used with `'--copy-to'`, enable process namespace sharing `in` the copy.

`-i, --stdin=false:`

Keep stdin open on the container(s) `in` the pod, even `if` nothing is attached.

`--target='':`

When using an ephemeral container, target processes `in` this container name.

`-t, --tty=false:`

Allocate a TTY `for` the debugging container.

Usage:

```
kubectl debug (POD | TYPE[.VERSION].GROUP)/NAME [ -- COMMAND [args...] ]
[options]
```

Use **"kubectl options"** for a list of global command-line options (applies to all commands).

范例:

```
[root@master1 ~]#kubectl run ephemeral-demo --image=wangxiaochun/pause:3.1 --
restart=Never
[root@master1 ~]#kubectl run ephemeral-demo --
image=registry.aliyuncs.com/google_containers/pause:3.9 --restart=Never
[root@master1 ~]#kubectl get pod ephemeral-demo
NAME             READY   STATUS    RESTARTS   AGE
ephemeral-demo   1/1     Running   0           57s

#如果你尝试使用 kubectl exec 来创建一个 shell, 你将会看到一个错误, 因为这个容器镜像中没有
shell。
[root@master1 ~]#kubectl exec -it ephemeral-demo -- sh
OCI runtime exec failed: exec failed: unable to start container process: exec:
"sh": executable file not found in $PATH: unknown
command terminated with exit code 126

#通过kubectl debug 添加调试容器。 如果你指定 -i 或者 --interactive 参数, kubectl 将自动
挂接到临时容器的控制台。
[root@master1 ~]#kubectl debug -it ephemeral-demo --image=busybox:1.32.0 --
target=ephemeral-demo
Targeting container "ephemeral-demo". If you don't see processes from this
container it may be because the container runtime doesn't support this feature.
Defaulting debug container name to debugger-84tq7.
If you don't see a command prompt, try pressing enter.
/ # ps
PID    USER     TIME   COMMAND
   1   root      0:00   /pause
  12   root      0:00   sh
  24   root      0:00   ps
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
3: eth0@if21: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 4e:06:00:4c:8f:79 brd ff:ff:ff:ff:ff:ff
    inet 10.244.3.51/24 brd 10.244.3.255 scope global eth0
        valid_lft forever preferred_lft forever

#只显示一个容器
[root@master1 ~]#kubectl get pod ephemeral-demo -o wide
NAME             READY   STATUS    RESTARTS   AGE     IP             NODE
NOMINATED NODE   READINESS GATES
ephemeral-demo   1/1     Running   0           8m35s   10.244.3.51    node3.wang.org
node3.wang.org   <none>   <none>

#查看到临时容器信息
[root@master1 ~]#kubectl describe pod ephemeral-demo
Name:             ephemeral-demo
Namespace:        default
```

Priority: 0
Service Account: default
Node: node3.wang.org/10.0.0.203
Start Time: Sun, 27 Nov 2022 22:28:24 +0800
Labels: run=ephemeral-demo
Annotations: <none>
Status: Running
IP: 10.244.3.51
IPs:
IP: 10.244.3.51
Containers:
ephemeral-demo:
Container ID:
docker://28b533485ead026d26536fc6319a25f35a782d7d0cb9995eec946da9f0f3bd5c
Image: wangxiaochun/pause:3.1
Image ID: docker-
pullable://wangxiaochun/pause@sha256:fcaff905397ba63fd376d0c3019f1f1cb6e7506131389edbc3d22719f1ae54d
Port: <none>
Host Port: <none>
State: Running
Started: Sun, 27 Nov 2022 22:28:51 +0800
Ready: True
Restart Count: 0
Environment: <none>
Mounts:
/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vb8l4
(ro)
Ephemeral Containers:
debugger-84tq7:
Container ID:
docker://47db8156367277277c4fad3915002edbf56f3e4a19fca6fa5cabe60c439e7d5f
Image: busybox:1.32.0
Image ID: docker-
pullable://busybox@sha256:141c253bc4c3fd0a201d32dc1f493bcf3fff003b6df416dea4f41046e0f37d47
Port: <none>
Host Port: <none>
State: Running
Started: Sun, 27 Nov 2022 22:31:06 +0800
Ready: False
Restart Count: 0
Environment: <none>
Mounts: <none>
Conditions:
Type Status
Initialized True
Ready True
ContainersReady True
PodScheduled True
Volumes:
kube-api-access-vb8l4:
Type: Projected (a volume that contains injected data from multiple sources)
TokenExpirationSeconds: 3607
ConfigMapName: kube-root-ca.crt
ConfigMapOptional: <nil>
DownwardAPI: true

```

QoS Class:                               BestEffort
Node-Selectors:                           <none>
Tolerations:                             node.kubernetes.io/not-ready:NoExecute op=Exists
for 300s
                                           node.kubernetes.io/unreachable:NoExecute op=Exists
for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
Normal    Scheduled   4m30s default-scheduler Successfully assigned
default/ephemeral-demo to node3.wang.org
Normal    Pulling     4m30s kubelet        Pulling image
"wangxiaochun/pause:3.1"
Normal    Pulled      4m3s  kubelet        Successfully pulled image
"wangxiaochun/pause:3.1" in 26.563175958s
Normal    Created     4m3s  kubelet        Created container ephemeral-demo
Normal    Started     4m3s  kubelet        Started container ephemeral-demo
Normal    Pulling     109s  kubelet        Pulling image "busybox:1.32.0"
Normal    Pulled      108s  kubelet        Successfully pulled image
"busybox:1.32.0" in 1.721851652s
Normal    Created     108s  kubelet        Created container debugger-84tq7
Normal    Started     108s  kubelet        Started container debugger-84tq7

```

#在运行Pod的宿主机可以看到两个容器

```

[root@node3 ~]#docker ps
CONTAINER ID   IMAGE                STATUS              PORTS          NAMES
47db81563672   busybox              Up 4 minutes       k8s_debugger-
84tq7_ephemeral-demo_default-d870ecf8-707f-437d-ae91-1af9c21413a9_0
28b533485ead   wangxiaochun/pause   Up 6 minutes       k8s_ephemeral-
demo_ephemeral-demo_default-d870ecf8-707f-437d-ae91-1af9c21413a9_0

```

范例：调试nginx服务

```

[root@master1 ~]#kubectl run mynginx --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/nginx:1.22.0
[root@master1 ~]#kubectl exec -it mynginx -- sh
# ip a
sh: 2: ip: not found
# ps
sh: 3: ps: not found
# exit
command terminated with exit code 127

[root@master1 ~]#kubectl debug -it mynginx --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0 --target=mynginx
Targeting container "mynginx". If you don't see processes from this container it
may be because the container runtime doesn't support this feature.
Defaulting debug container name to debugger-4x574.
If you don't see a command prompt, try pressing enter.
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever

```



```

2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 3a:c5:98:0c:a7:0d brd ff:ff:ff:ff:ff:ff
    inet 10.244.3.14/24 brd 10.244.3.255 scope global eth0
        valid_lft forever preferred_lft forever
/ # ps aux
PID    USER     TIME   COMMAND
   1   root         0:00 nginx: master process nginx -g daemon off;
  29  101         0:00 nginx: worker process
  30  101         0:00 nginx: worker process
  37   root         0:00 sh
  44   root         0:00 ps aux
/ # exit
Session ended, the ephemeral container will not be restarted but may be
reattached using 'kubectl attach mynginx -c debugger-lvspb -i -t' if it is still
running

[root@master1 ~]#kubectl get pod mynginx -o yaml |grep -A10 ephemeralContainers
ephemeralContainers:
- image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
  imagePullPolicy: IfNotPresent
  name: debugger-4x574
  resources: {}
  stdin: true
  targetContainerName: mynginx
  terminationMessagePath: /dev/termination-log
  terminationMessagePolicy: File
  tty: true
  nodeName: node3.wang.org

```

3.2 Pod 工作机制

3.2.1 Pod 基本原理

3.2.1.1 Pod 结构和特点

Kubernetes本质上是“以应用为中心”的现代应用基础设施，Pod是其运行应用及应用调度的最小逻辑单元

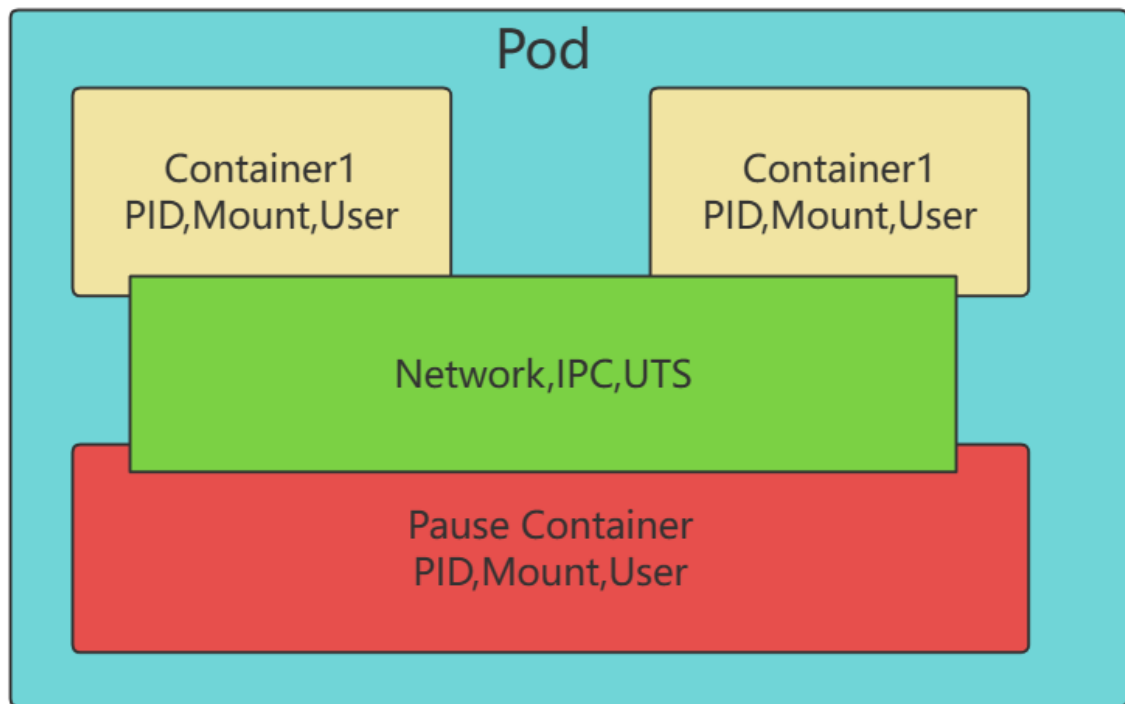
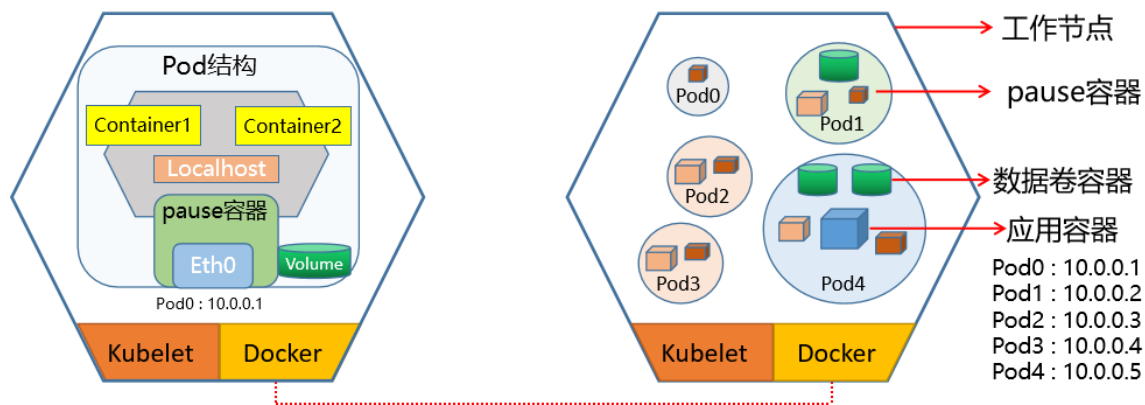
Kubernetes是通过组合容器将应用放置到一个个的逻辑单元Pod中，达到快速管理业务应用的效果

Pod是Kubernetes中最重要也最基本的概念。

交付单元

- 传统主机阶段 - 单台主机
- 容器环境阶段 - 单个容器
- 任务编排阶段 - Pod(容器集合)

完整结构



关于Pause容器

Pause容器 全称 infrastucture container (又叫infra) 基础容器

每个Pod包含了一个Pause容器,作为init pod存在, 其他pod都会从pause 容器中fork出来,即Pause容器是Pod的父容器,主要负责僵尸进程的回收管理,通过Pause容器可以使同一个Pod里面的多个容器共享存储、网络、UTS、IPC 等。

- 每个Pod里运行着一个Pause的容器, 其他容器则为业务容器, 这些业务容器共享Pause容器的网络栈和Volume挂载卷
- 每个Pod内的容器之间通信和数据交换更为高效, 在设计时可以充分利用这一特性将一组密切相关的服务进程放入同一个Pod中。
- 同一个Pod里的容器之间仅需通过localhost就能互相通信。
- Pod的IP是pause容器的地址
- Pod内部所有的应用容器, 继承Pause容器的信息

pause容器主要为每个业务容器提供以下功能

- 网络命名空间: Pod中的多个容器能够访问同一个IP和端口范围
- IPC命名空间: Pod中的多个容器能够使用SystemV IPC或POSIX消息队列进行通信
- UTS命名空间: Pod中的多个容器共享一个主机名和域名
- Volumes (共享存储卷) :Pod中的各个容器可以访问在Pod级别定义的Volumes
- PID命名空间: 默认PID是每个容器是隔离的, 但是 Pod中的不同应用程序可以看到其他应用程序的进程ID

注意: 存储卷隶属于 Pod, 而非容器

`pause` 容器的主要作用:

1. 持有 Pod 的网络命名空间

- **网络命名空间的创建与保持:** `pause` 容器是 Kubernetes 中每个 Pod 启动的第一个容器。它负责创建并持有 Pod 的网络命名空间。这意味着它为 Pod 内的其他所有容器提供了一个共享的网络环境。无论 Pod 中其他容器如何变化 (如重启或停止), 网络命名空间依然保持稳定, 因为 `pause` 容器一直在运行。
- **IP 地址的持久性:** 通过 `pause` 容器持有网络命名空间, Pod 的 IP 地址在整个 Pod 的生命周期内保持不变, 即使其中的应用容器被销毁或重新启动。

2. 持有 Pod 的其他命名空间

- **共享命名空间的持有者:** 除了网络命名空间外, `pause` 容器还持有 Pod 的其他命名空间, 如 PID 命名空间、IPC 命名空间、用户命名空间等。这使得 Pod 内的所有容器可以共享这些命名空间, 实现进程间通信、共享内存等操作。
- **进程命名空间:** `pause` 容器持有 Pod 的进程命名空间, 使得所有 Pod 中的容器可以看到彼此的进程 ID, 允许某些容器管理或监控其他容器的进程。

3. 管理 Pod 生命周期

- **生命周期的基础容器:** `pause` 容器的生命周期与整个 Pod 的生命周期绑定。只要 Pod 存在, `pause` 容器就会持续运行, 确保 Pod 的命名空间和网络环境持续存在。这使得 Pod 内的容器可以在 `pause` 容器的基础上启动、停止或重启, 而不会影响到 Pod 本身的网络 and 命名空间。
- **Pod 的退出信号:** 当 `pause` 容器终止时, 整个 Pod 也会终止。这意味着 `pause` 容器充当了 Pod 生命周期的一个基准点, 其存在与否直接决定了 Pod 的状态。

4. 轻量级资源占用

- **低资源消耗:** `pause` 容器运行一个几乎空闲的进程 (通常是一个简单的 `sleep` 或等待循环), 因此它的资源占用非常低。尽管它始终运行, 但几乎不消耗 CPU 和内存资源, 这使得它能够高效地承担起命名空间的持有任务。

5. 支持多种 CNI 插件

- **兼容性和扩展性:** Docker 的容器默认不直接支持 CNI, `pause` 容器能够与 Kubernetes 支持的各种 CNI (Container Network Interface) 插件配合工作, 不论是 Calico、Flannel、Weave, 还是其他网络插件。它为这些网络插件提供了一个稳定的网络命名空间, 使得 Pod 内容器的网络配置更为一致和可靠。

pod 资源关系特点解析

关系	解析
Pod vs 容器	一个Pod可以有多个容器，彼此间共享网络和存储资源。 每一个Pod中有一个Pause容器保存所有容器的状态，通过管理 Pause 容器，达到管理pod中所有容器的效果。 只有紧密关系的容器才可以放到一个pod中，不同业务的容器分别放在不同的pod中
Pod vs 节点	同一Pod中的容器总会被调度到相同Node节点 不同节点间Pod的通信是基于虚拟二层或三层网络技术实现的。
Pod vs Pod	普通的Pod和静态Pod(一般不用)
Pod vs 应用	每个Pod都是应用的一个(子应用)实例，有专用的ip，与容器暴露的端口组合为一个service的endpoint地址 每个service由kube-proxy转换为本地的ipvs或iptables规则
应用 vs 应用	每个service的endpoint地址由coredns组件解析为对应的服务名称 其他service的pod通过访问该名称达到应用间通信的效果
外部 vs pod	外部流量通过节点网卡进入到k8s集群，基于ipvs或iptables规则找到对应的service，进而找到对应pod地址

Pod 通信机制

- Pod内多容器通信 - 容器间通信(容器模型)借助于pause容器实现
- 单节点内多Pod通信 - 主机间容器通信(host模型),利用kube-proxy实现
- 多节点内多Pod通信 - 跨主机网络解决方案(overlay模型),利用网络插件flannel,calico等实现

范例: 查看pause容器

```
[root@master1 ~]#kubectl get pod -o wide |grep node1
pod-test1    1/1      Running   7 (7h41m ago)    14d    172.16.3.39    node1.wang.org
<none>      <none>

#有几个pod就有几个对应的pause容器
[root@node1 ~]#docker ps |grep pod-test1
701b0cb33630   ae893c58d83f
"nginx -g 'daemon of..."   8 hours ago           Up 8 hours
k8s_nginx-web01_pod-test1_default_4595942c-adb4-47a1-9782-0619bf43920a_7
2e1aa91c323d   registry.cn-hangzhou.aliyuncs.com/google_containers/pause:3.5
"/pause"       8 hours ago           Up 8 hours
k8s_POD_pod-test1_default_4595942c-adb4-47a1-9782-0619bf43920a_15
```

3.2.1.2 静态和动态 Pod

基于控制的特性Pod 主要有两类：静态pod和动态pod

- 动态Pod
 - 之前创建管理的pod都是动态pod，也是应用最广泛的pod
 - 动态Pod 直接被集群中的API Server 进行管理
- 静态pod
 - 由特定节点上的kubelet进程来管理,对于API Server 只能查看，而不能管理。
 - 在本质上与动态pod没有区别，只是在于静态pod只能在特定的节点上运行

常用于某些节点上的一些隐私操作或者核心操作，而且这些操作往往受到特定节点的场景约束，比如某些定向监控、特定操作。

kubeadm部署的集群中控制平面中组件对应的都是动态Pod,包括:etcd,kube-apiserver,kube-scheduler,kube-controller-manager

kubelet 默认会加载/etc/kubernetes/manifests/*.yaml 从而生成的静态Pod

静态pod实现方式主要有两种：配置文件或者http方式。

- 配置文件：

所谓的配置文件的方式，其实就是在特定的目录下存放我们定制好的资源对象文件，然后节点上的kubelet服务周期性的检查该目录下的所有内容，对静态pod进行增删改查。其配置方式主要有两步

- 1 定制kubelet服务定期检查配置目录
- 2 增删定制资源文件

- http方式：

- 1 准备http方式提供资源文件的web站点
- 2 工作节点的kubelet配置-manifest-url=<资源文件的url下载地址>

这两种方式实现的原理基本上是一致的，只不过一个是在本地设置资源对象文件，一个是在网络设置，我们通过http的方式来获取。

参考资料：

```
https://kubernetes.io/zh/docs/tasks/administer-cluster/static-pod/  
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/static-pod/
```

范例: 信息查看

```
#软件组成  
[root@master1 ~]#dpkg -L kubelet  
...  
/lib/systemd/system/kubelet.service  
...  
/usr/bin/kubelet  
  
#服务状态  
[root@master1 ~]#systemctl status kubelet  
• kubelet.service - kubelet: The Kubernetes Node Agent  
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: enabled)  
   Drop-In: /etc/systemd/system/kubelet.service.d  
            └─10-kubeadm.conf  
   Active: active (running)  
   ...  
#结果显示: kubelet服务，系统服务文件是kubelet.service，但是在启动服务时候，还加载了  
kubelet.service.d目录下的10-kubeadm.conf文件的内容  
  
#查看加载文件内容  
[root@master1 ~]#ls /lib/systemd/system/kubelet.service.d  
10-kubeadm.conf  
  
[root@master1 ~]# cat /lib/systemd/system/kubelet.service.d/10-kubeadm.conf
```

```
# Note: This dropin only works with kubeadm and kubelet v1.11+
[Service]
...
Environment="KUBELET_CONFIG_ARGS=--config=/var/lib/kubelet/config.yaml"
...
EnvironmentFile=--/var/lib/kubelet/kubeadm-flags.env
...
#结果显示: kubelet的配置参数主要是在/var/lib/kubelet/config.yaml文件中定义

#查看kubelet配置参数
[root@master1 ~]#grep static /var/lib/kubelet/config.yaml
staticPodPath: /etc/kubernetes/manifests

#结果显示: 该文件是一个yaml格式的, 里面设置了大量的配置参数, 其中有一项关键的就是
staticPodPath。其实我们还可以直接 在/etc/kubernetes/kubelet.conf文件中使用 --pod-
manifest-path=<the directory>的方式指定静态pod的路径。

#以上观察可以看出对于kubeadm的环境, 默认情况下已经配置好了静态pod的专用目
录/etc/kubernetes/manifests, 而我们没有看到静态pod的效果, 主要是该目录中并没有任何资源对象
文件。

#node节点查看效果, 没有静态pod
[root@node1 ~]# ls /etc/kubernetes/manifests/
[root@node1 ~]#

#master节点查看效果
[root@master ~]# ls /etc/kubernetes/manifests/
etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-
scheduler.yaml
#结果显示: 只有master节点上才有默认启动的核心操作。
```

范例: 创建和删除静态pod

```
#master查看集群当前效果
[root@master1 ~]# kubectl get pod
No resources found.

#直接在node3节点的/etc/kubernetes/manifests 目录下创建资源对象文件即可
[root@node3 ~]#cat >/etc/kubernetes/manifests/pod-static.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: pod-static
spec:
  containers:
  - name: nginx01
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.22.0
EOF

#保存文件后, 不apply该文件, 直接回到master节点上检查效果
[root@master1 ~]#kubectl get pod pod-static-master1.wang.org
NAME                                READY    STATUS    RESTARTS   AGE
pod-static-master1.wang.org        1/1      Running   0           58s
#结果显示: 静态pod目录, 只要配置文件发生变化, 立刻都会显示出来。

#删除静态pod该文件
[root@master1 ~]#kubectl get pod pod-static-master1.wang.org -o wide
```

NAME		READY	STATUS	RESTARTS	AGE	IP
pod-static-master1.wang.org	1/1	Running	0	56s	172.16.7.36	
node3.wang.org	<none>	<none>				

#无法用下面方式删除静态pod

```
[root@master1 ~]# kubectl delete pod pod-static-master1.wang.org
```

#删除后会自动重新创建

```
[root@master1 ~]# kubectl get pod pod-static-master1.wang.org -o wide
```

NAME		READY	STATUS	RESTARTS	AGE	IP
pod-static-master1.wang.org	1/1	Running	0	56s	172.16.7.36	
node3.wang.org	<none>	<none>				

#结果显示:由于删除pod的增删主要是基于节点上的kublet来实现的,重新创建的pod的ip什么的属性都没发生变化。

#编辑该对象

```
[root@master ~]# kubectl edit pod pod-static-master1.wang.org
```

#检查效果

```
[root@master ~]# kubectl get pod -o wide
```

#结果显示:静态pod没有办法被kubectl进行编辑

#回到node3节点上移除资源定义文件

```
[root@node3 ~]# mv /etc/kubernetes/manifests/pod-static.yaml /tmp/
```

#回到master节点上,查看效果

```
[root@master ~]# kubectl get pod
```

No resources found.

#结果显示:只要静态pod的文件一旦有变动,在集群中立刻都能看到变化

3.2.1.3 Pod 核心思想

Kubernetes资源对象Pod在它实现的内部有一套独有的设计理念和核心思想,主要有应用交付、资源调度、实现细节三个方面

- 应用交付

Kubernetes是一个自动化的容器管理平台,存在的目的就是快速管理各种业务应用。所以它所面对的本质就是业务应用。

随着应用的进一步拆分和微服务架构的设计,一个应用被拆分成了多个独立的子模块应用。

虽然Kubernetes将这些应用的各个子模块单独部署在容器上,但是Kubernetes不再是对一个个子应用容器进行管理,它是将这些子应用容器放在一个逻辑单元pod中,而这个逻辑单元针对的是应用或服务,通过对逻辑单元的操作,达到对应用快速管理,从而实现业务的快速交付。

所以说Pod面向的是应用或服务,而不是具体的应用容器或者子应用容器。

- 资源管理

kubernetes是一个分布式的资源管理平台，分布的特点就是资源分散，所以Kubernetes的应用，肯定会出现业务应用的不同模块被分配到多个Node工作节点，那么随之而来，就会出现两种问题：

- 1)需要设计一个复杂的资源调度算法，根据业务需求，从不同Node上获取所有特定的数据
- 2)强耦合的业务应用，被分配到了不同节点，可能造成不可预测的功能缺失或潜在问题

所以，与其花费大力气解决上述两个问题，还不如把这些功能直接打包到一个逻辑单元pod中：

对内：所有的子应用容器可以在Pod内部进行单独管理，即使pod在调度到其他Node节点，Pod内部的所有子应用容器仍可以正常运行。

对外：对于整个业务系统来说，面向的是一个个的应用或业务功能(即Pod)，可以轻松实现管理。

• 实现细节

应用业务下放权力到Pod内部了，那么Pod如何管理子应用呢？

通信机制：Pod内有个Pause容器，它有独立的ip地址，其他子应用容器基于container网络模式实现统一的对外服务，大大简化了关联业务容器之间的通信问题。

存储机制：Pod内有专用的数据存储资源对象，其他子应用容器共享该数据卷，实现多容器数据信息的统一存储。

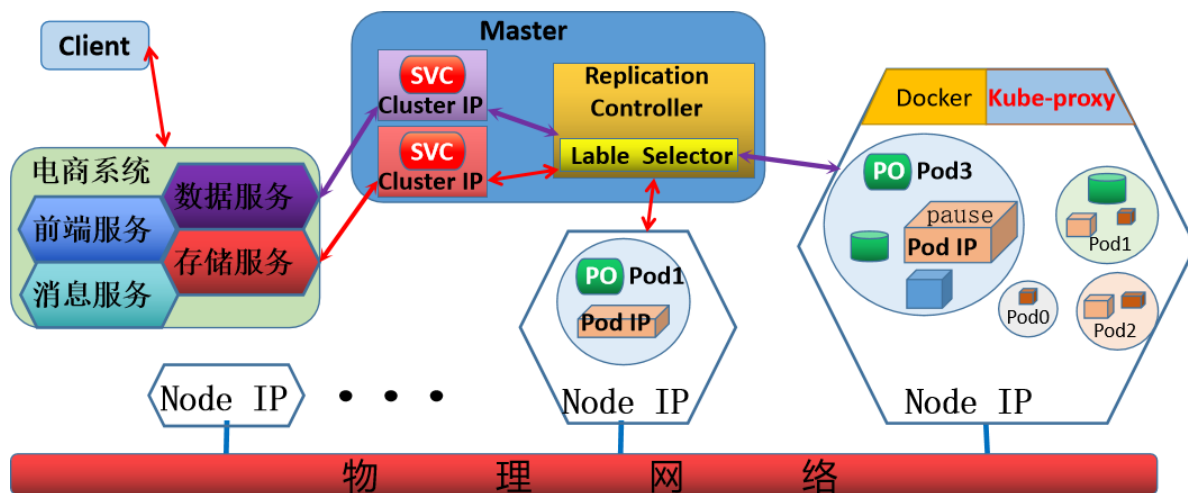
管理机制：所有子应用容器的信息都保存在pause容器中，通过对pause容器的管理，达到管理所有容器效果。

3.2.2.3 Pod 管理机制

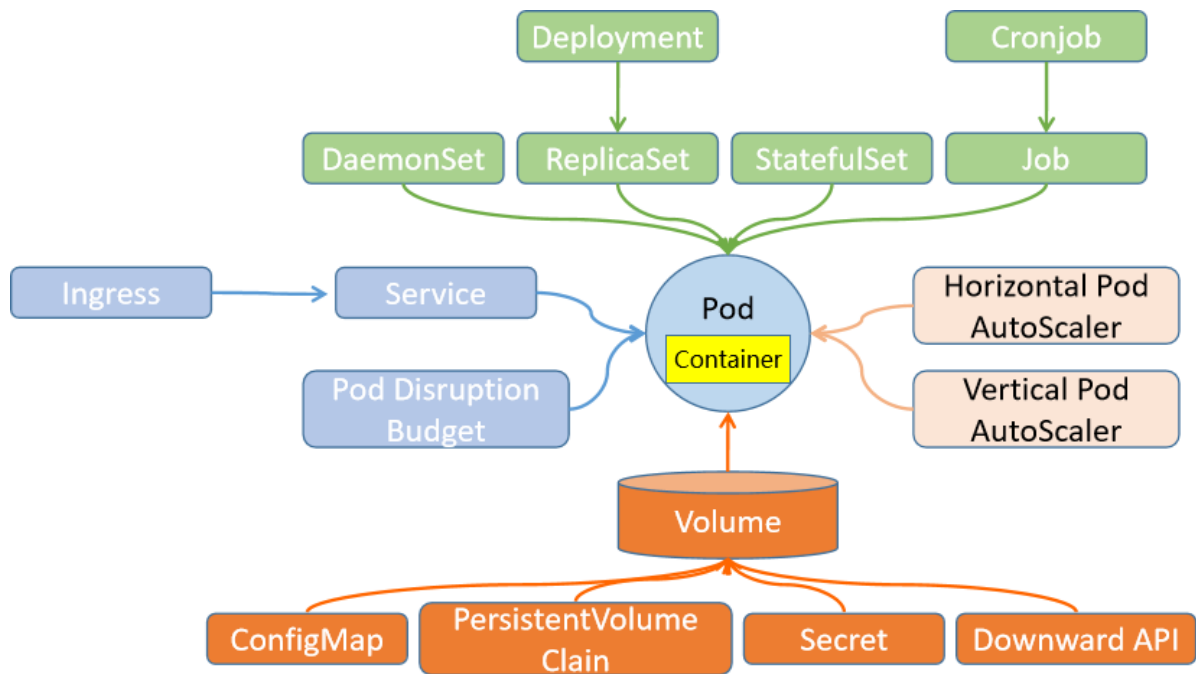
Kubernetes 集群通过pod实现了大量业务应用容器的统一管理

而Kubernetes 也提供了大量的资源对象来对pod进行管理：

- 通过控制器比如deployment来确保 pod的运行和数量符合用户的期望状态
- 通过网络资源比如service来确保pod的应用服务可以被外部的服务来进行访问

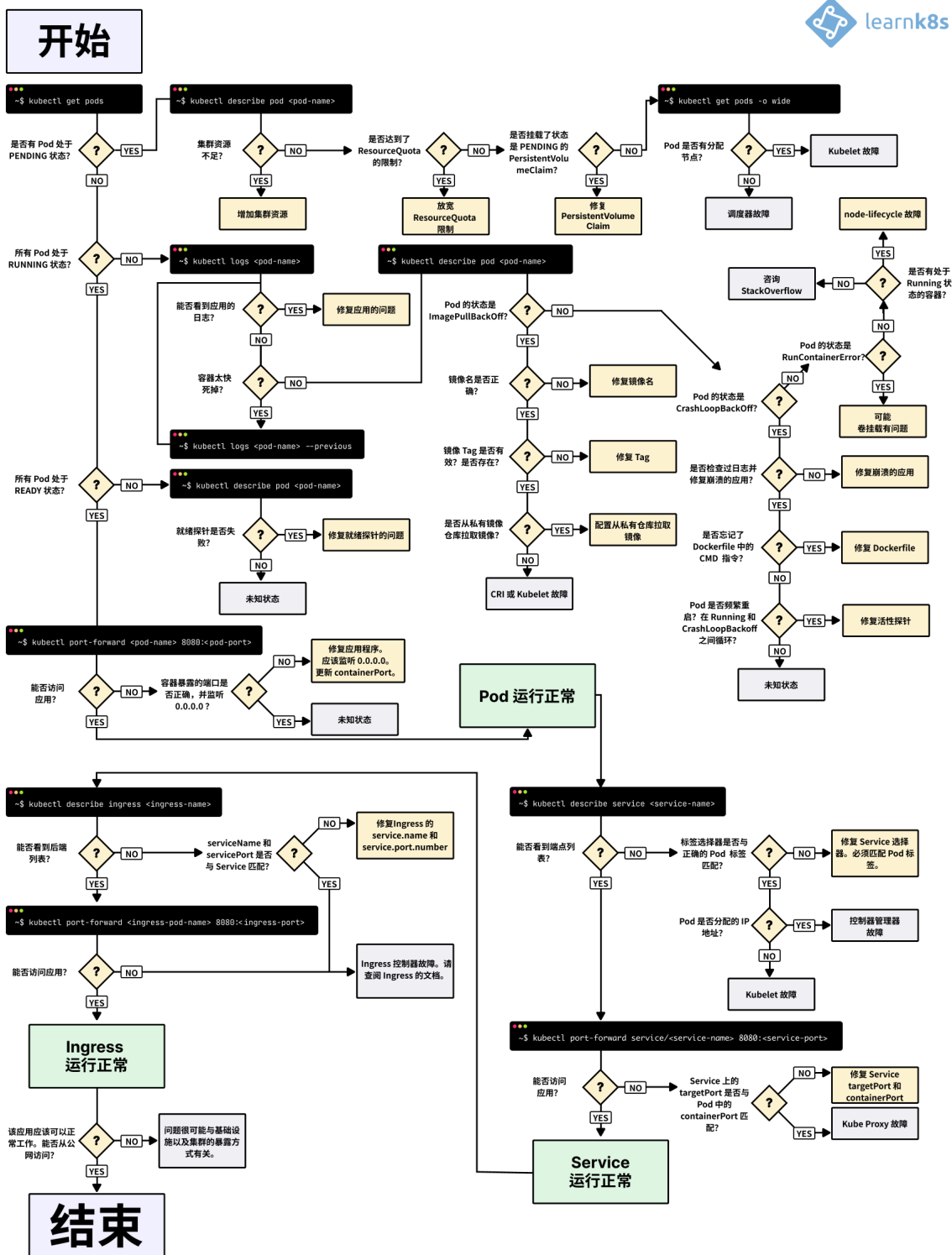


Pod 相关的资源对象



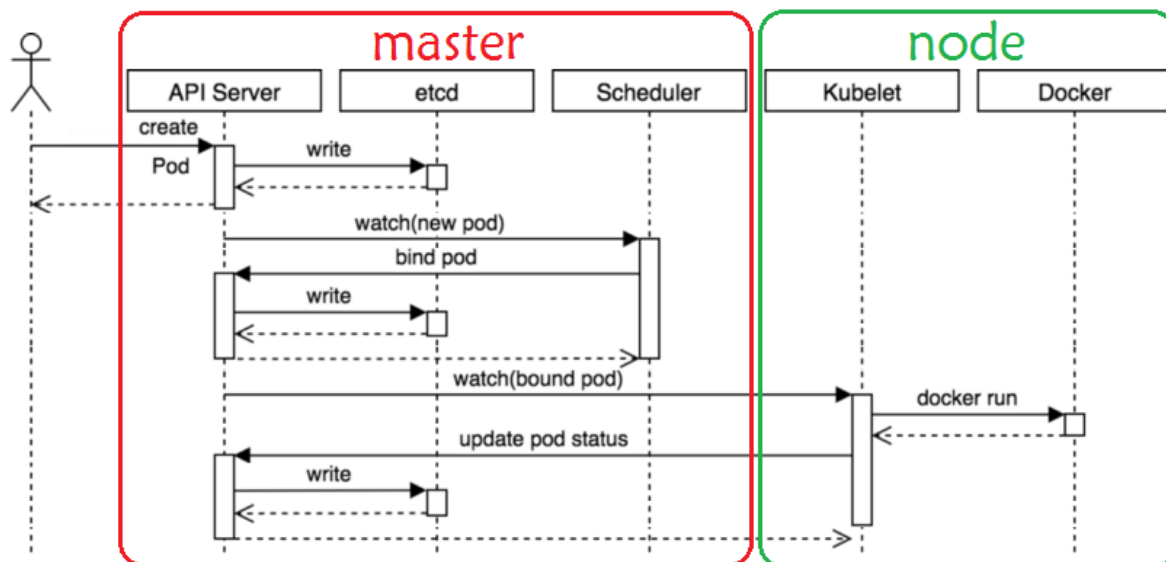
- 工作负载型资源: Pod、Deployment、DaemonSet、Replica、StatefulSet、Job、Cronjob、Operator
- 服务发现和负载均衡资源: Service、Ingress
- 配置和存储: configMap、Secret、PersistentVolume、PersistentVolumeChain、DownwardAPI
- 资源隔离权限控制资源: namespace、nodes、clusterroles、Roles
- 动态调整资源: HPA、VPA

3.2.2 Pod 工作流程



3.2.2.1 Pod 创建流程

Pod 创建流程示意图

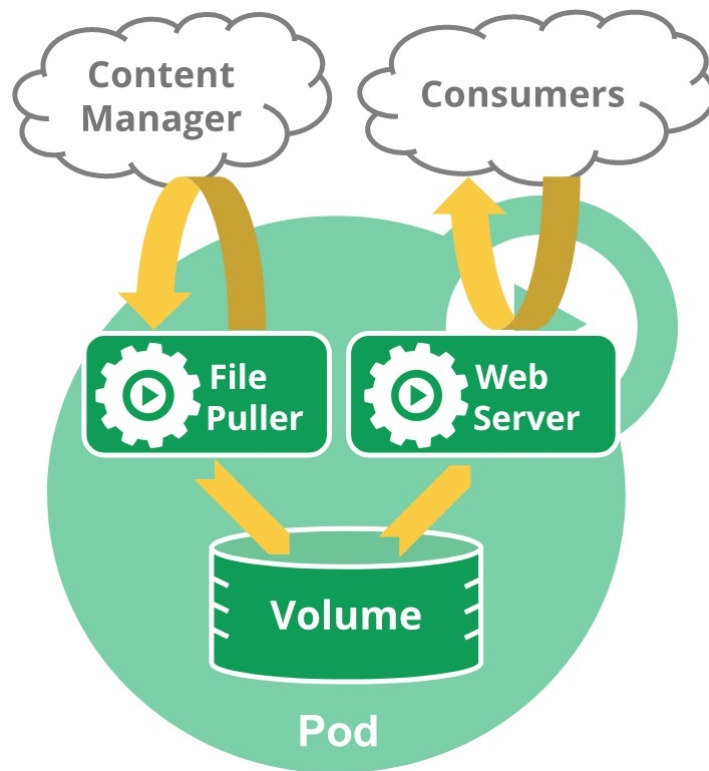


Pod 创建流程

- 用户可以通过多种方式向 Master节点上的 API Server 发起创建一个 Pod 的请求,检查权限通过后才允许继续
- API Server 将该信息写入 etcd 后,并同步到其它etcd数据库
- etcd 通过Watch机制发现数据变化则主动通知给所有API Server, API Server再返回成功结果给用户
- Scheduler 通过Watch检测到 API Server上有建立Pod请求更新, 开始通知 API Server 调度该Pod 请求到合适的Node,这期间存在选择哪个合适节点的复杂过程,
- Scheduler 通知API Server 更新信息到所有 etcd, etcd 返回结果给API Server
- API Server 通知 kubelet 有新的 Pod 调度, kubelet 通过容器引擎运行该 Pod 对象
- kubelet 通过 container runtime 取到 Pod 状态, 并同步信息到 API Server, 最终由 API Server更新信息到 etcd

3.2.2.2 Pod 生命周期

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>
<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-pod-initialization/>
<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/attach-handler-lifecycle-event/>
<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/>



和一个个独立的应用容器一样，Pod 也被认为是相对临时性（而不是长期存在）的实体。

Pod 会被创建、赋予一个唯一的 ID (UID)，并被调度到节点，并在终止（根据重启策略）或删除之前一直运行在该节点。

如果一个节点死掉了，调度到该节点的 Pod 也被计划在给定超时期限结束后删除。

Pod 自身不具有自愈能力。如果 Pod 被调度到某节点而该节点之后失效，Pod 会被删除。类似地，Pod 无法在因节点资源耗尽或者节点维护而被驱逐期间继续存活。

Kubernetes 使用一种高级抽象来管理这些相对而言可随时丢弃的 Pod 实例，称作控制器。

任何给定的 Pod（由 UID 定义）从不会被“重新调度（rescheduled）”到不同的节点；

相反，这一 Pod 可以被一个新的、几乎完全相同的 Pod 替换掉。

如果需要，新 Pod 的名字可以不变，但是其 UID 会不同。

如果某资源对象声称其生命期与某 Pod 相同，例如存储卷，这就意味着该对象在此 Pod（UID 亦相同）存在期间也一直存在。

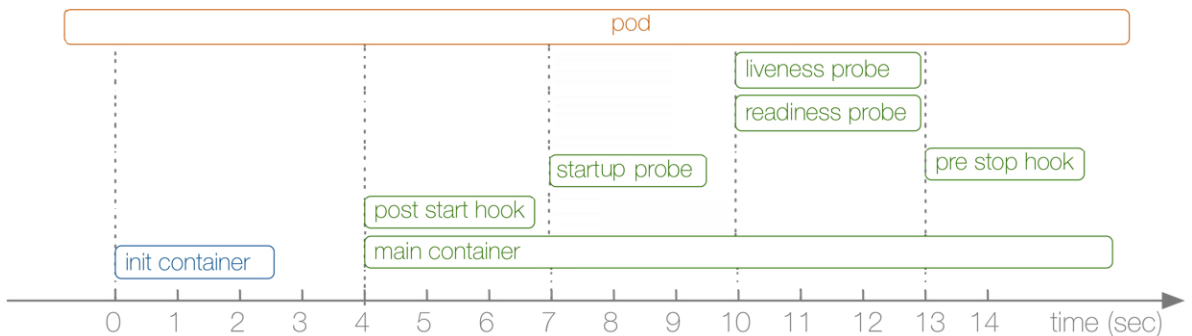
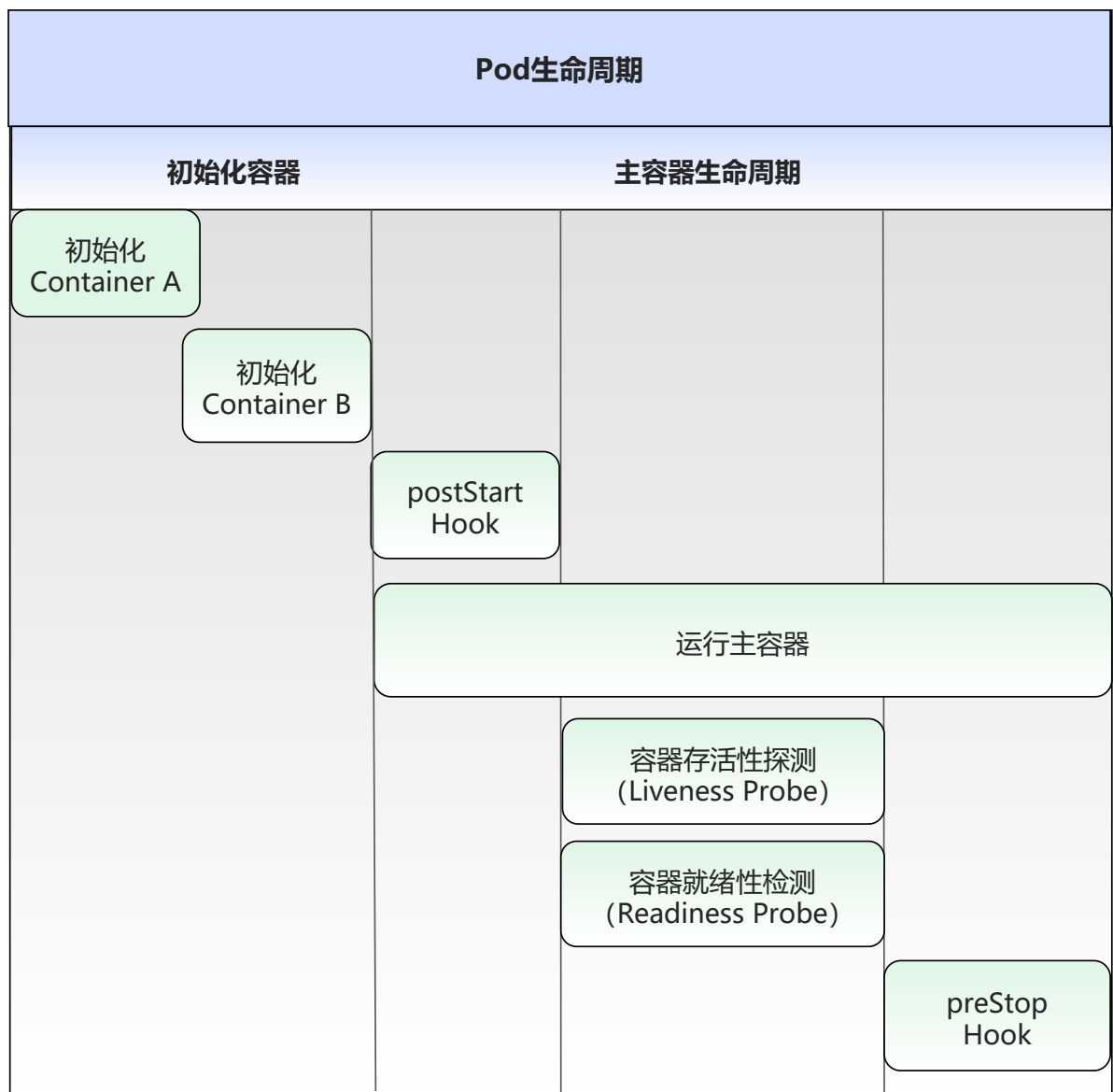
如果 Pod 因为任何原因被删除，甚至相同的替代 Pod 被创建时，这个相关的对象（例如这里的卷）也会被删除并重建。

3.2.2.2.1 Pod 生命令周期流程

公司的项目中业务应用都工是有生命周期的

Kubernetes是基于业务应用或服务进行统一管理的平台，每一个应用服务对应一个Pod，所以伴随着业务应用Pod也有相应的生命周期

Pod 生命令周期流程



3.2.2.2.2 启动 Pod 流程

1. 创建指令到送到 api server
2. 通知 scheduler 调度此请求到合适的节点
3. init 容器

初始化容器，独立于主容器之外，即和主容器是隔离的

Pod可以拥有任意数量的init容器，init顺序执行。最后一个执行完成后，才启动主容器。

init 容器不支持探针检测功能

它主要是为了主容器准备运行环境的功能，比如：给主容器准备配置文件,向主容器的存储卷写入数据，然后将存储卷挂载到主容器上,下载相关资源,监测主容器依赖服务等

4. 启动后钩子PostStart: 与主容器同时启动

5. 状态监测

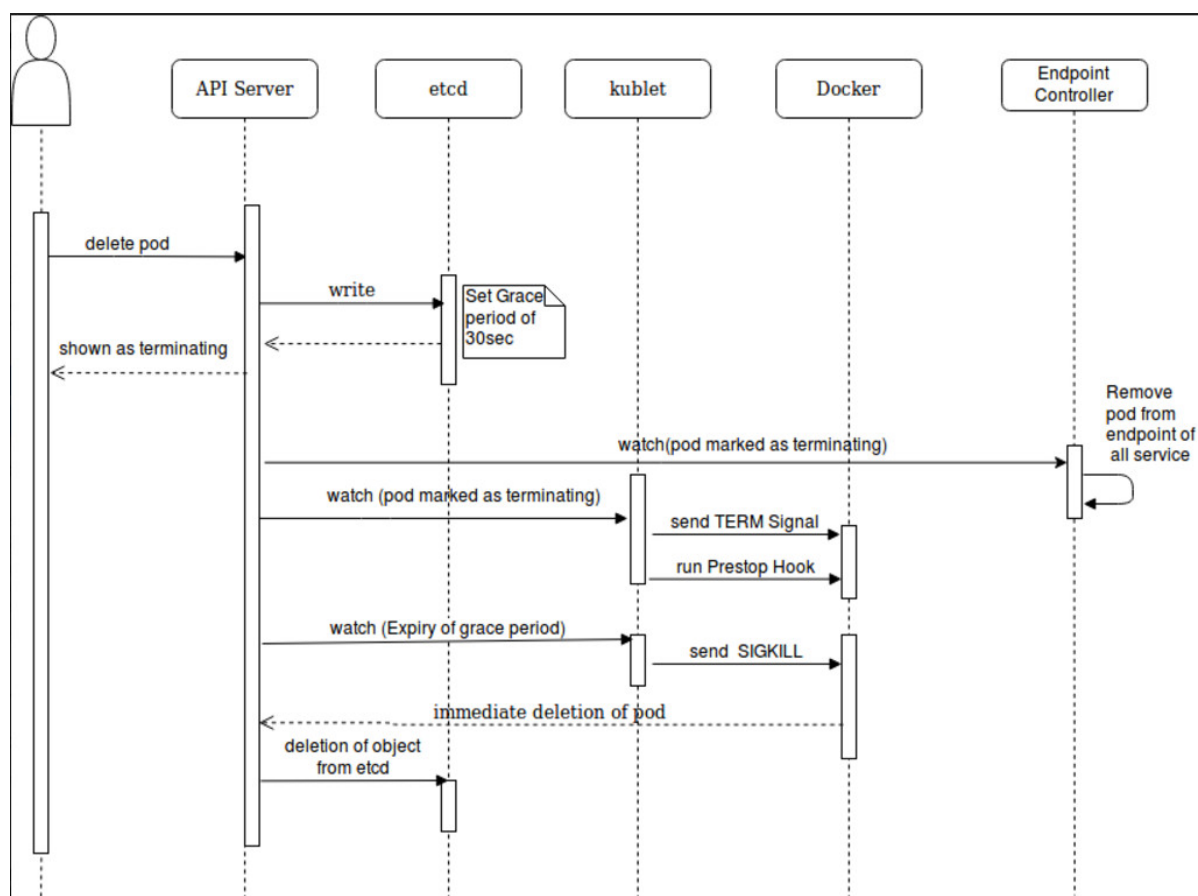
- Startup probe : 启动探针
- Liveness probe: 判断当前pod是否处于存活状态,是Readiness存活的前提,对应READY状态的m/n的n值
- Readiness probe: 判断当前pod的主应用容器是否可以正常的对外提供服务,只有Liveness 为存活,Readiness才有可能存活,对应READY状态的m/n的m值

6. Service 关联 Pod

7. 接收用户请求

3.2.2.2.3 关闭 Pod 流程

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#pod-termination>



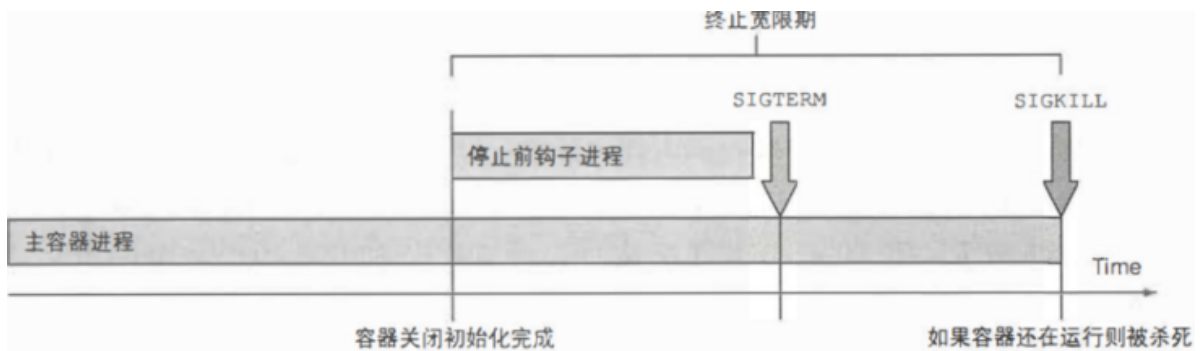
1. 你使用 `kubectl` 工具手动删除某个特定的 Pod，而该 Pod 的体面终止限期是默认值（30 秒）。
2. API 服务器中的 Pod 对象被更新，记录涵盖体面终止限期在内 Pod 的最终死期，超出所计算时间点则认为 Pod 已死（dead）。如果你使用 `kubectl describe` 来查验你正在删除的 Pod，该 Pod 会显示为 "Terminating"（正在终止）。在 Pod 运行所在的节点上：`kubelet` 一旦看到 Pod 被标记为正在终止（已经设置了体面终止限期），`kubelet` 即开始本地的 Pod 关闭过程。
 - 如果 Pod 中的容器之一定义了 `preStop` 回调，`kubelet` 开始在容器内运行该回调逻辑。如果超出体面终止限期时，`prestop` 回调逻辑 仍在运行，`kubelet` 会请求给予该 Pod 的宽限期一次性增加 2 秒钟。

说明： 如果 `prestop` 回调所需要的时间长于默认的体面终止限期，你必须修改 `terminationGracePeriodSeconds` 属性值来使其正常工作。

- `kubelet` 接下来触发容器运行时发送 TERM 信号给每个容器中的进程 1。

说明： Pod 中的容器会在不同时刻收到 TERM 信号，接收顺序也是不确定的。如果关闭的顺序很重要，可以考虑使用 `prestop` 回调逻辑来协调。

- 与此同时，`kubelet` 启动体面关闭逻辑，控制面会将 Pod 从对应的端点列表（以及端点切片列表，如果启用了的话）中移除，过滤条件是 Pod 被对应的服务和其他工作负载资源 不再将关闭进程中的 Pod 视为合法的、能够提供服务的副本。关闭动作很慢的 Pod 也无法继续处理请求数据，因为负载均衡器（例如服务代理）已经在终止宽限期开始的时候将其从端点列表中移除。
- 超出终止宽限期限时，`kubelet` 会触发强制关闭过程。容器运行时向 Pod 中所有容器内仍在运行的进程发送 `SIGKILL` 信号。`kubelet` 也会清理隐藏的 `pause` 容器，如果容器运行时使用了这种容器的话。
- `kubelet` 触发强制从 API 服务器上删除 Pod 对象的逻辑，并将体面终止限期设置为 0（这意味着马上删除）
- API 服务器删除 Pod 的 API 对象，从任何客户端都无法再看到该对象



设置终止宽限期

`spec.terminationGracePeriod`,

#默认为30s,此值为优雅终止宽限期

#如果程序的关闭时间和在Pod配置的`preStop` Hook之和超过30秒，应将`terminationGracePeriodSeconds`重新设置，调整为大于30秒，确保容器优雅退出。

#删除命令: `kubectl delete pod mypod --grace-period=5`

#强制删除: `kubectl delete pod mypod --grace-period=0 --force`

Pod终止流程和网络规则更新流程

在删除Pod时K8s会同时进行Pod终止和网络规则更新两个主要的流程。Pod终止流程和网络更新流程如下。

Pod终止流程

- `kube-apiserver`收到Pod删除请求，将Pod标记为 `Terminating` 状态。
- 如果Pod定义了`preStop` Hook，将执行`preStop` Hook。
- K8s集群向容器发送`SIGTERM`信号。
- 等待容器停止，或等待Pod删除宽限期超时。

说明

Pod中容器删除宽限期 `terminationGracePeriodSeconds` 默认为30秒。

- 超时Pod删除宽限期后容器仍未终止，K8s发送`SIGKILL`信号给容器。
- Pod被完全删除。

网络规则更新流程

1. kube-apiserver收到Pod删除请求，将Pod标记为 `Terminating` 状态。
2. Endpoint Controller从Endpoint对象中删除Pod的IP。

范例：

```
[root@master1 ~]#kubectl explain pod.spec.terminationGracePeriodSeconds
KIND:      Pod
VERSION:   v1

FIELD:     terminationGracePeriodSeconds <integer>

DESCRIPTION:
  Optional duration in seconds the pod needs to terminate gracefully. May be
  decreased in delete request. Value must be non-negative integer. The value
  zero indicates stop immediately via the kill signal (no opportunity to shut
  down). If this value is nil, the default grace period will be used instead.
  The grace period is the duration in seconds after the processes running in
  the pod are sent a termination signal and the time when the processes are
  forcibly halted with a kill signal. Set this value longer than the expected
  cleanup time for your process. Defaults to 30 seconds.

[root@master1 ~]#kubectl explain
pod.spec.containers.livenessProbe.terminationGracePeriodSeconds
KIND:      Pod
VERSION:   v1

FIELD:     terminationGracePeriodSeconds <integer>

DESCRIPTION:
  Optional duration in seconds the pod needs to terminate gracefully upon
  probe failure. The grace period is the duration in seconds after the
  processes running in the pod are sent a termination signal and the time when
  the processes are forcibly halted with a kill signal. Set this value longer
  than the expected cleanup time for your process. If this value is nil, the
  pod's terminationGracePeriodSeconds will be used. Otherwise, this value
  overrides the value provided by the pod spec. Value must be non-negative
  integer. The value zero indicates stop immediately via the kill signal (no
  opportunity to shut down). This is a beta field and requires enabling
  ProbeTerminationGracePeriod feature gate. Minimum value is 1.
  spec.terminationGracePeriodSeconds is used if unset.
```

示例：

```
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-
liveness-readiness-startup-
probes/#%E6%8E%A2%E9%92%88%E5%B1%82%E9%9D%A2%E7%9A%84-
terminationgraceperiodseconds

spec:
  terminationGracePeriodSeconds: 3600 # Pod 级别设置
  containers:
  - name: test
    image: ...

  ports:
```



```

- name: liveness-port
  containerPort: 8080
  hostPort: 8080

livenessProbe:
  httpGet:
    path: /healthz
    port: liveness-port
  failureThreshold: 1
  periodSeconds: 60
  # 重载 Pod 级别的 terminationGracePeriodSeconds
  terminationGracePeriodSeconds: 60

```

3.2.2.2.4 两种钩子PostStart和PreStop

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/attach-handler-lifecycle-event/>

根据上面Pod的启动流程，当容器中的进程启动前或者容器中的进程终止之前都会有一些额外的动作执行，这是由kubelet所设置的，在这里，我们称之为 pod hook。

对于Pod的流程启动，主要有两种钩子：

- postStart，容器创建完成后立即运行，不保证一定会于容器中ENTRYPOINT之前运行,而Init Container可以实现
- preStop，容器终止操作之前立即运行，在其完成前会阻塞删除容器的操作调用

关于钩子函数的执行主要有两种方式：

- Exec，在钩子事件触发时，直接在当前容器中运行由用户定义的命令,用于执行一段特定的命令，不过要注意的是该命令消耗的资源会被计入容器。
- HTTP，在当前容器中向某URL发起HTTP请求

3.2.2.2.4.1 Poststart 钩子

属性信息：kubect explain pods.spec.containers.lifecycle

```

[root@master1 ~]#kubectl explain pods.spec.containers.lifecycle
KIND:      Pod
VERSION:   v1

FIELD: lifecycle <Lifecycle>

DESCRIPTION:
  Actions that the management system should take in response to container
  lifecycle events. Cannot be updated.
  Lifecycle describes actions that the management system should take in
  response to container lifecycle events. For the PostStart and PreStop
  lifecycle handlers, management of the container blocks until the action is
  complete, unless the container process fails, in which case the handler is
  aborted.

FIELDS:
  postStart <LifecycleHandler>
    PostStart is called immediately after a container is created. If the handler
    fails, the container is terminated and restarted according to its restart
    policy. Other management of the container blocks until the hook completes.

```

More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

preStop <LifecycleHandler>

PreStop is called immediately before a container is terminated due to an API request or management event such as liveness/startup probe failure, preemption, resource contention, etc. The handler is not called **if** the container crashes or exits. The Pod's **termination grace period countdown** begins before the PreStop hook is executed. Regardless of the outcome of the handler, the container will eventually terminate within the Pod's **termination grace period** (unless delayed by finalizers). Other management of the container blocks **until** the hook completes or **until** the termination grace period is reached. More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

```
[root@master1 ~]#kubectl explain pods.spec.containers.lifecycle.postStart
```

KIND: Pod

VERSION: v1

RESOURCE: postStart <Object>

DESCRIPTION:

PostStart is called immediately after a container is created. If the handler fails, the container is terminated and restarted according to its **restart** policy. Other management of the container blocks **until** the hook completes. More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

LifecycleHandler defines a specific action that should be taken **in** a lifecycle hook. One and only one of the fields, except TCPSocket must be specified.

FIELDS:

exec <Object>

Exec specifies the action to take.

httpGet <Object>

HTTPGet specifies the http request to perform.

tcpSocket <Object>

Deprecated. TCPSocket is NOT supported as a LifecycleHandler and kept **for** the backward compatibility. There are no validation of this field and lifecycle hooks will fail **in** runtime when tcp handler is specified.

```
[root@master1 ~]#kubectl explain pods.spec.initContainers.lifecycle
```

KIND: Pod

VERSION: v1

RESOURCE: lifecycle <Object>

DESCRIPTION:

Actions that the management system should take **in** response to container lifecycle events. Cannot be updated.

Lifecycle describes actions that the management system should take **in** response to container lifecycle events. For the PostStart and PreStop lifecycle handlers, management of the container blocks **until** the action is complete, unless the container process fails, **in** which case the handler is aborted.

FIELDS:

postStart <Object>

PostStart is called immediately after a container is created. If the handler fails, the container is terminated and restarted according to its **restart** policy. Other management of the container blocks **until** the hook completes. More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

preStop <Object>

PreStop is called immediately before a container is terminated due to an API request or management event such as liveness/startup probe failure, preemption, resource contention, etc. The handler is not called **if** the container crashes or exits. The Pod's **termination grace period countdown** begins before the PreStop hook is executed. Regardless of the outcome of the handler, the container will eventually terminate within the Pod's **termination grace period** (unless delayed by finalizers). Other management of the container blocks **until** the hook completes or **until** the termination grace period is reached. More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

范例:

```
[root@master1 ~]#cat pod-poststart.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-poststart
spec:
  containers:
  - name: busybox
    #image: busybox:1.32.0
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    lifecycle:
      postStart:
        exec:
          command: ["/bin/sh", "-c", "echo lifecycle poststart at $(date) >
/tmp/poststart.log"]
          command: ['sh', '-c', 'echo The app is running at $(date) && sleep 3600']

#启动文件
[root@master1 ~]#kubectl apply -f pod-poststart.yaml
pod/pod-poststart created

[root@master1 ~]#kubectl get pod
NAME           READY   STATUS    RESTARTS   AGE
pod-poststart  1/1     Running   0           3s

#查看效果
```

```
[root@master1 ~]#kubectl exec pod-poststart -- ls /tmp/
poststart.log

#观察Pod启动时运行的命令和poststart定义的指令是同时执行的
[root@master1 ~]#kubectl logs pod-poststart
lifecycle poststart at Mon Jul 15 09:37:05 UTC 2024

[root@master1 ~]#kubectl exec pod-poststart -- cat /tmp/poststart.log
lifecycle poststart at Mon Jul 15 09:37:05 UTC 2024

#清理环境
[root@master1 ~]#kubectl delete -f pod-poststart.yaml
pod "pod-poststart" deleted
```

3.2.2.2.4.2 Prestop 钩子

功能：实现pod对象移除之前，需要做一些清理工作，比如：释放资源，解锁等

实现方式：

- exec
- httpGet
- tcpSocket

钩子处理程序的日志不会在 Pod 事件中公开。如果处理程序由于某种原因失败，它将播放一个事件。对于 PostStart，这是 FailedPostStartHook 事件，对于 PreStop，这是 FailedPreStopHook 事件。

可以通过运行 `kubectl describe pod <pod_name>` 命令来查看这些事件。

```
[root@master1 ~]#kubectl explain pods.spec.containers.lifecycle.preStop
KIND:      Pod
VERSION:   v1

RESOURCE: preStop <Object>

DESCRIPTION:
    PreStop is called immediately before a container is terminated due to an
    API request or management event such as liveness/startup probe failure,
    preemption, resource contention, etc. The handler is not called if the
    container crashes or exits. The Pod's termination grace period countdown
    begins before the PreStop hook is executed. Regardless of the outcome of
    the handler, the container will eventually terminate within the Pod's
    termination grace period (unless delayed by finalizers). Other management
    of the container blocks until the hook completes or until the termination
    grace period is reached. More info:
    https://kubernetes.io/docs/concepts/containers/container-lifecycle-
    hooks/#container-hooks

    LifecycleHandler defines a specific action that should be taken in a
    lifecycle hook. One and only one of the fields, except TCPSocket must be
    specified.

FIELDS:
    exec <Object>
        Exec specifies the action to take.

    httpGet <Object>
        HTTPGet specifies the http request to perform.
```

tcpSocket <Object>

Deprecated. TCP Socket is NOT supported as a LifecycleHandler and kept for the backward compatibility. There are no validation of this field and lifecycle hooks will fail in runtime when tcp handler is specified.

```
[root@master1 ~]#kubectl explain pods.spec.initContainers.lifecycle.preStop
```

KIND: Pod

VERSION: v1

RESOURCE: preStop <Object>

DESCRIPTION:

PreStop is called immediately before a container is terminated due to an API request or management event such as liveness/startup probe failure, preemption, resource contention, etc. The handler is not called if the container crashes or exits. The Pod's termination grace period countdown begins before the PreStop hook is executed. Regardless of the outcome of the handler, the container will eventually terminate within the Pod's termination grace period (unless delayed by finalizers). Other management of the container blocks until the hook completes or until the termination grace period is reached. More info:

<https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/#container-hooks>

LifecycleHandler defines a specific action that should be taken in a lifecycle hook. One and only one of the fields, except TCP Socket must be specified.

FIELDS:

exec <Object>

Exec specifies the action to take.

httpGet <Object>

HTTPGet specifies the http request to perform.

tcpSocket <Object>

Deprecated. TCP Socket is NOT supported as a LifecycleHandler and kept for the backward compatibility. There are no validation of this field and lifecycle hooks will fail in runtime when tcp handler is specified.

范例:

#由于默认情况下，删除的动作和日志我们都没有办法看到，那么我们这里采用一种间接的方法，在删除动作之前，给本地目录创建第一个文件，输入一些内容

```
[root@master1 ~]#cat pod-prestop.yaml
```

apiVersion: v1

kind: Pod

metadata:

name: pod-prestop

spec:

volumes:

- name: vol-prestop

hostPath:

path: /tmp

```

containers:
- name: prestop-pod-container
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
  volumeMounts:
  - name: vol-prestop
    mountPath: /tmp
  command: ['sh', '-c', 'echo The app is running at $(date) && sleep 3600']
  lifecycle:
    postStart:
      exec:
        command: ['/bin/sh', '-c', 'echo lifecycle poststart at $(date) >
/tmp/poststart.log']
    preStop:
      exec:
        command: ['/bin/sh', '-c', 'echo lifecycle prestop at $(date) >
/tmp/prestop.log']

#运行前确定所有节点目录文件列表
[root@node1 ~]#ls /tmp

#启动Pod
[root@master1 ~]#kubectl apply -f pod-prestop.yaml

#查看到此pod运行在node1节点上
[root@master1 ~]#kubectl get pod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE
NOMINATED NODE      READINESS GATES
pod-prestop         1/1     Running   0           25s   172.16.3.22     node1.wang.org
<none>              <none>

#查看到node1的目录下生成文件
[root@node1 ~]#ls /tmp
poststart.log

[root@node1 ~]#cat /tmp/poststart.log
lifecycle poststart at Mon Jul 15 09:40:05 UTC 2024

#删除pod
[root@master1 ~]#kubectl delete -f pod-prestop.yaml

#再查看文件
[root@node1 ~]#ls /tmp
poststart.log prestop.log
[root@node1 ~]#cat /tmp/prestop.log
lifecycle prestop at Mon Jul 15 09:42:15 UTC 2024

```

范例: 后端Pod滚动升级时平滑下线Pod

```

#清单文件
cat pod-prestop-demo.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prestop-demo
spec:
  replicas: 3
  selector:

```

```

matchLabels:
  app: prestop-demo
template:
  metadata:
    labels:
      app: prestop-demo
  spec:
    containers:
      - name: prestop-demo
        image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
        ports:
          - containerPort: 80
        lifecycle: # 设置preStop Hook函数，使kube-apiserver等待10秒后再向Pod发送
SIGTERM信号。
          preStop: # preStop钩子设置。
            exec: # 通过执行命令的方式来实现preStop操作。
              command: # 定义要执行的命令。
                - /bin/sh
                - -c
                - "sleep 10" # 执行sleep操作，暂停10秒。
          terminationGracePeriodSeconds: 45 # 设置Pod删除宽限期，一定要大于preStop的执行
时间，否则会导致preStop执行没有完成，就立即删除Pod
---
apiVersion: v1
kind: Service
metadata:
  name: prestop-demo-svc
spec:
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  selector:
    app: prestop-demo
  type: NodePort

#test.sh
#!/bin/bash
HOST="SVC_IP"
#HOST="demo.wang.org"
printf "Response Code|| TIME \n" >> log.txt

while true; do
  #RESPONSE=$(curl -H Host:$HOST -s -o /dev/null -w "%{http_code}" -m 1
http://$HOST/)
  RESPONSE=$(curl -H Host:$HOST -s -o /dev/null -w "%{http_code}" -m 1
http://$HOST/)
  TIMESTAMP=$(date +%Y-%m-%d_%H:%M:%S)
  echo "$TIMESTAMP - $RESPONSE" >> log.txt
  sleep 1
done

#运行test.sh
bash test.sh

#重新部署应用，触发Pod滚动升级

```

```
kubectl rollout restart deploy prestop-demo
```

```
#查看log.txt
```

```
cat log.txt
```

3.2.2.3 Pod 状态

<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

3.2.2.3.1 Pod 状态说明

- 正常情况:

在Kubernetes集群中，真正工作的是Node节点，所以业务的子应用也就是Pod，肯定会被分配到相应的Node节点上。Pod一旦绑定到Node节点，正常情况下不会移动，即便Node节点发生重启。

Pod作为一种资源对象，在整个业务生命周期中，会随着操作而出现五种状态：pending, running, succeeded, failed, Unknown

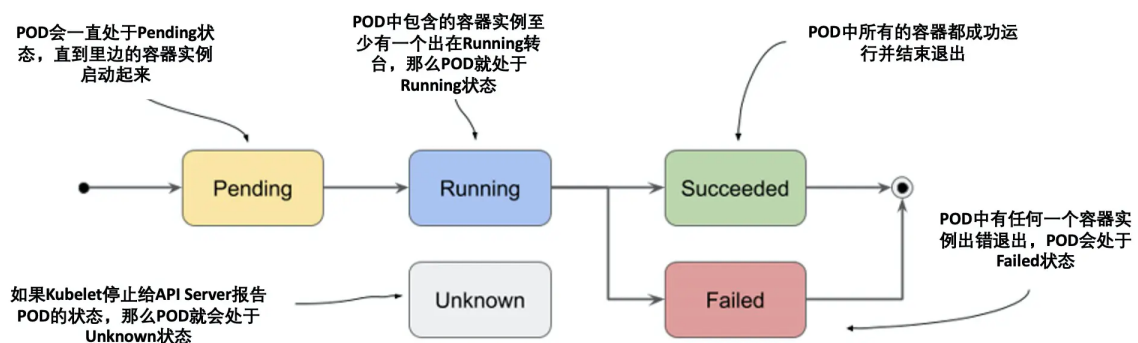
业务应用作为系统的一部分，会有相应的资源对象对它们进行统一管理，所以不建议手工单独创建一个Pod

- 异常情况:

业务运行过程中不可避免会出现意外，这个时候有三种策略对Pod进行管理：OnFailure, Never 和Always（默认）

3.2.2.3.2 Pod phase 阶段（相位）

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#pod-phase>



Value	Description
Pending	Pod 已被 Kubernetes 集群接受，但一个或多个容器尚未设置并准备好运行。这包括 Pod 等待调度所花费的时间以及通过网络下载容器镜像所花费的时间。
Running	Pod 已经绑定到一个节点，并且所有的容器都已经创建。至少有一个容器仍在运行，或者正在启动或重新启动。
Succeeded	Pod 中的所有容器都已成功终止，并且不会重新启动。
Failed	Pod 中的所有容器都已终止，并且至少有一个容器因故障而终止。也就是说，容器要么以非零状态退出，要么被系统终止。
Unknown	由于某种原因，无法获取 Pod 的状态。此阶段通常是由于与 Pod 应运行的节点通信时出现错误而发生的。

#查看状态说明

```
[root@master1 ~]#kubectl explain pods.status.phase
```

```
KIND:      Pod
```

```
VERSION:   v1
```

```
FIELD:     phase <string>
```

DESCRIPTION:

The phase of a Pod is a simple, high-level summary of where the Pod is in its lifecycle. The conditions array, the reason and message fields, and the individual container status arrays contain more detail about the pod's status. There are five possible phase values:

Pending: The pod has been accepted by the Kubernetes system, but one or more of the container images has not been created. This includes time before being scheduled as well as time spent downloading images over the network, which could take a while. Running: The pod has been bound to a node, and all of the containers have been created. At least one container is still running, or is in the process of starting or restarting.

Succeeded: All containers in the pod have terminated in success, and will not be restarted. Failed: All containers in the pod have terminated, and at least one container has terminated in failure. The container either exited with non-zero status or was terminated by the system. Unknown: For some reason the state of the pod could not be obtained, typically due to an error in communicating with the host of the pod.

More info:

<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle#pod-phase>

#查看

```
[root@master1 ~]#kubectl get pod pod-test -o yaml | grep -A 2 phase
```

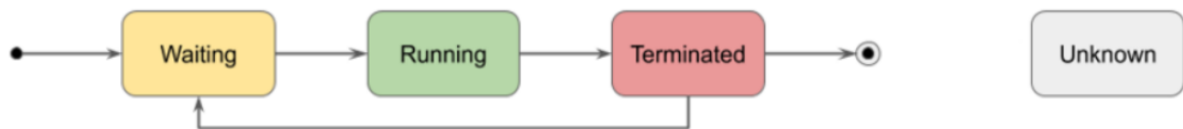
```
phase: Running
```

```
podIP: 172.16.4.38
```

```
podIPs:
```

3.2.2.3.3 Pod 内容器状态

容器的主要状态如下:



容器状态	描述
Waiting	容器处于 Running 和Terminated 状态之前的状态
Running	容器能够正常的运行状态
Terminated	容器已经被成功的关闭了

```
[root@master1 ~]#kubectl explain pods.status.containerStatuses.state
```

```
KIND:      Pod
```

```
VERSION:   v1
```

```
RESOURCE:  state <Object>
```

```
DESCRIPTION:
```

```
Details about the container's current condition.
```

```
ContainerState holds a possible state of container. Only one of its members may be specified. If none of them is specified, the default one is ContainerStateWaiting.
```

```
FIELDS:
```

```
running <Object>
```

```
Details about a running container
```

```
terminated <Object>
```

```
Details about a terminated container
```

```
waiting <Object>
```

```
Details about a waiting container
```

```
#查看
```

```
[root@master1 ~]#kubectl get pod pod-test -o yaml |grep -A 2 state
```

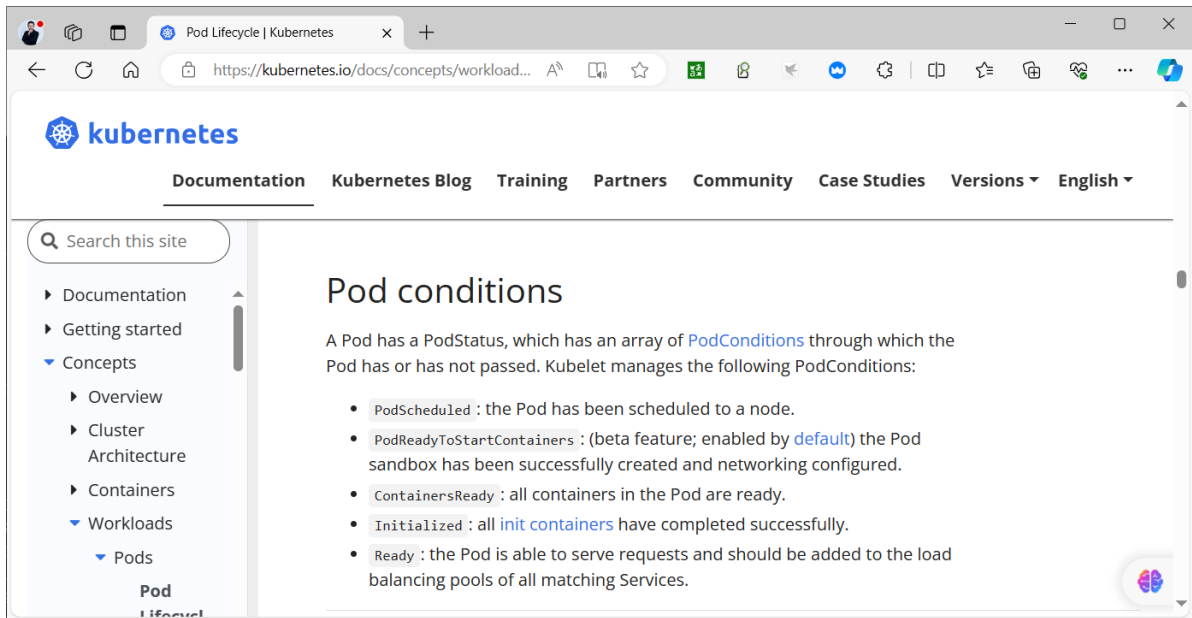
```
state:
```

```
running:
```

```
startedAt: "2020-03-17T06:32:12Z"
```

3.2.2.3.4 Pod 的启动流程状态

<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-conditions>



流程状态	描述
PodScheduled	Pod被调度到某一个节点
Ready	准备就绪，Pod可以处理请求
Initialized	Pod中所有初始init容器启动完毕
Unschedulable	由于资源等限制，导致pod无法被调度
ContainersReady	Pod中所有的容器都启动完毕了

```
[root@master1 ~]#kubectl explain pods.status.conditions.type
KIND:      Pod
VERSION:   v1

FIELD:     type <string>

DESCRIPTION:
    Type is the type of the condition. More info:
    https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle#pod-conditions

[root@master1 ~]#kubectl get pod pod-test -o yaml |grep -A 15 conditions:
- lastProbeTime: null
  lastTransitionTime: "2022-03-17T06:32:12Z"
  status: "True"
  type: Initialized
- lastProbeTime: null
  lastTransitionTime: "2022-03-17T06:32:13Z"
  status: "True"
  type: Ready
- lastProbeTime: null
  lastTransitionTime: "2022-03-17T06:32:13Z"
  status: "True"
  type: ContainersReady
- lastProbeTime: null
```

```
lastTransitionTime: "2022-03-17T06:32:12Z"
status: "True"
```

3.2.2.3.5 Pod 重启策略

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#restart-policy>

注意：同一个 Pod 内所有容器只能使用统一的重启策略

说明

重启策略	描述
Always	无论退出码exit code是否为0，都要重启，即只要退出就重启，并且重启次数并没有限制，此为默认值
OnFailure	容器终止运行退出码exit code不为0时才重启,重启次数并没有限制
Never	无论何种退出码exit code,Pod都不重启。主要针对Job和CronJob

```
[root@master1 ~]#kubectl explain pods.spec.restartPolicy
KIND:      Pod
VERSION:   v1

FIELD:     restartPolicy <string>

DESCRIPTION:
    Restart policy for all containers within the pod. One of Always, OnFailure,
    Never. Default to Always. More info:
    https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#restart-
    policy

#查看
[root@master1 ~]#kubectl get pod pod-test -o yaml |grep restartPolicy
restartPolicy: Always
```

范例：

```
[root@master1 ~]#cat pod-restart-policy.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: myapp
  name: myapp
  namespace: default
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    name: myapp
    restartPolicy: OnFailure #默认Always策略，会不断重启

#默认Always策略会不断重启，状态 CrashLoopBackOff
[root@master1 ~]#kubectl get pod
NAME      READY   STATUS    RESTARTS   AGE
```

```
myapp    0/1    CrashLoopBackOff    6 (62s ago)    6m46s
```

#指定OnFailure策略，退出码为0，不会重启

```
[root@master1 ~]#kubectl apply -f pod-test.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
myapp	0/1	Completed	0	85s

3.2.2.3.6 Pod 镜像拉取状态

拉取策略	描述
Always	总是拉取新镜像，注意：如果镜像的Tag为latest，拉取策略为always或ifNotPresent 都会重新拉取镜像
IfNotPresent	此为默认值，如果本地不存在的话，再拉取新镜像，例外情况:如果镜像的Tag为latest，仍会重新拉取镜像
Never	只使用本地的镜像，从不获取新镜像

```
[root@master1 ~]#kubectl explain pod.spec.containers.imagePullPolicy
```

KIND: Pod

VERSION: v1

FIELD: imagePullPolicy <string>

DESCRIPTION:

Image pull policy. One of Always, Never, IfNotPresent. Defaults to Always if :latest tag is specified, or IfNotPresent otherwise. Cannot be updated.
More info:
<https://kubernetes.io/docs/concepts/containers/images#updating-images>

#查看发现IfNotPresent是默认值

```
[root@master1 ~]#kubectl get pod pod-test -o yaml |grep imagePullPolicy
imagePullPolicy: IfNotPresent
```

范例:

```
[root@master1 pod]#cat pod-imagePullPolicy.yaml
```

apiVersion: v1

kind: Pod

metadata:

labels:

run: myapp

name: myapp

namespace: default

spec:

containers:

- image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0

imagePullPolicy: Never

name: myapp

restartPolicy: OnFailure

```
[root@master1 pod]#kubectl apply -f pod-imagePullPolicy.yaml
```

#如果没有镜像，会出现下面错误

```
[root@master1 pod]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
myapp	0/1	ErrImageNeverPull	0	3s

3.2.2.3.7 Pod 状态汇总

状态	描述
Pending	APIserver已经创建该pod,但是Pod因为某些原因无法调度到合适的节点运行
Running	Pod内所有的容器已创建,且至少有一个容器处于运行状态,正在启动或重启状态
Waiting	Pod 等待启动中
Terminating	Pod 正在删除,若超过终止宽限期仍无法删除,可以强制删除 kubectl delete pod <pod_name> -n --grace-period=0 --force
Succeeded	所有容器均成功执行退出,且不会再重启
Ready	Pod 已经准备好,可以提供服务
Failed	Pod内所有容器都已退出,其中至少有一个容器退出失败
Unknown	由于某种原因kubelet无法获取Pod的状态,比如网络不通
CrashLookBackOff	曾经启动Pod成功,但是后来异常情况下,重启次数过多导致异常终止 Pod退避算法:第1次0秒立刻重启,第二次10秒后重启,第三次20秒后重启,...第6次160秒后重启,第7次300秒后重启,如仍然重启失败,则为 CrashLookBackOff状态
Error	因为集群配置、安全限制、资源等原因导致Pod 启动过程中发生了错误
Evicted	集群节点系统内存或硬盘资源不足导致Pod出现异常
Completed	表示Pod已经执行完成,比如:一次性的Job或周期性的CronJob中的Pod执行完成后,会显示此状态
Unschedulable	Pod 不能调度到节点,一般可能是因为没有合适的节点主机
PodScheduled	Pod 正在被调度过程,但此状态的时间很短
Initialized	Pod中所有初始init容器启动完毕
ImagePullBackOff	Pod对应的镜像拉取失败
InvalidImageName	镜像名称无效导致镜像无法下载
ImageInspectError	镜像检查错误,通常因为镜像不完整
ErrImageNeverPull	拉取镜像因策略禁止错误,镜像仓库权限拒绝或私有导致
RegistryUnavailable	镜像仓库服务不可用,比如:网络原因或仓库服务器宕机
ErrImagePull	镜像拉取错误,可能是镜像地址错误,因为超时或拉取被强行终止
NetworkPluginNotReady	网络插件异常,会导致新建容器出错,但旧的容器不受影响
NodeLost	Pod所在节点无法联系
CreateContainerConfigError	创建容器配置错误

状态	描述
CreateContainerError	创建容器错误
RunContainerError	运行容器错误，比如:容器中没有PID为1的前台进程等原因
ContainersNotInitialized	容器没有初始化完成
ContainersNotReady	容器没有准备好
ContainerCreating	容器正在创建过程中
PodInitializing	容器正在初始化中
DockerDaemonNotReady	节点的Docker服务异常

范例：Pending状态

```
#如果所有worker无法工作，会导致pod无法调度，出现pending
[root@master1 pod]#kubectl describe pod myapp
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Warning         FailedScheduling    10s   default-scheduler    0/4 nodes are available: 1
node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }, 3
node(s) had untolerated taint {node.kubernetes.io/unreachable: }. preemption:
0/4 nodes are available: 4 Preemption is not helpful for scheduling.
```

3.3 Pod 的健康状态监测

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

3.3.1 健康状态检测三种探针

3.3.1.1 Pod 的状态监控

在以镜像打包技术为基础的环境中，容器运行起来的效果就类似于一个"黑盒"

默认情况下我们不知道里面是一种什么样的运行环境，为了实时的监控容器应用环境的运行状态，docker 容器提供了inspect的方法，让我们主动来采集相关的状态数据，但是这种方式太繁琐了。

实际上，我们需要一种可以及时的获取容器的各种运行状态数据，所以对于容器任务编排的环境下，他们都应该考虑到一种场景：主动的将容器运行的相关数据暴露出来 -- 数据暴露接口。比如：包含大量metric指标数据的API接口。

对于kubernetes内部的pod环境来说，常见的这些API接口有：



process health	#状态健康检测接口
readiness	#容器可读状态的接口
liveness	#容器存活状态的接口
metrics	#监控指标接口
tracing	#全链路监控的埋点(探针)接口
logs	#容器日志接口

3.3.1.2 Pod 的健康性监控

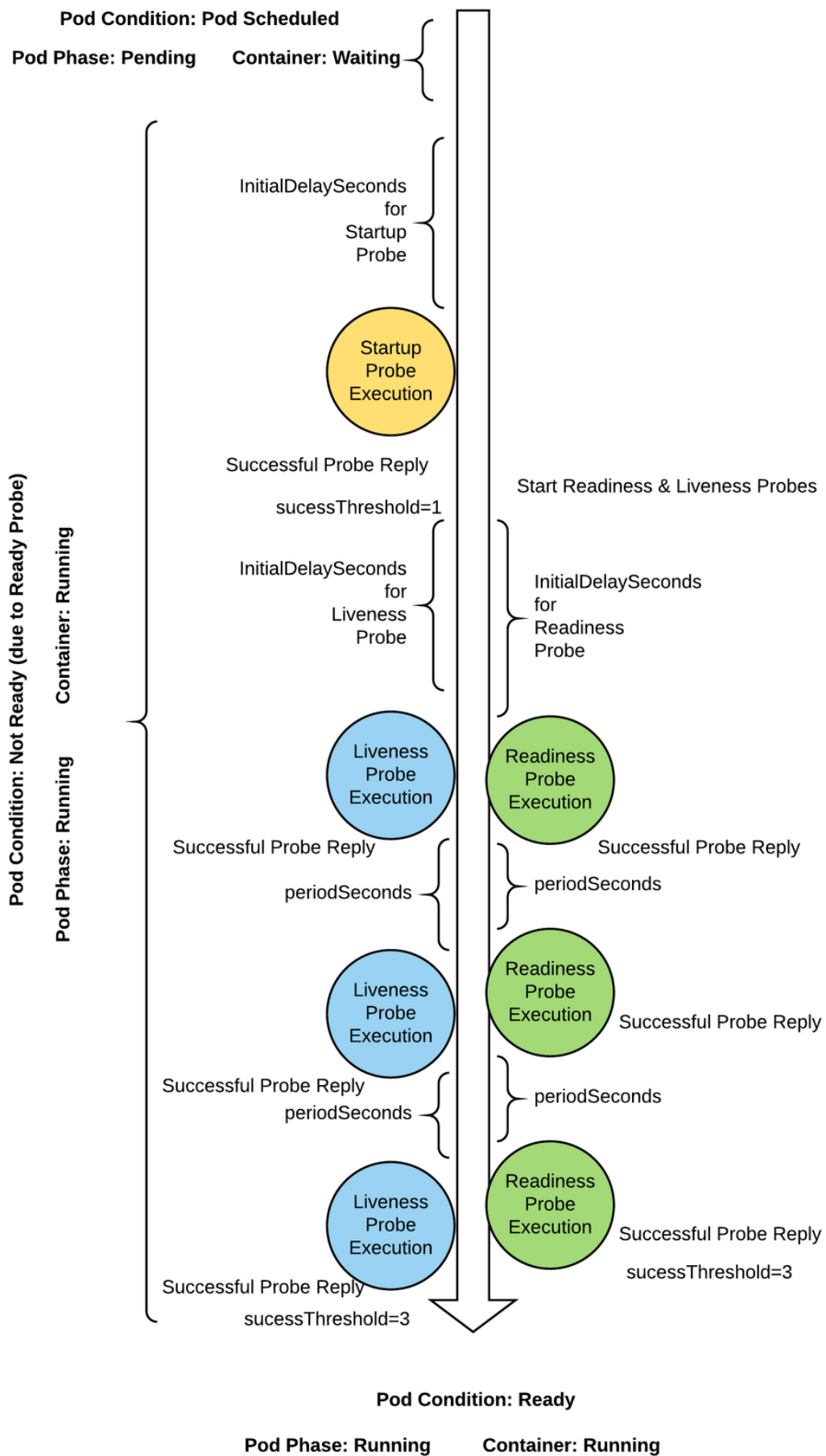
Pod 通过探针要制实现Pod 的健康性监控,当一旦检测出Pod故障时, 会重置Pod或将Pod从service后端 endpoint删除, 从而实现服务的高可用

3.3.1.2.1 探针类型

针对运行中的容器, `kubelet` 可以选择是否执行以下三种探针, 以及如何针对探测结果作出反应

```
https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#types-of-probe
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
```

Pod 支持的监测探针类型如下



- StartupProbe:

kuberenetes1.16引用, 启动探针, Pod 在启动时只进行一次, 主要用于一些启动较慢的应用

指示容器中的应用是否已经启动。如果提供了启动探针，则所有其他探针都会被禁用，直到此探针成功为止。

如果启动探测失败，kubelet 将杀死容器，而容器依其重启策略进行重启。

检测失败后，默认先删除Pod中容器，并且重新创建容器，但Pod的ID名称不变，如果继续失败，重复此过程

如果容器没有提供启动探测，则默认状态为 Success。

只有此探针成功,才能启动容器后续的LivenessProbe和ReadinessProbe探针,否则kubelet会关闭容器,并按restartPolicy处理

如果没有定义此探针,默认成功

实现用户自定义的一些启动探测机制，效果等同于livenessProbe，区别在于应用不同的参数或阈值

参考资料：kubectl explain pod.spec.containers.startupProbe

- LivenessProbe:

指示容器是否正在运行。如果存活态探测失败，则 kubelet 会杀死容器，并且容器将根据其[重启策略](#)决定未来。如果容器不提供存活探针，则默认状态为 `Success`

存活探针,周期性检测,用于持续跟踪探测容器是否运行,即只探测容器是否活着,但活着不一定能正常工作

检测未通过时，kubelet会关闭容器,并根据restartPolicy的定义来决定是否会重启该容器，默认会重置Pod（重新还原镜像）并重启，但Pod名称，UID，IP及内核配置保持不变

检测失败后,会在kubectl get pod 显示的STATUS状态为CrashLoopBackOff,

检测失败后，默认先删除Pod中容器，并且重新创建容器，但Pod的ID名称不变，如果继续失败，重复此过程

检测失败后，重启此Pod时，如果配置对应的Service，其后端endpoint也会删除此Pod的地址，但Pod重启后会再添加进endpoint

未定义时，Kubelet认为只要容器未终止，即为健康

参考资料：kubectl explain pod.spec.containers.livenessProbe

- ReadinessProbe:

就绪探针,周期性检测,一般用于持续跟踪探测容器内应用是否健康,即探测容器不仅活着,而且可以正常工作

指示容器是否准备好为请求提供服务。如果就绪态探测失败，端点控制器将从与 Pod 匹配的所有服务的端点列表中删除该 Pod 的 IP 地址。初始延迟之前的就绪态的状态值默认为 `Failure`。如果容器不提供就绪态探针，则默认状态为 `Success`。

检测检测失败，与该Pod关联的Service，会将该Pod从Service的后端可用端点列表中删除；直至再次就绪，重新添加回来。

如果检测失败，kubectl get pod 显示STATUS状态仍为Running，但是READY状态为0/1，此外重启Pod与否需要看情况

初始延迟前的就绪状态默认为Failure

未定义时，只要容器未终止，即为就绪

参考资料：kubectl explain pod.spec.containers.ReadinessProbe

范例：查看kube-apiserver 的Pod内置的探针

```
[root@master1 ~]#kubectl get pod -n kube-system kube-apiserver-master1.wang.org -o yaml |grep -A10 Probe
```

```

livenessProbe:
  failureThreshold: 8
  httpGet:
    host: 10.0.0.100
    path: /livez
    port: 6443
    scheme: HTTPS
  initialDelaySeconds: 10
  periodSeconds: 10
  successThreshold: 1
  timeoutSeconds: 15
--
readinessProbe:
  failureThreshold: 3
  httpGet:
    host: 10.0.0.100
    path: /readyz
    port: 6443
    scheme: HTTPS
  periodSeconds: 1
  successThreshold: 1
  timeoutSeconds: 15
resources:
--
startupProbe:
  failureThreshold: 24
  httpGet:
    host: 10.0.0.100
    path: /livez
    port: 6443
    scheme: HTTPS
  initialDelaySeconds: 10
  periodSeconds: 10
  successThreshold: 1
  timeoutSeconds: 15
...

```

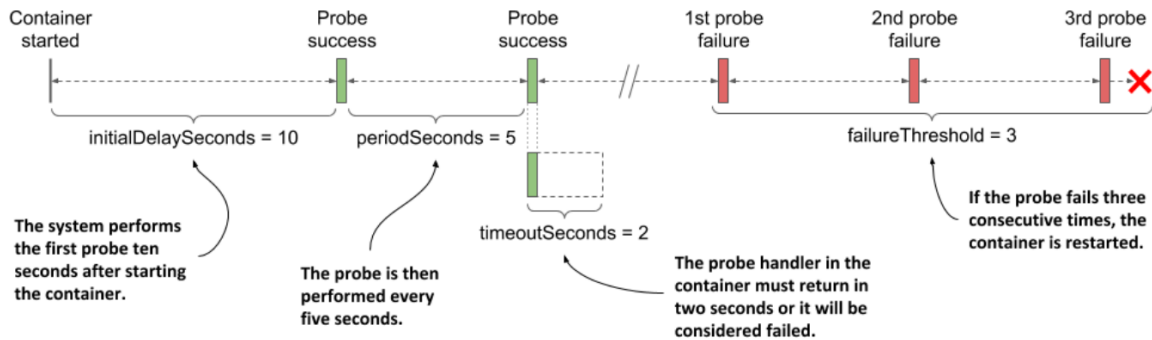
3.3.1.2.2 配置探针

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#configure-probes>

配置探针

Probe 有很多配置字段，可以使用这些字段精确地控制启动、存活和就绪检测的行为：

- **initialDelaySeconds**：容器启动后要等待多少秒后才启动启动、存活和就绪探针，默认是 0 秒，最小值是 0。
- **periodSeconds**：执行探测的时间间隔（单位是秒）。默认是 10 秒。最小值是 1。
- **timeoutSeconds**：探测的超时后等待多少秒。默认值是 1 秒。最小值是 1。
- **successThreshold**：探针在失败后，被视为成功的最小连续成功数。默认值是 1。存活和启动探测的这个值必须是 1。最小值是 1。
- **failureThreshold**：当探测失败时，Kubernetes 的重试次数。对存活探测而言，放弃就意味着重新启动容器。对就绪探测而言，放弃意味着 Pod 会被打上未就绪的标签。默认值是 3。最小值是 1。



```
spec:
  containers:
  - name: string
    image: string
    livenessProbe:
      exec <Object>                                # 命令式探针
      httpGet <Object>                              # http GET类型的探针
      tcpSocket <Object>                            # tcp Socket类型的探针
      initialDelaySeconds <integer>                 # 发起初次探测请求前的延迟时长，默认为0，生产根
                                                    据服务启动时长来设置，防止反复重启
      periodSeconds <integer>                       # 每次探测请求间隔，即探测的周期，默认10s，如果
Pod众多，可适当设长，比如60s
      timeoutSeconds <integer>                     # 探测的超时时长，默认是1s
      successThreshold <integer>                   # 连续成功几次才表示状态正常，默认值是1次，注
意：liveness和startup只能是1
      failureThreshold <integer>                   # 连续失败几次才表示状态异常，默认值是3次，即从
成功变为失败的检查次数

#注意：上面以livenessProbe为例，startupProbe和readinessProbe 的属性与livenessProbe是相
同的
```

3.3.2 实现探针的三种方式

监测的实现方式

对于Pod中多容器的场景，只有所有容器就绪，才认为Pod就绪

<https://kubernetes.io/zh-cn/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>

kubelet 定期执行LivenessProbe和ReadinessProbe探针来诊断Pod的健康状况，Pod探针的实现方式有很多，常见的有如下三种：

监测的实现方式	解析
Exec	直接执行指定的命令，根据命令结果的状态码\$?判断是否成功，成功则返回表示探测成功
TCPSocket	根据相应TCP套接字连接建立状态判断,如果端口能正常打开，即成功
HTTPGet	根据指定Http/Https服务URL的响应码结果判断，当2xx, 3xx的响应码表示成功
gRPC	使用 gRPC 执行一个远程过程调用。目标应该实现 gRPC健康检查。如果响应的状态是 "SERVING", 则认为诊断成功。gRPC 探针是一个 Alpha 特性，只有在你启用了 "GRPCContainerProbe" 特性门控时才能使用。

readinessProbe: exec: command: - /bin/health-check - --quiet initialDelaySeconds: 5 periodSeconds: 15	readinessProbe: tcpSocket: port: 80 initialDelaySeconds: 5 periodSeconds: 15	readinessProbe: httpGet: path: /_status/healthz port: 80 initialDelaySeconds: 5 periodSeconds: 15
livenessProbe: exec: command: - /bin/health-check - --quiet initialDelaySeconds: 15 periodSeconds: 15	livenessProbe: tcpSocket: port: 80 initialDelaySeconds: 15 periodSeconds: 15	livenessProbe: httpGet: path: /_status/healthz port: 80 initialDelaySeconds: 15 periodSeconds: 15

注意：每种探针都支持这三种探针实现方式

范例：

```
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-dns -o yaml
|grep -m2 -EA9 'livenessProbe|readinessProbe'
  livenessProbe:
    failureThreshold: 5
    httpGet:
      path: /health
      port: 8080
      scheme: HTTP
    initialDelaySeconds: 60
    periodSeconds: 10
    successThreshold: 1
    timeoutSeconds: 5
  --
  readinessProbe:
    failureThreshold: 3
    httpGet:
      path: /ready
      port: 8181
      scheme: HTTP
```

```
periodSeconds: 10
successThreshold: 1
timeoutSeconds: 1
resources:
```

3.3.3 探针案例

3.3.3.1 Exec 方式案例

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#define-a-liveness-command>

exec 其实就是尝试通过在容器内部来执行一个命令，看看能不能执行成功，如果成功，那么说明该对象是正常的，否则就是失败的。

exec的执行动作与我们之前用到的command作用一样。

范例：startupprobe的exec探针

```
#准备yaml文件
[root@master1 ~]#cat pod-startup-exec.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-startup-exec
  namespace: default
  labels:
    app: pod-startup-exec
spec:
  containers:
  - name: pod-startup-exec-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    startupProbe:
      exec:
        command: [ '/bin/sh', '-c', '[ "$(curl -s 127.0.0.1/livez)" == "OK" ]' ]
      initialDelaySeconds: 60
      timeoutSeconds: 1 #此容器返回结果需要花几秒时间，所以此处1s会导致检查失败，可以设为5s以上可以成功
      periodSeconds: 5
      successThreshold: 1
      failureThreshold: 1

#创建Pod
[root@master1 ~]#kubectl apply -f pod-startup-exec.yaml

#查看Pod，发现READY为0/1表示没有
[root@master1 ~]#kubectl get pod pod-startup-exec -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
pod-startup-exec	0/1	Running			
12s	10.244.2.116	node2.wang.org	<none>	<none>	0

```
#此应用刚启动首次访问默认需要等5s后才能访问
[root@master1 ~]#curl 10.244.2.116/livez
^C
```

#模拟故障

```
[root@master1 ~]#curl -XPOST -d 'livez=FAIL' 10.244.2.116/livez
```

#访问故障

```
[root@master1 ~]#curl 10.244.2.116/livez
```

FAIL

#查看状态

```
[root@master1 ~]#kubectl describe pod pod-startup-exec
```

.....

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	49s	default-scheduler	Successfully assigned default/pod-startup-exec to node2.wang.org
Normal	Pulled	49s	kubelet	Container image "wangxiaochun/pod-test:v0.1" already present on machine
Normal	Created	49s	kubelet	Created container pod-startup-exec-container
Normal	Started	49s	kubelet	Started container pod-startup-exec-container
warning	Unhealthy	16s	kubelet	Startup probe errored: rpc error: code = Unknown desc = deadline exceeded ("DeadlineExceeded"): context deadline exceeded
warning	Unhealthy	14s	kubelet	Startup probe failed:
Normal	Killing	14s	kubelet	Container pod-startup-exec-container failed startup probe, will be restarted

#跟踪状态，发现重启，并修复故障

```
[root@master1 ~]#kubectl get pod pod-startup-exec -w
```

NAME	READY	STATUS	RESTARTS	AGE
pod-startup-exec	0/1	Running	0	5s
pod-startup-exec	0/1	Running	1 (0s ago)	91s
pod-startup-exec	0/1	Running	1 (65s ago)	2m36s
pod-startup-exec	1/1	Running	1 (65s ago)	2m36s

#清理环境

```
[root@master1 ~]#kubectl delete -f pod-startup-exec.yaml
```

范例:

```
[root@master1 ~]#cat pod-liveness-exec-cmd.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-exec-cmd
  namespace: default
spec:
  containers:
  - name: pod-liveness-exec-cmd-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    imagePullPolicy: IfNotPresent
    command: ["/bin/sh", "-c", "touch /tmp/healthy; sleep 3; rm -f /tmp/healthy;sleep 3600"]
    livenessProbe:
      exec:
        command: ["test", "-e", "/tmp/healthy"]
```



```
initialDelaySeconds: 1
periodSeconds: 3
```

#创建容器

```
[root@master1 ~]#kubectl apply -f pod-liveness-exec-cmd.yaml
```

#实时监控,发现不断重启

```
[root@master1 ~]#kubectl get pod pod-liveness-exec-cmd -w
```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-exec-cmd	1/1	Running	0	19s
pod-liveness-exec-cmd	1/1	Running	1 (1s ago)	44s
pod-liveness-exec-cmd	1/1	Running	2 (0s ago)	85s
pod-liveness-exec-cmd	1/1	Running	3 (0s ago)	2m7s
pod-liveness-exec-cmd	1/1	Running	4 (1s ago)	2m50s
pod-liveness-exec-cmd	1/1	Running	5 (0s ago)	3m31s
pod-liveness-exec-cmd	0/1	CrashLoopBackOff	5 (0s ago)	4m13s
pod-liveness-exec-cmd	1/1	Running	6 (89s ago)	5m42s
pod-liveness-exec-cmd	0/1	CrashLoopBackOff	6 (0s ago)	6m22s
pod-liveness-exec-cmd	1/1	Running	7 (2m47s ago)	9m9s
pod-liveness-exec-cmd	0/1	CrashLoopBackOff	7 (0s ago)	9m49s
...				

```
[root@master1 ~]#kubectl get pod pod-liveness-exec-cmd -o yaml
```

```
.....
```

spec:

containers:

- command:

```
- /bin/sh
```

```
- -C
```

```
- touch /tmp/healthy; sleep 3; rm -f /tmp/healthy;sleep 3600
```

```
image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
```

```
imagePullPolicy: IfNotPresent
```

livenessProbe:

exec:

command:

```
- test
```

```
- -e
```

```
- /tmp/healthy
```

```
failureThreshold: 3
```

```
initialDelaySeconds: 1
```

```
periodSeconds: 3
```

```
successThreshold: 1
```

```
timeoutSeconds: 1
```

```
name: pod-liveness-exec-cmd-container
```

```
.....
```

#状态描述

```
[root@master1 ~]#kubectl describe pod pod-liveness-exec-cmd
```

```
.....
```

Containers:

pod-liveness-exec-cmd-container:

Container ID:

```
docker://27587b1ea13d19fe768b2c2d95703971239e065069c8a519ad877acea486b2e7
```

```
Image: busybox
```

```
Image ID: docker-
```

```
pullable://busybox@sha256:5acba83a746c7608ed544dc1533b87c737a0b0fb730301639a0179f9344b1678
```

```

Port:          <none>
Host Port:     <none>
Command:
  /bin/sh
  -C
  touch /tmp/healthy; sleep 3; rm -f /tmp/healthy;sleep 3600
State:         CrashLoopBackOff
  Started:      Thu, 17 Mar 2020 21:51:35 +0800
Ready:         True
Restart Count: 0
Liveness:      exec [test -e /tmp/healthy] delay=1s timeout=1s period=3s
#success=1 #failure=3
.....
Events:
  Type            Reason            Age             From              Message
  ----            -
Normal          Scheduled         10m             default-scheduler Successfully
assigned default/pod-liveness-exec-cmd to node2.wang.org
Normal          Killing           8m47s (x3 over 10m) kubelet            Container pod-
liveness-exec-cmd-container failed liveness probe, will be restarted
Normal          Pulled            8m17s (x4 over 10m) kubelet            Container image
"busybox" already present on machine
Normal          Created           8m17s (x4 over 10m) kubelet            Created container
pod-liveness-exec-cmd-container
Normal          Started           8m17s (x4 over 10m) kubelet            Started container
pod-liveness-exec-cmd-container
Warning         Unhealthy         8m11s (x10 over 10m) kubelet            Liveness probe
failed:
Warning         BackOff           10s (x26 over 6m11s) kubelet            Back-off
restarting failed container

```

范例:

```

[root@master1 ~]#cat pod-liveness-exec-http.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-exec-http
  namespace: default
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    livenessProbe:
      exec:
        command: ['/bin/sh', '-c', '[ "$(curl -s 127.0.0.1/livez)" == "OK" ]']
        initialDelaySeconds: 5
        timeoutSeconds: 1
        periodSeconds: 5

```

#注意:此处的/livez 接口是此容器自定义的接口地址, 并非所有容器都支持的

```

[root@master1 ~]#kubectl apply -f pod-liveness-exec-http.yaml

[root@master1 ~]#kubectl get pod pod-liveness-exec -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE
pod-liveness-exec                   READY   Running   0           1m    10.1.1.10       node1

```

```
pod-liveness-exec-http 1/1 Running 0 14m 172.16.3.11
node1.wang.org <none> <none>
```

```
[root@master1 ~]#curl -I 172.16.3.11/livez
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 2
Server: werkzeug/1.0.0 Python/3.8.2
```

#修改容器状态,当前的/livez支持post方法,用于更改接口的状态,从而来模拟容器的检测效果

```
[root@master1 ~]#curl -XPOST -d 'livez=FAIL' 172.16.3.11/livez
```

#访问容器访问失败

```
[root@master1 ~]#curl -I 172.16.3.11/livez
FAIL
[root@master1 ~]#curl -I 172.16.3.11/livez
HTTP/1.0 506 VARIANT ALSO NEGOTIATES
Content-Type: text/html; charset=utf-8
Content-Length: 4
Server: werkzeug/1.0.0 Python/3.8.2
Date: Thu, 17 Mar 2020 14:06:49 GMT
```

```
[root@master1 ~]#kubectl describe pod pod-liveness-exec-http
```

```
....
      State:          Running
        Started:      Thu, 17 Mar 2022 22:07:31 +0800
      Last State:      Terminated
        Reason:        Error
        Exit Code:     137
        Started:      Thu, 17 Mar 2020 21:50:32 +0800
        Finished:     Thu, 17 Mar 2020 22:07:31 +0800
      Ready:           True
      Restart Count:   1
.....
      Warning Unhealthy 14s (x8 over 23m) kubelet Liveness probe
failed:
      Normal Killing 14s (x2 over 6m44s) kubelet Container pod-test
failed liveness probe, will be restarted

Events:
  Type     Reason      Age           From          Message
  ----     -
  Normal   Scheduled   5m54s         default-scheduler Successfully
assigned default/pod-liveness-exec-http to node2.wang.org
  Warning Unhealthy   54s (x3 over 64s) kubelet Liveness probe
failed:
  Normal   Killing     54s           kubelet Container pod-test
failed liveness probe, will be restarted
  Normal   Pulled      24s (x2 over 5m54s) kubelet Container image
"wangxiaochun/pod-test:v0.1" already present on machine
  Normal   Created     24s (x2 over 5m54s) kubelet Created container
pod-test
  Normal   Started     24s (x2 over 5m53s) kubelet Started container
pod-test
  Warning Unhealthy   16s (x2 over 5m41s) kubelet Liveness probe
errored: rpc error: code = Unknown desc = deadline exceeded
("DeadlineExceeded"): context deadline exceeded
```

#结果显示：探针探测失败

.....

#过一会儿发现Pod重启，自动修复故障

```
[root@master1 pod]#kubectl get pod pod-liveness-exec-http
```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-exec-http	1/1	Running	1 (14s ago)	8m54s

#再次访问正常

```
[root@master1 ~]#curl -I 172.16.3.11/livez
```

HTTP/1.0 200 OK

Content-Type: text/html; charset=utf-8

Content-Length: 2

Server: Werkzeug/1.0.0 Python/3.8.2

#如果执行了两次POST方法，会显示重启了两次

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-exec	1/1	Running	2 (117s ago)	26m

范例：监测 Redis 服务

```
[root@master1 ~]#cat pod-liveness-exec-redis.yaml
```

apiVersion: apps/v1

kind: Deployment

metadata:

name: redis-deployment

spec:

replicas: 1

selector:

matchLabels:

app: redis-6379

template:

metadata:

labels:

app: redis-6379

spec:

containers:

- name: redis

image: registry.cn-beijing.aliyuncs.com/wangxiaochun/redis:6.2.5

ports:

- containerPort: 6379

livenessProbe:

#readinessProbe:

exec:

command: ['/bin/sh', '-c', 'redis-cli info']

initialDelaySeconds: 10

periodSeconds: 3

timeoutSeconds: 3

successThreshold: 1

failureThreshold: 3

apiVersion: v1

kind: Service

metadata:

name: redis-svc

```
spec:
  ports:
  - name: redis
    port: 6379
    targetPort: 6379
    protocol: TCP
  selector:
    app: redis-6379
```

#执行

```
[root@master1 ~]#kubectl apply -f pod-liveness-exec-redis.yaml
```

#查看状态

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS	GATES	
redis-deployment-566d77d68d-gtw6m	1/1	Running	0	5s	
10.244.3.89	node3.wang.org	<none>	<none>		

#查看svc和endpoint

```
[root@master1 ~]#kubectl get svc,ep redis-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/redis-svc	ClusterIP	10.102.54.249	<none>	6379/TCP	12s

NAME	ENDPOINTS	AGE
endpoints/redis-svc	10.244.3.89:6379	12s

#直接访问Pod

```
[root@master1 ~]#kubectl exec -it redis-deployment-566d77d68d-gtw6m -- redis-cli
info server
```

Server

```
redis_version:5.0.0
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:9a5fa86bdce33ad2
redis_mode:standalone
os:Linux 5.15.0-56-generic x86_64
arch_bits:64
multiplexing_api:epoll
atomicvar_api:atomic-builtin
gcc_version:6.3.0
process_id:1
run_id:cb598ce89bfb91116eac06c10a357fcd06f0e586
tcp_port:6379
uptime_in_seconds:180
uptime_in_days:0
hz:10
configured_hz:10
lru_clock:9526851
executable:/data/redis-server
config_file:
```

#测试通过svc来访问Pod

```
[root@master1 ~]#echo -e "info server\nquit\nexit" | nc 10.102.54.249 6379
$461
```

Server

```
redis_version:5.0.0
redis_git_sha1:00000000
```

```
redis_git_dirty:0
redis_build_id:9a5fa86bdce33ad2
redis_mode:standalone
os:Linux 5.15.0-56-generic x86_64
arch_bits:64
multiplexing_api:epoll
atomicvar_api:atomic-builtin
gcc_version:6.3.0
process_id:1
run_id:cb598ce89bfb91116eac06c10a357fcd06f0e586
tcp_port:6379
uptime_in_seconds:311
uptime_in_days:0
hz:10
configured_hz:10
lru_clock:9526982
executable:/data/redis-server
config_file:
```

+OK

#模拟故障

```
[root@master1 ~]#echo "SHUTDOWN" | nc 10.102.54.249 6379
```

#马上查看，可以看到Pod重启，并且将pod从endpoint删除

```
[root@master1 ~]#kubectl get pod,svc,ep -o wide
```

NAME	READY	STATUS	RESTARTS
AGE IP NODE	NOMINATED	NODE READINESS GATES	
pod/redis-deployment-566d77d68d-gtw6m	0/1	CrashLoopBackOff	1 (11s ago)
9m6s 10.244.3.89 node3.wang.org	<none>	<none>	

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
SELECTOR					
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	29d
<none>					
service/redis-svc	ClusterIP	10.102.54.249	<none>	6379/TCP	9m6s
app=redis-6379					

NAME	ENDPOINTS	AGE
endpoints/kubernetes	10.0.0.200:6443	29d
endpoints/redis-svc		9m6s

#过一会儿再查看，恢复正常

```
[root@master1 ~]#kubectl get pod,svc,ep -o wide
```

NAME	READY	STATUS	RESTARTS	AGE
IP NODE	NOMINATED NODE	READINESS GATES		
pod/redis-deployment-566d77d68d-gtw6m	1/1	Running	2 (2m28s ago)	11m
10.244.3.89 node3.wang.org	<none>	<none>		

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
SELECTOR					
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	29d
<none>					
service/redis-svc	ClusterIP	10.102.54.249	<none>	6379/TCP	11m
app=redis-6379					

NAME	ENDPOINTS	AGE
endpoints/kubernetes	10.0.0.200:6443	29d

```
endpoints/redis-svc      10.244.3.89:6379      11m
```

#删除

```
[root@master1 ~]#kubectl delete -f pod-liveness-exec-redis.yaml
```

#修改配置

```
[root@master1 ~]#vim pod-liveness-exec-redis.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: redis-deployment
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: redis-6379
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: redis-6379
```

```
    spec:
```

```
      containers:
```

```
      - name: redis
```

```
        image: redis:5.0.0
```

```
        ports:
```

```
        - containerPort: 6379
```

```
      #livenessProbe:
```

```
      readinessProbe:      #只修改此行
```

```
        exec:
```

```
          command: ['/bin/sh', '-c', 'redis-cli info']
```

```
.....
```

#应用

```
[root@master1 ~]#kubectl apply -f pod-liveness-exec-redis.yaml
```

#查看Pod

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
redis-deployment-865fd789d5-pc26n	1/1	Running	0	7s	
10.244.3.102	node3.wang.org	<none>	<none>		

#关闭Redis服务

```
[root@master1 ~]# echo -e "SHUTDOWN" | nc 10.244.3.102 6379
```

#查看重启,为什么?

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
redis-deployment-865fd789d5-pc26n	1/1	Running	1 (9s ago)	105s

#查看endpoint下线

```
[root@master1 ~]#kubectl get svc,ep redis-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/redis-svc	ClusterIP	10.109.113.230	<none>	6379/TCP	45s

NAME	ENDPOINTS	AGE
endpoints/redis-svc		45s

#稍过一会儿endpoint上线

```
[root@master1 ~]#kubectl get svc,ep redis-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/redis-svc	ClusterIP	10.109.113.230	<none>	6379/TCP	69s

NAME	ENDPOINTS	AGE
endpoints/redis-svc	10.244.3.102:6379	69s

3.3.3.2 Tcpsocket 方式案例

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#tcp-probes>

使用此配置，kubelet 将尝试在指定端口上打开容器的套接字。如果可以建立连接，容器被认为是健康的，如果不能就认为是失败的，实际上就是检查端口。

对于 TCP 探测而言，kubelet 在节点上（不是在 Pod 里面）发起探测连接，这意味着你不能在 `host` 参数上配置服务名称，因为 kubelet 不能解析服务名称。

3.3.3.2.1 Liveness 的 Tcpsocket 探针

范例：

```
[root@master1 ~]#cat pod-liveness-tcpsocket.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-liveness-tcpsocket
```

```
  namespace: default
```

```
spec:
```

```
  containers:
```

```
  - name: pod-liveness-tcpsocket-container
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
```

```
    imagePullPolicy: IfNotPresent
```

```
    ports:
```

```
    - name: http #给指定端口定义别名
```

```
      containerPort: 80
```

```
    securityContext: #添加特权，否则添加iptables规则会提示: getsockopt failed
```

```
strangely: Operation not permitted
```

```
    capabilities:
```

```
      add:
```

```
      - NET_ADMIN
```

```
  livenessProbe:
```

```
    tcpSocket:
```

```
      port: http #引用上面端口的定义
```

```
      periodSeconds: 5
```

```
      initialDelaySeconds: 5
```

#注意：由于此镜像应用对外暴露的端口是80端口，所以要探测80端口

```
[root@master1 ~]#kubectl apply -f pod-liveness-tcpsocket.yaml
```

#查看状态正常

```
[root@master1 ~]#kubectl get pod pod-liveness-tcpsocket
```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-tcpsocket	1/1	Running	0	2s


```
#模拟探测失败,添加防火墙规则,禁止探测
[root@master1 ~]#kubectl exec pod-liveness-tcpsocket -- iptables -A INPUT -p tcp --dport 80 -j REJECT

#查看状态异常
[root@master1 ~]#kubectl describe pod pod-liveness-tcpsocket
.....
Events:
  Type            Reason            Age             From              Message
  ----            -
  Normal          Scheduled         54s             default-scheduler Successfully assigned
  default/pod-liveness-tcpsocket to node1.wang.org
  Normal          Pulled            53s             kubelet           Container image
  "wangxiaochun/pod-test:v0.1" already present on machine
  Normal          Created           53s             kubelet           Created container
  pod-liveness-tcpsocket-container
  Normal          Started           53s             kubelet           Started container
  pod-liveness-tcpsocket-container
  warning          Unhealthy         13s (x3 over 23s) kubelet           Liveness probe
  failed: dial tcp 172.16.3.43:80: i/o timeout
  Normal          Killing           13s             kubelet           Container pod-
  liveness-tcpsocket-container failed liveness probe, will be restarted

#会自动重启,但无法清除iptables规则,所以会循环重启
[root@master1 ~]#kubectl get pod -w
NAME                                READY    STATUS    RESTARTS   AGE
pod-liveness-tcpsocket              1/1     Running   0          70s
pod-liveness-tcpsocket              1/1     Running   1 (0s ago) 96s
pod-liveness-tcpsocket              1/1     Running   2 (0s ago) 2m26s

#手动清除防火墙规则后,不再重启
[root@master1 ~]#kubectl exec pod-liveness-tcpsocket -- iptables -F
```

3.3.3.2 Readiness 的 Tcpsocket 探针

范例:

```
[root@master1 ~]#cat pod-readiness-tcpsocket.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-readiness-tcpsocket
  labels:
    app: pod-readiness-tcpsocket
spec:
  containers:
  - name: pod-readiness-tcpsocket-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    readinessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 15
```

```

    periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: pod-readiness-tcpsocket-svc
spec:
  ports:
    - name: http
      port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: pod-readiness-tcpsocket #指定上面Pod相同的标签

[root@master1 ~]#kubectl apply -f pod-readiness-tcpsocket.yaml

[root@master1 ~]#kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
                                NOMINATED NODE   READINESS GATES
pod-readiness-tcpsocket            1/1     Running   0           17s   172.16.3.41     node1.wang.org
node1.wang.org                    <none>   <none>
[root@master1 ~]#curl 172.16.3.41
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>

#删除Pod
[root@master1 ~]#kubectl delete pod pod-readiness-tcpsocket

#修改端口为8080
[root@master1 ~]#vim pod-readiness-tcpsocket.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-readiness-tcpsocket
spec:
  containers:
    - name: pod-readiness-tcpsocket-container
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      readinessProbe:
        tcpSocket:
          port: 8080 #只修改此行，不会导致重启Pod
          initialDelaySeconds: 5
          periodSeconds: 10
      livenessProbe:
        tcpSocket:
          port: 80 #如果修改此行为8080，才会导致Pod重启
          initialDelaySeconds: 15
          periodSeconds: 20

#查看状态,不会重启Pod
[root@master1 ~]#kubectl get pod -w
NAME                                READY   STATUS    RESTARTS   AGE
pod-readiness-tcpsocket            0/1     Running   0           22s

#显示readinessProbe失败，但不会重启Pod

```

```
[root@master1 ~]#kubectl describe pod pod-readiness-tcpsocket
.....
    Ready:          False
    Restart Count:   2
    Liveness:        tcp-socket :80 delay=15s timeout=1s period=20s #success=1
#failure=3
    Readiness:       tcp-socket :8080 delay=5s timeout=1s period=10s #success=1
#failure=3
    .....
```

Type	Reason	Age	From	Message
Normal	Scheduled	2m3s	default-scheduler	Successfully assigned default/pod-readiness-tcpsocket to node2.wang.org
Normal	Pulled	2m2s	kubelet	Container image "wangxiaochun/nginx:1.20.0" already present on machine
Normal	Created	2m2s	kubelet	Created container pod-readiness-tcpsocket-container
Normal	Started	2m2s	kubelet	Started container pod-readiness-tcpsocket-container
Warning	Unhealthy	3s (x13 over 113s)	kubelet	Readiness probe failed: dial tcp 10.244.2.115:8080: connect: connection refused

#可以看到：由于容器内部没有该端口，所以探测的时候是失败的，所以pod一直无法正常启动。

3.3.3.3 HttpGet 方式案例

HTTP 探测通过对容器内容开放的web服务，进行http方法的请求探测，如果探测成功(状态码为2XX和3XX)，那么表示http服务是正常的，否则就是失败的。

HTTP Probes 允许针对 httpGet 配置额外的字段：

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#http-probes>

host：连接使用的主机名，默认是 Pod 的 IP。也可以在 HTTP 头中设置 “Host” 来代替。一般不配置此项

scheme：用于设置连接主机的方式（HTTP 还是 HTTPS）。默认是 "HTTP"。一般不配置此项

path：访问 HTTP 服务的路径。默认值为 "/"。一般会配置此项

port：访问容器的端口号或者端口名。如果数字必须在 1~65535 之间。一般会配置此项

httpHeaders：请求中自定义的 HTTP 头。HTTP 头字段允许重复。一般不配置此项

#示例：

httpHeaders:

- name: user_agent
value: curl

3.3.3.3.1 Liveness 的 httpGet 探针

范例:

```
[root@master1 ~]#cat pod-liveness-http.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-http
spec:
```

```

containers:
- name: pod-liveness-http-container
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0 #系统镜像
  #无法持续前台运行，所以无法启动，k8s会自动重启
  ports:
  - name: http
    containerPort: 80
  livenessProbe: #无livenessProbe，k8s也会自动重启
    httpGet:
      port: http #或者指定80，如果非标准端口可以指定端口号
      path: /index.html
      initialDelaySeconds: 1
      periodSeconds: 3

```

```

[root@master1 ~]#kubectl apply -f pod-liveness-http.yaml
pod/pod-liveness-http created

```

#状态检查

```

[root@master1 ~]# kubectl describe pod pod-liveness-http

```

```

.....
    Port:          80/TCP
    Host Port:     0/TCP
    State:         waiting
      Reason:      CrashLoopBackOff
    Last State:    Terminated
      Reason:      Completed
    Exit Code:     0
    Started:       Thu, 17 Mar 2020 22:30:54 +0800
    Finished:      Thu, 17 Mar 2020 22:30:54 +0800
    Ready:         False
    Restart Count: 2
    Liveness:      http-get http://:http/index.html delay=1s timeout=1s
period=3s #success=1 #failure=3
.....
Events:
  Type     Reason      Age           From          Message
  ----     -
  Normal    Scheduled   42s           default-scheduler Successfully assigned
default/pod-liveness-http to node2.wang.org
  Normal    Pulled      41s           kubelet       Successfully pulled
image "busybox:latest" in 404.22751ms
  Normal    Created     25s (x2 over 41s) kubelet       Created container
pod-liveness-http-container
  Normal    Started     25s (x2 over 41s) kubelet       Started container
pod-liveness-http-container
  Normal    Pulled      25s           kubelet       Successfully pulled
image "busybox:latest" in 15.428359037s
  Warning   BackOff     22s (x3 over 24s) kubelet       Back-off restarting
failed container
  Normal    Pulling     8s (x3 over 42s) kubelet       Pulling image
"busybox:latest"
.....

```

```

[root@master1 ~]#kubectl get pod -w

```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-http	0/1	CrashLoopBackOff	1 (21s ago)	39s
pod-liveness-http	0/1	Completed	2 (30s ago)	48s
pod-liveness-http	0/1	CrashLoopBackOff	2 (2s ago)	49s

pod-liveness-http	0/1	Completed	3 (44s ago)	91s
pod-liveness-http	0/1	CrashLoopBackOff	3 (2s ago)	93s
pod-liveness-http	0/1	Completed	4 (58s ago)	2m29s
pod-liveness-http	0/1	CrashLoopBackOff	4 (1s ago)	2m30s.

范例:

```
[root@master1 ~]#cat pod-liveness-httpget.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-httpget
  namespace: default
spec:
  containers:
  - name: pod-liveness-httpget-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    livenessProbe:
      httpGet:
        path: '/livez'
        port: 80
        scheme: HTTP
        initialDelaySeconds: 5
[root@master1 ~]#kubectl apply -f pod-liveness-httpget.yaml
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS GATES			
pod-liveness-httpget	1/1	Running	0	59s	172.16.3.13	
node1.wang.org	<none>		<none>			

```
[root@master1 ~]#curl -I 172.16.3.13/livez
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 2
Server: werkzeug/1.0.0 Python/3.8.2
```

#模拟失败

```
[root@master1 ~]#curl -XPOST -d 'livez=FAIL' 172.16.3.13/livez
[root@master1 ~]#curl -I 172.16.3.13//livez
HTTP/1.0 506 VARIANT ALSO NEGOTIATES
Content-Type: text/html; charset=utf-8
Content-Length: 4
Server: werkzeug/1.0.0 Python/3.8.2
```

#实时监控

```
[root@master1 ~]#kubectl get pod pod-liveness-httpget -w
```

NAME	READY	STATUS	RESTARTS	AGE
pod-liveness-httpget	1/1	Running	0	2m34s
pod-liveness-httpget	1/1	Running	1 (1s ago)	3m21s

#查看状态

```
[root@master1 ~]#kubectl describe pod pod-liveness-httpget
```

warning unhealthy	4m1s (x3 over 4m21s)	kubelet	Liveness probe
failed: HTTP probe failed with statuscode: 506			

```
Normal    Killing    4m1s      kubelet      Container pod-
liveness-httpget-container failed liveness probe, will be restarted
#结果提示: (x2 over 6m20s) 表示连续提示3次提示失败就表示失败, 然后重启pod
```

3.3.3.2 Readiness 的 httpGet 探针

通过readiness的属性, 尝试判断资源对象是否准备好了相关服务, 来接受用户请求。

如果readiness检测失败, 相关的service会将此pod从Endpoints中移除, 但不会重启pod

范例:

```
[root@master1 ~]#cat pod-readiness-http.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-readiness-http
spec:
  containers:
  - name: pod-readiness-http-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    ports:
    - name: http
      containerPort: 80
    readinessProbe:
      httpGet:
        port: http
        path: /index.html
        initialDelaySeconds: 1
        periodSeconds: 3

#创建容器
[root@master1 ~]#kubectl apply -f pod-readiness-http.yaml

#实时监控
[root@master1 ~]#kubectl get pod -w
NAME                READY   STATUS             RESTARTS   AGE
pod-readiness-http  0/1     CrashLoopBackOff   1 (26s ago) 43s
pod-readiness-http  0/1     Completed          2 (32s ago) 49s

#状态检测
[root@master1 ~]#kubectl describe pod pod-readiness-http
....
State:          waiting
Reason:         CrashLoopBackOff
Last State:     Terminated
Reason:         Completed
Exit Code:      0
Started:        Fri, 28 Jun 2021 18:11:44 +0800
Finished:        Fri, 28 Jun 2021 18:11:44 +0800
Ready:          False
Restart Count:  3
Readiness:      http-get http://:http/index.html delay=1s timeout=1s
period=3s #success=1 #failure=3
.....

#删除资源
[root@master1 ~]#kubectl delete -f pod-readiness-http.yaml
```

范例:

```
[root@master1 ~]#cat pod-readiness-httpget.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-readiness-httpget
  namespace: default
  labels:
    app: pod-readiness-httpget
spec:
  containers:
  - name: pod-readiness-httpget-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    readinessProbe:
      httpGet:
        path: '/readyz'
        port: 80
        scheme: HTTP
        initialDelaySeconds: 15 #首次监测需要等15s
        timeoutSeconds: 2
        periodSeconds: 5
        failureThreshold: 3
      restartPolicy: Always

---
#创建service文件
apiVersion: v1
kind: Service
metadata:
  name: pod-readiness-httpget-svc
spec:
  ports:
  - name: http
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: pod-readiness-httpget #指定上面Pod相同的标签

#创建容器和service
[root@master1 ~]#kubectl apply -f pod-readiness-httpget.yaml

#刚开始不会马上READY
[root@master1 ~]#kubectl get pod

```

NAME	READY	STATUS	RESTARTS	AGE
pod-readiness-httpget	0/1	Running	0	7s

```

#刚开始service中Endpoints没有对应的pod
[root@master1 ~]#kubectl describe svc pod-readiness-httpget-svc|grep Endpoints
Endpoints:

#过一会儿变为READY
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS GATES			
pod-readiness-httpget	1/1	Running	0	35s	172.16.3.15	
node1.wang.org	<none>	<none>				

#过一会儿service中Endpoints出现对应的pod

[root@master1 ~]#kubectl get ep pod-readiness-httpget-svc

NAME	ENDPOINTS	AGE
pod-readiness-httpget-svc	172.16.3.15:80	117s

[root@master1 ~]#kubectl describe svc pod-readiness-httpget-svc

```

Name:                pod-readiness-httpget-svc
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:            app=pod-readiness-httpget
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  192.168.105.57
IPs:                 192.168.105.57
Port:                http 80/TCP
TargetPort:          80/TCP
Endpoints:           172.16.3.15:80
Session Affinity:    None
Events:              <none>

```

#默认访问正常

```

[root@master1 ~]#curl -I 172.16.3.15/readyz
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 2
Server: Werkzeug/1.0.0 Python/3.8.2
Date: Thu, 17 Mar 2022 15:01:38 GMT

```

#通过service和Pod的IP都可以访问

```

[root@master1 ~]#curl 192.168.105.57/readyz
OK
[root@master1 ~]#curl 172.16.3.15/readyz
OK

```

#持续访问Service观察结果，可以访问

```

[root@master1 ~]#while true;do curl 192.168.105.57;sleep 1;done
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: pod-liveness-
httpget , ServerIP: 192.168.105.57!

```

#模拟访问故障

```

[root@master1 ~]#curl -XPOST -d 'readyz=FAIL' 172.16.3.15/readyz

```

#持续访问Service观察结果，访问失败

```

[root@master1 ~]#while true;do curl 192.168.105.57;sleep 1;done
curl: (7) Failed to connect to 192.168.105.57 port 80: Connection refused

```

#访问失败

```

[root@master1 ~]#curl -I 172.16.3.15/readyz
HTTP/1.0 507 INSUFFICIENT STORAGE
Content-Type: text/html; charset=utf-8
Content-Length: 4

```


Server: werkzeug/1.0.0 Python/3.8.2
Date: Thu, 17 Mar 2022 15:02:20 GMT

#结果显示: 虽然容器是running状态, 但是就绪状态不正常, 所以READY为 0/1

[root@master1 ~]#kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS	GATES		
pod-readiness-httpget	0/1	Running	0	2m38s	172.16.3.15	node1.wang.org
	<none>		<none>			

#查看状态, 不会自动重启Pod

[root@master1 ~]#kubectl describe pod pod-readiness-httpget

Ready: False

Restart Count: 0

Readiness: http-get http://:80/readyz delay=15s timeout=2s period=5s

#success=1 #failure=3

.....

warning Unhealthy 4m29s (x3 over 4m39s) kubelet Readiness probe
failed: Get "http://172.16.3.15:80/readyz": context deadline exceeded
(Client.Timeout exceeded while awaiting headers)

warning Unhealthy 96s (x19 over 3m1s) kubelet Readiness probe
failed: HTTP probe failed with statuscode: 507

#发现service将pod从Endpoints中移除

[root@master1 ~]#kubectl get ep pod-readiness-httpget-svc

NAME	ENDPOINTS	AGE
pod-readiness-httpget-svc		3m58s

[root@master1 ~]#kubectl describe svc pod-readiness-httpget-svc

Name: pod-readiness-httpget-svc

Namespace: default

Labels: <none>

Annotations: <none>

Selector: app=pod-readiness-httpget

Type: ClusterIP

IP Family Policy: SingleStack

IP Families: IPv4

IP: 192.168.105.57

IPs: 192.168.105.57

Port: http 80/TCP

TargetPort: 80/TCP

Endpoints:

Session Affinity: None

Events: <none>

#手动恢复正常

[root@master1 ~]#curl -XPOST -d 'readyz=OK' 172.16.3.15/readyz

[root@master1 ~]#curl 172.16.3.15/readyz -I

HTTP/1.0 200 OK

Content-Type: text/html; charset=utf-8

Content-Length: 2

Server: werkzeug/1.0.0 Python/3.8.2

Date: Thu, 17 Mar 2022 15:36:49 GMT

#验证还原

[root@master1 ~]#kubectl get pod

```

NAME                READY    STATUS    RESTARTS   AGE
pod-readiness-httpget 1/1      Running   0           6m19s
#结果显示：一旦就绪状态ok，则READY变为 1/1

#查看状态
[root@master1 ~]#kubectl describe pod pod-readiness-httpget
.....
    Ready:          True
    Restart Count:   0
    Readiness:       http-get http://:80/readyz delay=15s timeout=2s period=5s
#success=1 #failure=3

#service再次将pod加入
kubectl get ep pod-readiness-httpget-svc
NAME                                ENDPOINTS          AGE
pod-readiness-httpget-svc          172.16.3.15:80     7m21s

[root@master1 ~]#kubectl describe svc pod-readiness-httpget-svc
Name:                             pod-readiness-httpget-svc
Namespace:                         default
Labels:                           <none>
Annotations:                       <none>
Selector:                         app=pod-readiness-httpget
Type:                             ClusterIP
IP Family Policy:                 SingleStack
IP Families:                      IPv4
IP:                                192.168.105.57
IPs:                              192.168.105.57
Port:                             http 80/TCP
TargetPort:                       80/TCP
Endpoints:                       172.16.3.15:80
Session Affinity:                 None
Events:                           <none>

#持续访问Service观察结果，可以看到恢复访问
[root@master1 ~]#while true;do curl 192.168.105.57;sleep 1;done
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: pod-liveness-
httpget , ServerIP: 192.168.105.57!

```

3.4 Pod 资源限制

3.4.1 资源限制说明

Kubernetes是分布式的容器管理平台，所有资源都有Node节点提供，而Node节点上运行着很多Pod业务有很多，在一个项目中，有的业务占用的资源比重大，有的小，想要高效的利用Node资源，必须进行资源限制，那么Pod的资源限制应该如何处理？

kubernetes 可以支持在容器级及namespace级分别实现资源限制

Kubernetes 已经对Pod做了相应的资源配额设置，这些资源主要体现在：CPU和内存、存储，因为存储在k8s中有专门的资源对象来进行管控，所以当前的pod资源限制，主要指的计算资源，即CPU和内存。

为了方便与k8s的其他单独的资源对象区分开来，一般将CPU和内存其称为计算资源。

```
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/assign-cpu-resource/  
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/assign-memory-resource/
```

如果运行的Pod使用的资源超过宿主机的最大可用资源,会导致OOM和Pod驱逐到其它宿主机

3.4.1.1 可限制的资源单位

常见在容器级别的CPU和内存的限制

- CPU
特点：是一种可压缩资源，cpu资源是支持抢占的并共享的
单位：CPU的资源单位是CPU(Core)的数量,是一个绝对值
大小：在Kubernetes中通常以千分之一的CPU(Core)为最小单位，用毫 m 表示,即一个CPU核心表示为1000m
经验：一个资源占用不多的容器占用的CPU通常在100~300m，即0.1-0.3个CPU
注意：mi 代表是1024进制的
- 内存
特点：是不可压缩资源，当pod资源扩展的时候，如果node上资源不够，那么就会发生资源抢占，或者OOM问题
单位：内存的资源以字节数为单位，是一个绝对值
大小：内存配额对于绝大多数容器来说很重要，在Kubernetes中通常以Mi,Gi为单位来分配。通常分配1G,2G,最多16G或32G
注意：如果内存分配不足,可能会出现OOM现象

注意：

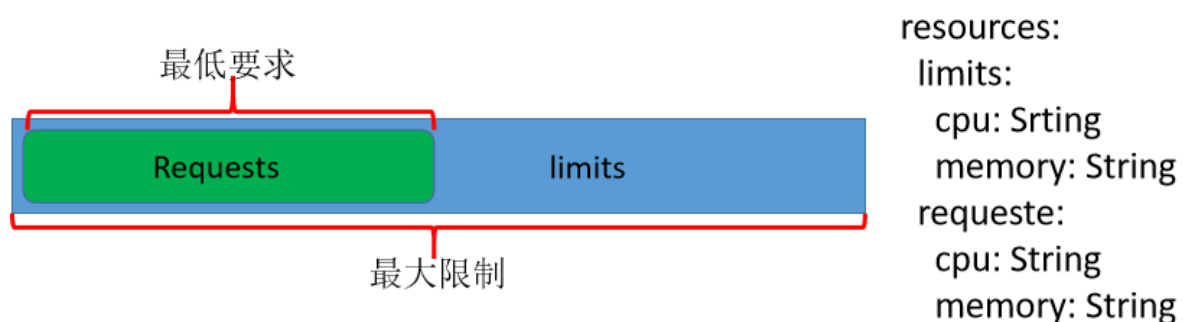
- CPU属于可压缩（compressible）型资源，即资源额度可按需收缩
- 内存（当前）则是不可压缩型资源，对其执行收缩操作可能会导致某种程度的问题，例如进程崩溃等。

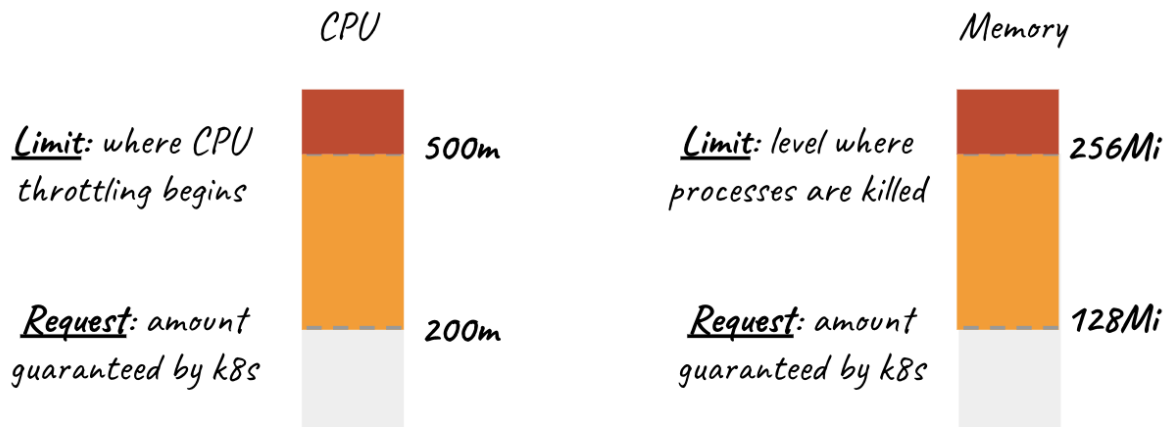
Extended Resources 扩展资源限制

- 所有不属于kubernetes.io域的资源,为扩展资源,如:"nvidia.com/gpu"
- kubernetes 也支持针对扩展资源限制

3.4.1.2 配额限制参数

Kubernetes中，对于每种资源的配额限定都需要两个参数：Requests和Limits





- 资源需求Requests

业务运行时资源预留的最小使用量，即所需资源的最低**下限**，该参数的值必须满足，若不满足，业务无法运行。

容器运行时可能用不到这些额度的资源，但用到时必须确保有相应数量的资源可用

这意味着即使此部分资源当前不使用，也不能分配给它人

资源需求的定义会影响调度器的决策,只会将Pod调度至满足所有容器总的资源需求的节点

当资源不足时，实际使用的资源超出 Requests 的部分，可能会被回收

不能超过对应的limits值

不能超过物理节点可用分配的资源值

相当于最低工资标准

- 资源限制 Limits

运行时资源允许使用最大可用量，即所需资源的最高**上限**，该参数的值不能被突破，超出该额度的资源使用请求通常会被拒绝

如果因为特殊原因突破此限制，该业务的资源对象会被重启或删除等意外操作

该限制需要大于等于requests的值，但系统在其某项资源紧张时，会从容器那里回收其使用的超出其requests值的那部分

针对内存而言,为防止上面回收情况的发生,一般建议将内存的 Requests 和 Limits 设为相同

资源限制的定義不影响调度器的决策

不能低于对应的limits值

可以超过物理节点可用分配的资源值

相当于公路上的限高杆，或者NBA球队的工资帽

提示:为保证性能,生产推荐Requests和Limits设置为相同的值

要实现资源限制,需要先安装metrics-server

官方链接

```
https://github.com/kubernetes-sigs/metrics-server
```

范例: 安装 metrics-server

```
#没有部署metrics-server时,会提示下面错误
[root@master1 ~]#kubectl top node
error: Metrics API not available
```

```

#建议安装metrics-server,可以通过dashboard查看更多信息,此步可选
#官方链接:https://github.com/kubernetes-sigs/metrics-server
[root@master1 ~]#curl -LO https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

#默认文件需要修改才能工作,因为默认需要内部证书验证和镜像地址k8s.gcr.io所以修改
[root@master1 ~]#vim components.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
labels:
  k8s-app: metrics-server
name: metrics-server
namespace: kube-system
spec:
selector:
  matchLabels:
    k8s-app: metrics-server
strategy:
  rollingUpdate:
    maxUnavailable: 0
template:
  metadata:
  labels:
    k8s-app: metrics-server
  spec:
    containers:
    - args:
      - --cert-dir=/tmp
      - --secure-port=4443
      - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
      - --kubelet-use-node-status-port
      - --metric-resolution=15s
      - --kubelet-insecure-tls #添加本行和下面两行
      image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server:v0.7.2
      #image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server:v0.7.1
      #image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server:v0.6.1
      #image: k8s.gcr.io/metrics-server/metrics-server:v0.6.1
      imagePullPolicy: IfNotPresent
      livenessProbe:
        .....

#应用此文件
[root@master1 ~]#kubectl apply -f components.yaml

#查看性能信息
[root@master1 ~]#kubectl top nodes

```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
master1.wang.org	121m	6%	1010Mi	54%
master2.wang.org	130m	6%	1091Mi	58%
master3.wang.org	116m	5%	1020Mi	55%
node1.wang.org	57m	2%	830Mi	44%
node2.wang.org	52m	2%	453Mi	24%
node3.wang.org	565m	28%	510Mi	27%

```
[root@master1 ~]#kubectl top pod
```

NAME	CPU(cores)	
MEMORY(bytes)		
pod-test-5b8795b9bc-99p8d	1m	17Mi
pod-test-5b8795b9bc-bg765	1m	18Mi
pod-test-5b8795b9bc-pg4rm	1m	27Mi

3.4.2 资源限制实现

范例: limits和requests值大小

```
[root@master1 ~]#cat pod-limit-request.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-limit-request
spec:
  containers:
  - name: pod-limit-request-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    imagePullPolicy: IfNotPresent
    resources:
      requests:
        memory: "500Mi"
        cpu: "250m"
      limits:
        memory: "200Mi"
        cpu: "500m"
```

#提示错误

```
[root@master1 ~]#kubectl apply -f pod-limit-request.yaml
The Pod "pod-limit-request" is invalid: spec.containers[0].resources.requests:
Invalid value: "500Mi": must be less than or equal to memory limit
```

#修改requests和limits值

```
[root@master1 ~]#vim pod-limit-request.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-limit-request
spec:
  containers:
  - name: pod-limit-request-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    imagePullPolicy: IfNotPresent
    resources:
      requests:
        memory: "200Mi" #修改此行
        cpu: "250m"
      limits:
        memory: "500Mi" #修改此行
        cpu: "500m"
```

#再次创建

```
[root@master1 ~]#kubectl apply -f pod-limit-request.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-limit-request	1/1	Running	0	11s

#验证结果

```
[root@master1 ~]#kubectl describe pod pod-limit-request
```

.....

Ready: True

Restart Count: 0

Limits:

cpu: 500m

memory: 500Mi

Requests:

cpu: 250m

memory: 200Mi

Environment: <none>

#说明limits必须大于等于requests值

#再次修改

```
[root@master1 ~]#vim pod-limit-request.yaml
```

apiVersion: v1

kind: Pod

metadata:

name: pod-limit-request

spec:

containers:

- name: pod-limit-request-container

image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0

imagePullPolicy: IfNotPresent

resources:

requests:

memory: "40000Mi" #超出物理节点内存

cpu: "250m"

limits:

memory: "50000Mi" #超出物理节点内存

cpu: "500m"

```
[root@master1 ~]#kubectl apply -f pod-limit-request.yaml
```

#显示为pending状态

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-limit-request	0/1	Pending	0	7s

#显示不可容忍的didn't tolerate

```
[root@master1 ~]#kubectl describe pod pod-limit-request
```

Events:

Type	Reason	Age	From	Message
Warning	FailedScheduling	109s	default-scheduler	0/6 nodes are available: 3 Insufficient memory, 3 node(s) had taint {node-role.kubernetes.io/master: }, that the pod didn't tolerate.
Warning	FailedScheduling	39s	default-scheduler	0/6 nodes are available: 3 Insufficient memory, 3 node(s) had taint {node-role.kubernetes.io/master: }, that the pod didn't tolerate.

#说明requests不可以超过物理节点可用的资源,而limits可以超过

#删除限制

```
[root@master1 ~]#kubectl delete limitranges limit-mem-cpu-per-container
```

#查看资源限制仍然存在

```
[root@master1 ~]#kubectl describe pod pod-nginx
```

#重新创建后限制取消

```
[root@master1 ~]#kubectl delete pod pod-nginx
```

```
[root@master1 ~]#kubectl apply -f pod-nginx.yaml
```

```
[root@master1 ~]#kubectl describe pod pod-nginx
```

范例: 压力测试

```
[root@master1 ~]#cat pod-stress.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-stress
```

```
spec:
```

```
  containers:
```

```
  - name: pod-stress
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/stress-ng:v0.03.11
```

```
    imagePullPolicy: IfNotPresent
```

```
    # -m 3 表示内存性能压力测试3个进程,每个进程默认占256M内存, -c 2 表示cpu压力测试2个进程
```

```
    #command: ["/usr/bin/stress-ng","-m 3","-c 2","--metrics-brief"]
```

```
    command: ["/usr/bin/stress-ng","-c 2","--metrics-brief"]
```

```
    resources:
```

```
      requests:
```

```
        memory: 128Mi
```

```
        cpu: 200m
```

```
      limits:
```

```
        memory: 256Mi
```

```
        cpu: 500m
```

```
[root@master1 ~]#kubectl apply -f pod-stress.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-stress	1/1	Running	0	3m33s

```
[root@master1 ~]#kubectl describe pod pod-stress
```

```
.....
```

```
  Limits:
```

```
    cpu: 500m
```

```
    memory: 256Mi
```

```
  Requests:
```

```
    cpu: 200m
```

```
    memory: 128Mi
```

#查看效果

```
[root@master1 ~]#kubectl exec pod-stress -- top
```

```
Mem: 2064504K used, 1930156K free, 4280K shrd, 191364K buff, 1201540K cached
```

```
CPU: 26% usr 0% sys 0% nic 72% idle 0% io 0% irq 0% sirq
```

```
Load average: 1.13 1.33 0.75 3/374 16
```

```
  PID  PPID  USER      STAT  VSZ %VSZ  CPU %CPU  COMMAND
```

```
    7    1  root       R     6900   0%    1  12% {stress-ng-cpu} /usr/bin/stress-ng
```



```

      8      1 root      R      6900    0%    0    12% {stress-ng-cpu} /usr/bin/stress-ng
      1      0 root      S      6256    0%    0    0% /usr/bin/stress-ng -c 2 --metrics-
      9      0 root      R      1512    0%    1    0% top
...

```

#当前worker节点主机是2核，CPU资源限制是500m即0.5核，相当于cpu全部资源的1/4，也就是25%左右，而这里一共是两个线程，所以每个占据12%左右。

#在运行Pod的节点查看可以看到下面状态

```
[root@node2 ~]#docker stats --no-stream 19770849562a
```

```

CONTAINER ID   NAME
CPU %          MEM USAGE / LIMIT   MEM %           NET I/O        BLOCK I/O       PIDS
19770849562a   k8s_pod-stress_pod-stress_default_a7822b5c-149d-40b1-84e2-af0de23a23a6_0      50.63%          7.086MiB / 256MiB   2.77%           0B / 0B          0B / 0B          3

```

#如果安装了metrics-server,多次执行kubectl top pod可以看到性能状态,刚开始会显示如下

```
[root@master1 ~]#kubectl top pod pod-stress
```

```
Error from server (NotFound): podmetrics.metrics.k8s.io "default/pod-stress" not found
```

#过一会儿会显示下面性能信息

```
[root@master1 ~]#kubectl top pod pod-stress
```

```

NAME           CPU(cores)   MEMORY(bytes)
pod-stress      500m         7Mi

```

范例: 内存OOM

```
[root@master1 ~]#cat pod-memleak.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-memleak
```

```
spec:
```

```
  containers:
```

```
  - name: simmemleak
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/simmemleak:v0.1
```

```
    resources:
```

```
      requests:
```

```
        memory: "64Mi"
```

```
        cpu: "1"
```

```
      limits:
```

```
        memory: "64Mi"
```

```
        #memory: "6400Mi" #simmemleak:v0.1有bug, 最多的内存, 也会出现OOMKilled
```

```
        cpu: "1"
```

```
[root@master1 ~]#kubectl apply -f pod-memleak.yaml
```

```
[root@master1 ~]#kubectl describe pods pod-memleak
```

```
.....
```

```

State:           waiting
Reason:          CrashLoopBackOff
Last State:      Terminated
Reason:          OOMKilled
.....

```

```
[root@master1 ~]#kubectl get pod -w
```

```

NAME          READY   STATUS             RESTARTS   AGE
pod-memleak    0/1     CrashLoopBackOff   3 (55s ago) 2m42s
pod-memleak    0/1     OOMKilled           4 (70s ago) 2m57s

```

pod-memleak	0/1	CrashLoopBackOff	4 (13s ago)	3m10s
pod-memleak	0/1	OOMKilled	5 (98s ago)	4m35s
pod-memleak	0/1	CrashLoopBackOff	5 (13s ago)	4m48s

范例: 通过kubectl top 查看性能

```
[root@master1 ~]#cat deployment-pod-stress.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-pod-stress
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pod-stress
  template:
    metadata:
      labels:
        app: pod-stress
    spec:
      containers:
      - name: pod-stress
        image: registry.cn-beijing.aliyuncs.com/wangxiaochun/stress-ng:v0.03.11
        imagePullPolicy: IfNotPresent
        # -m 3 表示内存性能压力测试3个进程 -c 2 表示cpu压力测试2个进程
        #command: ["/usr/bin/stress-ng","-m 4","-c 2","--metrics-brief"]
        command: ["/usr/bin/stress-ng","-c 2","--metrics-brief"]
        resources:
          requests:
            memory: 128Mi
            cpu: 200m
          limits:
            memory: 256Mi
            cpu: 500m

[root@master1 ~]#kubectl top pod
```

3.4.3 LimitRange 实现使用资源限制

默认情况下，Kubernetes 集群上的容器运行使用的[计算资源](#)没有限制。

使用资源配额，集群管理员可以以[名字空间](#)为单位，限制其资源的使用与创建。

在命名空间中，一个 Pod 或 Container 最多能够使用命名空间的资源配额所定义的 CPU 和内存用量。

有人担心，一个 Pod 或 Container 会垄断所有可用的资源。

LimitRange 是在命名空间内限制资源分配（给多个 Pod 或 Container）的策略对象。

一个 **LimitRange（限制范围）** 对象提供的限制能够做到：

- 在一个命名空间中实施对每个 Pod 或 Container 最小和最大的资源使用量的限制。
- 在一个命名空间中实施对每个 PersistentVolumeClaim 能申请的最小和最大的存储空间大小的限制。
- 在一个命名空间中实施对一种资源的申请值和限制值的比值的控制。
- 设置一个命名空间中对计算资源的默认申请/限制值，并且自动的在运行时注入到多个 Container 中。

```
https://kubernetes.io/zh-cn/docs/concepts/policy/limit-range/  
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/manage-resources/  
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/manage-resources/cpu-  
default-namespace/  
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/manage-resources/cpu-  
constraint-namespace/  
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/manage-  
resources/memory-default-namespace/  
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/manage-  
resources/memory-constraint-namespace/
```

能够使用限制范围创建的策略示例有：

- 在一个有两个节点，8 GiB 内存与16个核的集群中，限制一个命名空间的 Pod 申请 100m 单位，最大 500m 单位的 CPU，以及申请 200Mi，最大 600Mi 的内存。
- 为 spec 中没有 cpu 和内存需求值的 Container 定义默认 CPU 限制值与需求值 150m，内存默认需求值 300Mi。

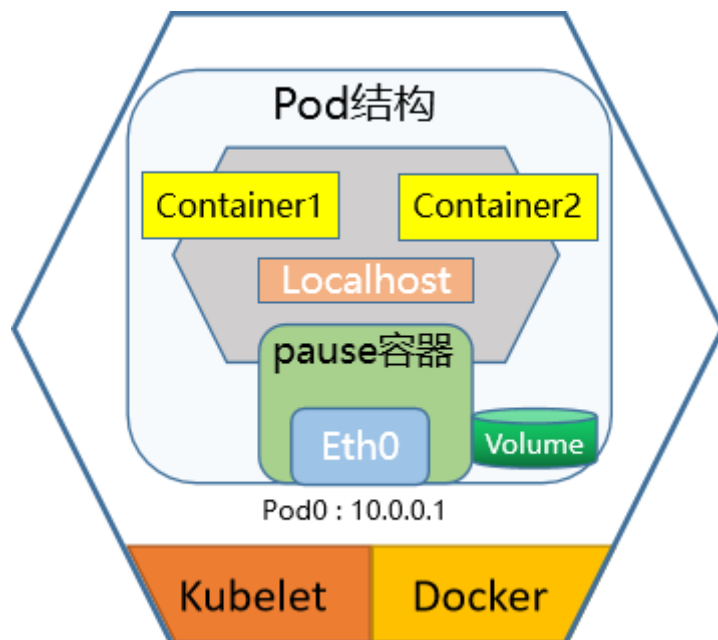
3.5 Pod 安全

3.5.1 资源安全属性

容器一般是基于6个命名空间实现资源的隔离，而一个 Pod 就是一个容器集合，包含多个容器

这个命名空间是按照下面方式来使用的：

- Pod内部的所有容器共享底层 pause容器的 UTS，Network，IPC 三个名称空间资源
- 可选共享命名空间：PID
- MNT和USER 两个命名空间通常没有共享,从而实现应用的隔离。



容器安全上下文context

pod在其生命周期中，涉及到多种场景(多容器的关联关系)，将这种相关的作用关系称为容器上下文，其中有一些涉及到权限、限制等安全相关的内容，称其为安全上下文。

安全上下文是一组用于决定容器是如何创建和运行的约束条件，它们代表着创建和运行容器时使用的运行时参数。

它根据约束的作用范围主要包括三个级别：

- Pod级别：针对pod范围内的所有容器
- 容器级别：仅针对pod范围内的指定容器
- PSP级别：PodSecurityPolicy，全局级别的Pod安全策略，涉及到准入控制相关知识

Pod和容器的安全上下文设置主要包括以下几个方面

- 容器进程运行身份及资源访问权限
- Linux Capabilities
- 自主访问控制DAC
- seccomp: securecomputing mode，实现限制程序使用某些系统调用
- AppArmor: Application Armor 与SELinux类似，可以限制程序的功能，比如程序可以读、写或运行哪些文件，是否可以打开端口等
- SELinux
- Privilege Mode
- Privilege Escalation 特权提升

安全上下文相关属性

```
#Pod级安全上下文
~]#kubectl explain pod.spec.securityContext
#容器级安全上下文
~]#kubectl explain pod.spec.containers.securityContext
apiVersion: v1
kind: Pod
metadata: {...}
spec:
  securityContext:                                # Pod级别的安全上下文，对内部所有容器均有效
    runAsUser <integer>                          # 以指定的用户身份运行容器进程，默认由镜像中的USER指定
    runAsGroup <integer>                          # 以指定的用户组运行容器进程，默认使用的组随容器运行时
    supplementalGroups <[]integer>               # 为容器中1号进程的用户添加的附加组
    fsGroup <integer>                             # 为容器中的1号进程附加的一个专用组，其功能类似于sgid
    runAsNonRoot <boolean>                        # 是否以非root身份运行
    selinuxOptions <Object>                       # SELinux的相关配置
    sysctls <[]Object>                            # 应用到当前Pod上的名称或网络空间级别的sysctl参数设置列表
    windowsOptions <Object>                       # windows容器专用的设置
  containers:
    - name: ...
      image: ...
      securityContext:                             # 容器级别的安全上下文，仅生效于当前容器
        runAsUser <integer>                       # 以指定的用户身份运行容器进程
        runAsGroup <integer>                     # 以指定的用户组运行容器进程
        runAsNonRoot <boolean>                  # 是否以非root身份运行
        allowPrivilegeEscalation <boolean>       # 是否允许特权升级
        readOnlyRootFilesystem <boolean>        # 是否设置rootfs只读
        capabilities <Object>                   # 于当前容器上添加（add）或删除（drop）的操作内核某些资源的能力
          add <[]string>                         # 添加由列表格式定义的内核能力
          drop <[]string>                       # 移除由列表格式定义的内核能力
          privileged <boolean>                  # 是否运行特权容器
          procMount <string>                    # 设置容器的procMount类型，默认为DefaultProcMount;
          readOnlyRootFilesystem <boolean>       # 是否将根文件系统设置为只读模式
          selinuxOptions <Object>               # SELinux的相关配置
```

```
windowsoptions <Object>
```

```
# windows容器专用的设置
```

#注意：上面的属性仅是最常用的属性，而不是所有的属性

3.5.2 资源策略

<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/security-context/>

3.5.2.1 用户身份

默认Pod的容器进程是以root身份运行，可以通过下面属性指定其它用户

但此root用户只具有对容器内的相关资源比如文件系统等有管理权限，对于宿主机的内核不具有管理权限，即是个“伪”root用户

相关的属性：

- runAsUser
- runAsGroup

范例：默认为root身份运行pod内的进程

```
#定制资源配置文件
[root@master1 ~]#cat pod-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-test
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1

[root@master1 ~]#kubectl apply -f pod-test.yaml

#查看用户
[root@master1 ~]#kubectl exec pod-test -- id
uid=0(root) gid=0(root)
groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)

#删除资源
[root@master1 ~]#kubectl delete -f pod-test.yaml

#注意：默认情况下一旦pod创建好后，是不允许对用户归属权限进行任意修改的，所以需要修改的话，必须先关闭，再开启
```

范例：添加安全上下属性实现指定用户身份运行Pod内的进程

```
[root@master1 ~]#cat pod-securitycontext-runasuser.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-securitycontext-runasuser
  namespace: default
spec:
```

```

containers:
- name: pod-test
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
  imagePullPolicy: IfNotPresent
  #env:
  #- name: PORT
  #   value: "8080"
  securityContext:
    runAsUser: 1001      #指定运行身份
    runAsGroup: 1001

[root@master1 ~]#kubectl get pod pod-securitycontext-runasuser -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
NODE                                NOMINATED NODE   READINESS GATES
pod-securitycontext-runasuser       1/1     Running   0           25s   172.16.3.48
node1.wang.org                     <none>         <none>

[root@master1 ~]#curl 172.16.3.48:8080
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-
securitycontext-runasuser, ServerIP: 172.16.3.48!
[root@master1 ~]#kubectl exec pod-securitycontext-runasuser -- ps aux
PID    USER     TIME   COMMAND
   1   1001      0:00 python3 /usr/local/bin/demo.py
   9   1001      0:00 ps aux

[root@master1 ~]#kubectl exec pod-securitycontext-runasuser -- id
uid=1001 gid=1001

[root@master1 ~]#kubectl exec -it pod-securitycontext-runasuser -- sh
[@pod-securitycontext-runasuser /]$ ss -nltnl
State          Recv-Q          Send-Q          Local
Address:Port    Peer Address:Port Process

LISTEN         0                128             0.0.0.0:88
0.0.0.0:88     0.0.0.0:*

#默认Pod内所有端口都是非特权端口
[@pod-securitycontext-runasuser /]$ sysctl -a |grep
net.ipv4.ip_unprivileged_port_start
net.ipv4.ip_unprivileged_port_start = 0

#Pod和宿主机的内核配置不同
[root@node3 ~]#sysctl -a|grep net.ipv4.ip_unprivileged_port_start
net.ipv4.ip_unprivileged_port_start = 1024

```

3.5.2.2 资源能力

参考资料

<https://man7.org/linux/man-pages/man7/capabilities.7.html>
~]#man 7 capabilities

Linux 中相关的资源能力

```
CAP_CHOWN      #改变文件的所有者和所属组
CAP_MKNOD      #mknod(), 创建设备文件
CAP_NET_ADMIN  #网络管理权限
CAP_SYS_TIME   #更改系统的时钟
CAP_SYS_MODULE #装载卸载内核模块
CAP_NET_BIND_SERVER: #允许普通用户绑定1024以内的特权端口
CAP_SYS_ADMIN  #大部分的管理权限,基本相当于root权限
```

范例: flannel的Pod中添加了Linux 中相关的资源能力

```
[root@master1 ~]#kubectl get pod -n kube-flannel kube-flannel-ds-jqvct -o yaml
|grep -A10 securityContext:
securityContext:
  capabilities:
    add:
      - NET_ADMIN
      - NET_RAW
    privileged: false
  terminationMessagePath: /dev/termination-log
  terminationMessagePolicy: File
  volumeMounts:
  - mountPath: /run/flannel
    name: run
```

范例: 默认能力无法直接创建iptables规则

```
#定制资源配置文件
[root@master1 ~]#cat pod-securitycontext-capabilities.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-securitycontext-capabilities
  namespace: default
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ["/bin/sh", "-c"]
    args: ["/sbin/iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT -
-to-port 80 && /usr/bin/python3 /usr/local/bin/demo.py"]
```

#注意: 可以在容器内部通过 `command + args` 运行一个自定义的容器启动命令

#创建资源

```
[root@master1 ~]#kubectl apply -f pod-securitycontext-capabilities.yaml
```

#检查效果

```
[root@master1 ~]#kubectl get pod pod-securitycontext-capabilities
NAME                                READY    STATUS    RESTARTS    AGE
pod-securitycontext-capabilities    0/1      Error     2 (30s ago)  67s
```

```
[root@master1 ~]#kubectl logs pod-securitycontext-capabilities
getsockopt failed strangely: Operation not permitted
```

#结果提示：当容器在启动的时候，默认是以linux系统普通用户启动的，所以普通用户是没有权限更改系统级别的内容的，所以导致我们更改命令权限失败

#删除

```
[root@master1 ~]#kubectl delete -f pod-securitycontext-capabilities.yaml
```

范例: 通过 add 指令添加特权

```
[root@master1 ~]#cat pod-securitycontext-capabilities.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-securitycontext-capabilities
  namespace: default
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ["/bin/sh", "-c"]
    args: ["/sbin/iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT -
-to-port 80 && /usr/bin/python3 /usr/local/bin/demo.py"]
#添加下面三行
  securityContext:
    capabilities:
      add: ['NET_ADMIN']
```

#注意：add后面的权限能力使用单引号(''),也可以使用双引号("")

#创建资源

```
[root@master1 ~]#kubectl apply -f pod-securitycontext-capabilities.yaml
```

#验证结果

```
[root@master1 ~]#kubectl get pod pod-securitycontext-capabilities -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS	GATES	
pod-securitycontext-capabilities	1/1	Running	0	67s	
172.16.7.45	node3.wang.org	<none>	<none>		

#结果显示：将容器里面的防火墙规则更改成功

```
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- iptables -nVL PREROUTING -t nat
```

Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)									
pkts	bytes	target	prot	opt	in	out	source	destination	
0	0	REDIRECT	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	
tcp dpt:8080 redir ports 80									

```
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- hostname -i
172.16.7.45
```

```
[root@master1 ~]#curl 172.16.7.45
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-
securitycontext-capabilities, ServerIP: 172.16.7.45!
```

```
[root@master1 ~]#curl 172.16.7.45:8080
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-
securitycontext-capabilities, ServerIP: 172.16.7.45!
```



```

#结果显示：默认情况下可以对容器里面的资源进行相应的修改权限操作
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- ls -l
/etc/hosts
-rw-r--r--    1 root    root           228 Mar 17 07:12 /etc/hosts
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- chown
1000:1000 /etc/hosts
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- ls -l
/etc/hosts
-rw-r--r--    1 1000    1000           228 Mar 17 07:12 /etc/hosts

#删除
[root@master1 ~]#kubectl delete -f pod-securitycontext-capabilities.yaml

```

范例: 使用 drop 指令删除特权

```

#修改资源配置文件
[root@master1 ~]#cat pod-securitycontext-capabilities.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-securitycontext-capabilities
  namespace: default
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ["/bin/sh", "-c"]
    args: ["/sbin/iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT -
-to-port 80 && /usr/bin/python3 /usr/local/bin/demo.py"]
    securityContext:
      capabilities:
        add: ['NET_ADMIN']
        drop: ['CHOWN'] #添加此行

#应用配置
[root@master1 ~]#kubectl apply -f pod-securitycontext-capabilities.yaml

#验证结果
[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- ls -l
/etc/hosts
-rw-r--r--    1 root    root           228 Mar 17 07:22 /etc/hosts

[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- chown
1001:1001 /etc/hosts
chown: /etc/hosts: Operation not permitted
command terminated with exit code 1

[root@master1 ~]#kubectl exec pod-securitycontext-capabilities -- ls -l
/etc/hosts
-rw-r--r--    1 root    root           228 Mar 17 07:22 /etc/hosts
#结果显示：文件权限不能随意的更改了

```

3.5.2.3 内核参数

默认情况下，Pod中的容器在启动的时候是具备了一定的特权用户权限的。

比如我们知道 kube-proxy 会将pod启动时候的一些规则转换成对应的iptables或者ipvs规则,这就涉及到宿主机内核级别的操作。

Pod 安全上下文级别默认只支持为数不多的内核安全参数,比如下面

```
net.ipv4.ip_local_port_range
net.ipv4.ip_unprivileged_port_start
net.ipv4.tcp_syncookies
kernel.shm_rmid_forced
```

上面几个内核参数可以在pod内部的容器自己独立的名称空间的内核级别进行调整，视为安全参数

但其它的绝大部分内核参数则不支持直接修改,视为不安全参数

如果想对其它不安全内核参数的修改，必须要所有的node节点上修改kubelet 的配置才能支持

注意：以上只能Pod级别修改，不支持在container级别设置

格式如下:

```
KUBELET_EXTRA_ARGS='--allowed-unsafe-
sysctls=net.core.somaxconn,net.ipv4.ip_nonlocal_bind'
```

范例: Pod可以有自己独立的内核配置

```
[root@master1 ~]#sysctl net.ipv4.ip_unprivileged_port_start
net.ipv4.ip_unprivileged_port_start = 1024

[root@master1 ~]#kubectl run myapp --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
pod/myapp created

#默认Pod内核中net.ipv4.ip_unprivileged_port_start = 0
[root@master1 ~]#kubectl exec -it myapp -- sh
[root@myapp /]# sysctl net.ipv4.ip_unprivileged_port_start
net.ipv4.ip_unprivileged_port_start = 0
```

范例:

```
[root@master1 ~]#cat pod-sysctl.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-sysctl
spec:
  securityContext:
    sysctls:
      - name: kernel.shm_rmid_forced
        value: "1"
      - name: net.ipv4.ip_unprivileged_port_start
        value: "60"
    # - name: net.core.somaxconn
    #   value: "6666"
```

```
containers:
- name: pod-sysctl
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
  securityContext:
    runAsUser: 1001
    runAsGroup: 1001
```

#参数说明:kernel.shm_rmid_forced 强制SHM空间和一个进程联系在一起，所以可以通过杀死进程来释放内存

```
[root@master1 ~]#kubectl get pod -o wide
NAME             READY   STATUS    RESTARTS   AGE     IP             NODE
NOMINATED NODE   READINESS GATES
pod-sysctl       1/1     Running   0           4m13s   172.16.3.29    node1.wang.org
<none>           <none>
```

[root@master1 ~]#curl 172.16.3.29
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-sysctl,
ServerIP: 172.16.3.29!

```
[root@master1 ~]#kubectl exec pod-sysctl -- cat /proc/sys/kernel/shm_rmid_forced
1
[root@master1 ~]#kubectl exec pod-sysctl -- cat
/proc/sys/net/ipv4/ip_unprivileged_port_start
60
```

范例:

```
#修改文件
[root@master1 ~]#cat pod-sysctl.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-sysctl
spec:
  securityContext:
    sysctls:
      - name: kernel.shm_rmid_forced
        value: "1"
      - name: net.ipv4.ip_unprivileged_port_start
        value: "100" #修改此行
      # - name: net.core.somaxconn
      #   value: "6666"
  containers:
  - name: pod-sysctl
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 #此容器使用
80端口
    securityContext:
      runAsUser: 1001
      runAsGroup: 1001

[root@master1 ~]#kubectl apply -f pod-sysctl.yml

[root@master1 ~]#kubectl get pod
NAME             READY   STATUS             RESTARTS   AGE
pod-sysctl       0/1     CrashLoopBackOff   1 (5s ago)  7s
```

```
#查看日志,提示1001不是特权用户,不能使用100以下的端口运行pod-test容器的80端口导致权限不足
[root@master1 ~]#kubectl logs pod-sysctl
Traceback (most recent call last):
  File "/usr/local/bin/demo.py", line 91, in <module>
    main(sys.argv[1:])
  File "/usr/local/bin/demo.py", line 87, in main
    app.run(host=str(host), port=int(port), debug=bool(debug))
  File "/usr/lib/python3.8/site-packages/flask/app.py", line 841, in run
    run_simple(host, port, self, **options)
  File "/usr/lib/python3.8/site-packages/werkzeug/serving.py", line 1052, in run_simple
    inner()
  File "/usr/lib/python3.8/site-packages/werkzeug/serving.py", line 996, in inner
    srv = make_server(
  File "/usr/lib/python3.8/site-packages/werkzeug/serving.py", line 862, in make_server
    return BaseWSGIServer(
  File "/usr/lib/python3.8/site-packages/werkzeug/serving.py", line 740, in __init__
    HTTPServer.__init__(self, server_address, handler)
  File "/usr/lib/python3.8/socketserver.py", line 452, in __init__
    self.server_bind()
  File "/usr/lib/python3.8/http/server.py", line 138, in server_bind
    socketserver.TCPServer.server_bind(self)
  File "/usr/lib/python3.8/socketserver.py", line 466, in server_bind
    self.socket.bind(self.server_address)
PermissionError: [Errno 13] Permission denied
```

#修改文件

```
[root@master1 ~]#cat pod-sysctl.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-sysctl
spec:
  securityContext:
    sysctls:
      - name: kernel.shm_rmid_forced
        value: "1"
      - name: net.ipv4.ip_unprivileged_port_start
        value: "100"
      # - name: net.core.somaxconn
      #   value: "6666"
  containers:
  - name: pod-sysctl
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    #securityContext:  #注释三行
    #  runAsUser: 1001
    #  runAsGroup: 1001
```

#再次执行成功

```
[root@master1 ~]#kubectl apply -f pod-sysctl.yml
```

#验证成功

```
[root@master1 ~]#kubectl exec pod-sysctl -- sysctl
net.ipv4.ip_unprivileged_port_start
```

```
net.ipv4.ip_unprivileged_port_start = 100

[root@master1 ~]#kubectl get pod -o wide
NAME             READY   STATUS    RESTARTS   AGE     IP             NODE
NOMINATED NODE   READINESS GATES
pod-sysctl       1/1     Running   0           3m40s   172.16.3.38    node1.wang.org
<none>           <none>
[root@master1 ~]#curl 172.16.3.38
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-sysctl,
ServerIP: 172.16.3.38!
```

范例:

```
[root@master1 ~]#cat pod-sysctl.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-sysctl
spec:
  securityContext:
    sysctls:
      - name: kernel.shm_rmid_forced
        value: "1"
      - name: net.ipv4.ip_unprivileged_port_start
        value: "60"
      - name: net.core.somaxconn
        value: "6666"
  containers:
    - name: pod-sysctl
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
      securityContext:
        runAsUser: 1001
        runAsGroup: 1001

[root@master1 ~]#kubectl apply -f pod-sysctl.yml
pod/pod-sysctl created

#结果显示: 默认情况下, 普通用户是不允许修改命名空间里相关内核参数的
[root@master1 ~]#kubectl get pod
NAME             READY   STATUS             RESTARTS   AGE
pod-sysctl       0/1     SysctlForbidden    0           7s

#显示内核参数net.core.somaxconn不在白名单中
[root@master1 ~]#kubectl describe pod pod-sysctl
Name:             pod-sysctl
.....
Status:           Failed
Reason:           SysctlForbidden
Message:          Pod forbidden sysctl: "net.core.somaxconn" not whitelisted
.....

#删除pod
[root@master1 ~]#kubectl delete pod pod-sysctl

#修改所有worker节点的kubelet的配置文件(如果文件不存在的话, 可以自己手工创建)
[root@node1 ~]#vim /etc/default/kubelet
```

```
[root@node1 ~]#cat /etc/default/kubelet
KUBELET_EXTRA_ARGS='--allowed-unsafe-sysctls=net.core.somaxconn'
#参数说明: net.core.somaxconn 网卡入栈的最大全连接队列

#在所有的node角色节点上配置该文件, 然后重启该服务
[root@node1 ~]#systemctl restart kubelet

#再次运行
[root@master1 ~]#kubectl apply -f pod-sysctl.yaml
pod/pod-sysctl created
[root@master1 ~]#kubectl get pod -o wide
NAME             READY   STATUS    RESTARTS   AGE   IP              NODE
NOMINATED NODE   READINESS GATES
pod-sysctl        1/1     Running   0           1s    172.16.3.37     node1.wang.org
<none>           <none>
#验证结果
[root@master1 ~]#curl 172.16.3.37
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-sysctl,
ServerIP: 172.16.3.37!
[root@master1 ~]#kubectl exec pod-sysctl -- cat /proc/sys/net/core/somaxconn
6666
```

3.5.2.4 特权

范例: Kube-proxy 的pod 添加特权

```
[root@master1 ~]#kubectl get pod -n kube-system kube-proxy-4qgmt -o yaml
|grep -A2 securityContext:
  securityContext:
    privileged: true
```

范例

```
[root@master1 ~]#cat pod-privileged.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-privileged-demo
spec:
  initContainers:
  - name: set-sysctl
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1
    command: ["sysctl", "-w", "vm.max_map_count=6666666"]
    #securityContext:
    #  privileged: true
  containers:
  - name: pod-privileged-demo
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1

[root@master1 ~]#kubectl apply -f pod-privileged.yaml

[root@master1 ~]#kubectl get pod
NAME             READY   STATUS    RESTARTS   AGE
pod-privileged-demo    0/1     Init:Error   0           33s

#修改
```

```
[root@master1 ~]#vim pod-privileged.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-privileged-demo
spec:
  initContainers:
  - name: set-sysctl
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1
    command: ["sysctl", "-w", "vm.max_map_count=666666"]
    securityContext:      #添加特权
      privileged: true
  containers:
  - name: pod-privileged-demo
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1

[root@master1 ~]#kubectl apply -f pod-privileged.yaml

[root@master1 ~]#kubectl get pod pod-privileged-demo
NAME                READY   STATUS    RESTARTS   AGE
pod-privileged-demo  1/1     Running   0           47s

#验证
[root@master1 ~]#kubectl exec pod-privileged-demo -- sysctl vm.max_map_count
Defaulted container "pod-privileged-demo" out of: pod-privileged-demo, set-
sysctl (init)
vm.max_map_count = 666666
```

3.6 Pod 服务质量 QoS

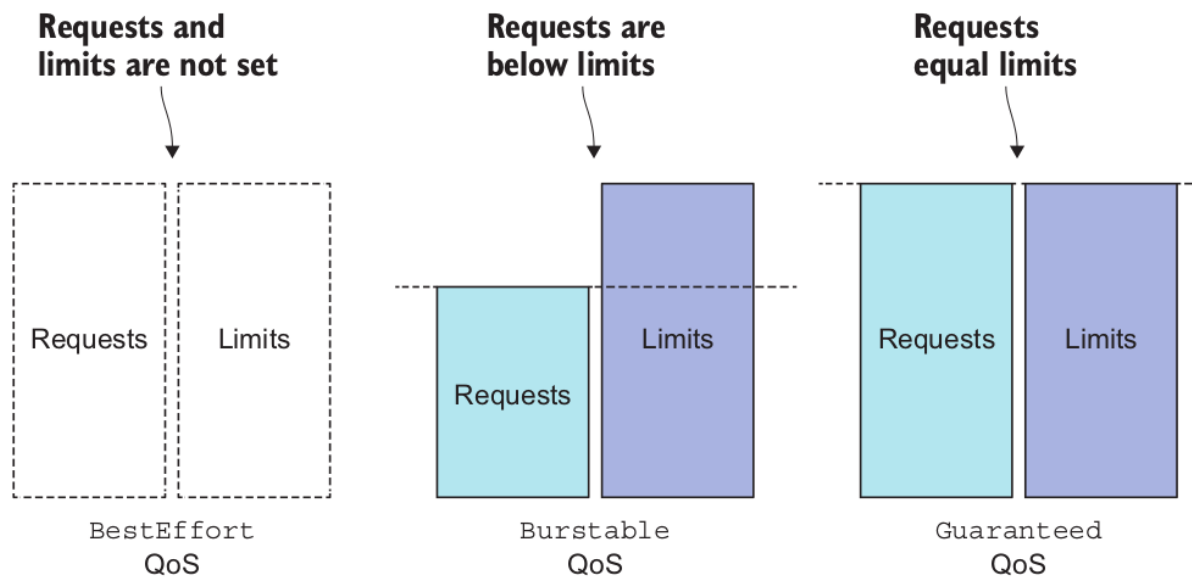
<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/quality-service-pod/>

3.6.1 服务质量分类

QoS（服务质量等级）是作用在 Pod 上的一个配置

当 Kubernetes 创建一个 Pod 时，它就会给这个 Pod 自动的分配一个 QoS 等级

QoS 三个等级



- 低优先级BestEffort
没有任何一个容器设置了requests或limits的属性。（最低优先级）
oom_score_adj: 1000
适用于可容忍型,要求不高的业务
- 中优先级Burstable 可爆破
Pod至少有一个容器设置了cpu或内存的requests和limits，且不相同
oom_score_adj: 2--999
适用于一般敏感型,需要弹性的业务
- 高优先级Guaranteed 保证
Pod内的每个容器同时设置了CPU和内存的requests和limits 而且所有值必须相等
oom_score_adj: -997
适用于敏感型,需要保障的业务

当主机出现OOM时,优先删除服务质量为BestEffort的Pod，然后在删除Burstable，Guaranteed最后被删除

如果所有Pod都设为最高优化级Guaranteed时,可能会导致资源浪费,一般只针对要求较高的特定应用才配置

3.6.2 案例

示例：高优先级

```
[root@master1 ~]#cat pod-qos-guaranteed.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-qos-guaranteed
spec:
  containers:
  - name: pod-qos-guaranteed-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    resources:
      limits:
        memory: "200Mi"
```



```

    cpu: "500m"
  requests:
    memory: "200Mi"
    cpu: "500m"

[root@master1 ~]#kubectl apply -f pod-qos-guaranteed.yaml

[root@master1 ~]#kubectl get pod
NAME                READY   STATUS    RESTARTS   AGE
pod-qos-guaranteed  1/1     Running   0           5s

[root@master1 ~]#kubectl describe pod pod-qos-guaranteed
....
QoS Class:           Guaranteed
.....

#查出Pod运行的主机节点
[root@master1 ~]#kubectl get pod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP             NODE
pod-qos-guaranteed  1/1     Running   0           65s   10.244.2.26    node3.wang.org
node3.wang.org      <none>   <none>

[root@master1 pod]#kubectl exec -it pod-qos-guaranteed -- bash
root@pod-limit-request:/# cat /proc/1/oom_score_adj
-997

#在节点上查看进程的oom_score_adj
[root@node3 ~]#ps aux|grep nginx
root      145599  0.0  0.0 10656 5916 ?        Ss   15:30   0:00 nginx: master
process nginx -g daemon off;
systemd+  145637  0.0  0.0 11092 2500 ?        S    15:30   0:00 nginx: worker
process
systemd+  145638  0.0  0.0 11092 2500 ?        S    15:30   0:00 nginx: worker
process
root      146234  0.0  0.0  9568 2388 pts/0    S+   15:31   0:00 grep --
color=auto nginx

[root@node3 ~]#cat /proc/145599/oom_score_adj
-997

[root@master1 ~]#kubectl delete -f pod-qos-guaranteed.yaml

```

示例: 中优先级

```

[root@master1 ~]#cat pod-qos-burstable.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-qos-burstable
spec:
  containers:
  - name: pod-qos-burstable-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    resources:
      limits:
        memory: "200Mi"

```

```
    cpu: "500m"
  requests:
    memory: "200Mi"
    cpu: "300m"
```

```
[root@master1 ~]#kubectl apply -f pod-qos-burstable.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-qos-burstable	1/1	Running	0	63s

```
[root@master1 ~]#kubectl describe pod pod-qos-burstable
```

```
.....
```

```
QoS Class:                      Burstable
```

```
.....
```

示例: 低优先级

```
[root@master1 ~]#cat pod-qos-besteffort.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-qos-besteffort
```

```
spec:
```

```
  containers:
```

```
  - name: pod-qos-besteffort-container
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
```

```
[root@master1 ~]#kubectl apply -f pod-qos-besteffort.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-qos-besteffort	1/1	Running	0	3s

```
[root@master1 ~]#kubectl describe pod pod-qos-besteffort
```

```
.....
```

```
QoS Class:                      BestEffort
```

```
.....
```

3.7 Pod 设计模式

3.7.1 容器设计模式

基于容器的分布式系统中常用三类设计模式

- 单容器模式：单一容器形式运行的应用,此模式应用最为广泛,相当于一个人
- 单节点多容器模式：由强耦合的多个容器协同共生,容器之间关系密切,通常需要工作在同一个 worker 主机上,相当于一个多成员的家庭,Pod 就是此模式的代表
- 多节点多容器模式：基于特定部署单元（Pod）实现分布式算法,通常容器间需要跨网络通信,相当于多个家庭此模式依赖于应用自身的实现,例如: MySQL 主从复制集群分布在不同节点的 Pod

3.7.2 单节点多容器模式

注意:此处的节点指一个pod,而非worker节点主机

3.7.2.1 单节点多容器模式

一种跨容器的设计模式

目的是在单个Pod之上同时运行多个共生关系的容器,因而容器管理系统需要由将它们作为一个原子单位进行统一调度

Pod概念就是这个设计模式的实现之一

一个容器有多个容器,分为两类

- 主容器: 完成主要核心业务
- 辅助容器: 提供辅助功能,比如监控,内核优化,代理等

3.7.2.2 单节点多容器模式的常见实现

辅助容器跟据和主容器的关系,可以由下面几种模式

- Init Container 模式

Init Container模式只会出现于容器初始化阶段

Init容器负责以不同于主容器的生命周期来完成那些必要的初始化任务,包括在文件系统上设置必要的特殊权限、数据库模式设置或为主应用程序提供初始数据等,但这些初始化逻辑无法包含在应用程序的镜像文件中,或者出于安全原因,应用程序镜像没有执行初始化活动的权限等等

一个Pod中可以同时定义多个Init容器

Init容器需要串行运行,且在所有Init容器均正常终止后,才能运行主容器

注意: 此模式用`kubectl get pods`显示一个Pod只有一个容器

- Sidecar 模式

Sidecar模式即边车(摩托车的挎斗)模式

Pod中的应用由主应用程序(通常是基于HTTP协议的应用程序)以及一个Sidecar的辅助容器组成

Sidecar做为辅助容器用于为主容器提供辅助服务以增强主容器的功能,是主应用程序是必不可少的一部分,但却未必非得运行为主容器的一部分,比如: 服务网格的Envoy代理, filebeat 日志收集

Sidecar容器可以对客户端和主容器之间的所有流量进行干预和监控

常见应用场景: 为主容器提供代理服务

- Ambassador 模式

Ambassador模式即大使模式,功能类似一国的大使

Pod中的应用由主应用程序和一个Ambassador容器组成

Ambassador辅助容器代表主容器发送网络请求至特定的外部环境中,因此可以将其视作主容器应用的“大使”

利用pod内多容器共享数据的特性,代理后端容器和外部的其他应用进行交流,有限的代理主容器做一些事情

Ambassador辅助容器通常对应用实现一些专用功能

比如:实现Redis smart 客户端功能从而代理普通Redis客户端可以访问Redis集群

典型应用场景: 云原生应用程序需要诸如断路、路由、计量和监视等功能,但更新已有的应用程序或现有代码库以添加这些功能可能很困难,使用Ambassador模式可以在进程外添加代理便成为一种有效的解决方案

相当于对主容器的正向代理

- Adapter 模式

Adapter模式即适配器模式

Pod中的应用由主应用程序和一个Adapter容器组成，主要为从外部应用向主应用容器访问提供支持，和Ambassador 模式方向相反

Adapter容器为主应用程序提供一致的接口，实现模块重用，支持主容器应用程序的标准化和规范化输出以便于从外部服务进行访问

Adapter容器帮助主容器通信过程中的信息格式修正,将客户端的数据格式转换成主容器能够识别的数据格式

常见应用场景: 外部prometheus服务通过Adapter 模式的Exporter容器抓取nginx容器的日志,中间Exporter需要转换两者之间的格式

相当于对主容器的反向代理

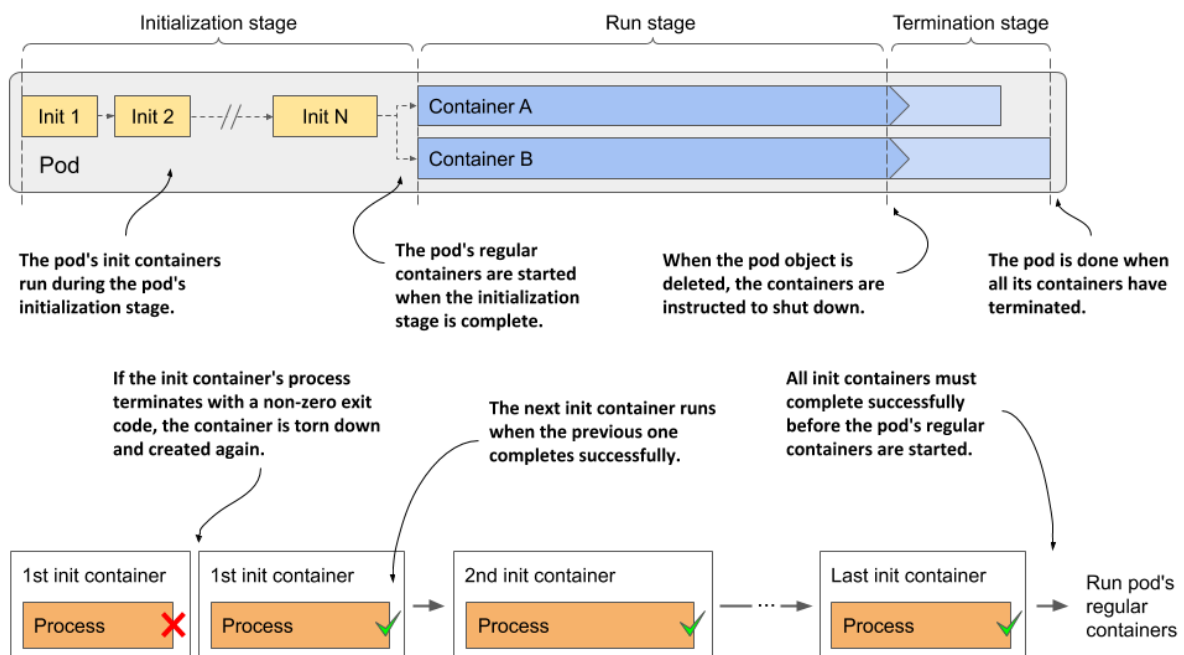
注意:

- 以上模式除Init Container 模式以外,配置语法都是一样,都是在一个Pod中配置多个容器
- 因此除Init Container 模式以外的三种模式,只是逻辑关系的分类而已
- 有时也可以统一称为SideCar模式

3.7.3 实战案例

3.7.3.1 init 模式案例

主容器启动之前要运行的容器，常用于执行一些预置和初始化操作



初始化init容器必须运行完成并成功后，主容器才能启动运行

如果有多个初始化容器,每个初始化容器都必须按定义顺序串行运行,并且全部成功退出后,主容器才能启动运行

```
#执行 kubectl logs <pod_name>, 可以看到如下提示
Error from server (BadRequest): container "<container_name>" in pod "<pod_name>"
is waiting to start: PodInitializing
```

属性信息:

kubectl explain pods.spec.initContainers

范例:

```
#通过init模式实现iptables规则的端口重定向
[root@master1 ~]#cat pod-init-container.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-init-container
  namespace: default
spec:
  initContainers:
  - name: iptables-init
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1
    imagePullPolicy: IfNotPresent
    command: ['/bin/sh', '-c']
    args: ['iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-
port 80']
    securityContext:
      capabilities:
        add:
        - NET_ADMIN
  containers:
  - name: demo
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    ports:
    - name: http
      containerPort: 80
```

```
[root@master1 ~]#kubectl apply -f pod-init-container.yaml
```

#查看容器IP

```
[root@master1 ~]#kubectl get pod init-container-demo -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod-init-container	1/1	Running	0	53s	172.16.4.37	
node2.wang.org	<none>		<none>			

#测试直接访问主容器

```
[root@master1 ~]#curl 172.16.4.37
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: init-container-
demo, ServerIP: 172.16.4.37!
```

#访问init容器指定的端口,实现间接访问主容器

```
[root@master1 ~]#curl 172.16.4.37:8080
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: init-container-
demo, ServerIP: 172.16.4.37!
```

范例: Init 容器的Pod

```
[root@master1 ~]#cat pod-init-test.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```

metadata:
  name: init-pod
  labels:
    app: init
spec:
  containers:
  - name: busybox
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    command: ['sh', '-c', 'echo The app is running! && sleep 3600']
  initContainers:
  - name: init-myservice
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    #image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    command: ['sh', '-c', 'until nslookup myservice; do echo waiting for
myservice; sleep 2; done;']

```

#启动文件

```
[root@master1 ~]#kubectl apply -f pod-init-test.yaml
```

#查看效果

```
[root@master1 ~]#kubectl get pod init-pod
```

NAME	READY	STATUS	RESTARTS	AGE
init-pod	0/1	Init:0/1	0	14s

#查看状态描述

```
[root@master1 ~]#kubectl describe pod init-pod
```

```

Name:          init-pod
...
Status:        Pending
IP:            10.244.2.8
Init Containers:
  init-myservice:
    ...
    State:      Running
    Started:    Fri, 28 Jun 2019 18:53:49 +0800
    Ready:      False
    ...
Containers:
  myapp-container:
    ...
    State:      Waiting
    Reason:     PodInitializing
    Ready:      False
    ...
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   36s   default-scheduler  Successfully assigned default/init-pod to node2
  Normal  Pulling     35s   kubelet, node2    Pulling image "busybox"
  Normal  Pulled      34s   kubelet, node2    Successfully pulled image
"busybox"
  Normal  Created     33s   kubelet, node2    Created container init-myservice
  Normal  Started     33s   kubelet, node2    Started container init-myservice
.

```

#查看pod中指定容器的日志效果

```
[root@master1 ~]#kubectl logs init-pod -c init-myservice|head
```

```

Server:      192.168.0.10
Address:     192.168.0.10:53

** server can't find myservice.default.svc.wang.org: NXDOMAIN

*** Can't find myservice.svc.wang.org: No answer
*** Can't find myservice.wang.org: No answer
*** Can't find myservice.wang.com: No answer
*** Can't find myservice.default.svc.wang.org: No answer
*** Can't find myservice.svc.wang.org: No answer

#创建所需要的service
[root@master1 ~]#vim pod-init-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: myservice
spec:
  ports:
  - protocol: TCP
    port: 80
    targetPort: 9527

#启动文件
[root@master1 ~]#kubectl apply -f pod-init-service.yaml
service/myservice created

#过一会儿再查看pod效果
[root@master1 ~]#kubectl get pod init-pod
NAME          READY   STATUS    RESTARTS   AGE
init-pod      1/1     Running   0           6m26s
#注意：这一步，有可能会因为dns的原因导致需要很长时间才会生效

```

3.7.3.2 sidecar 模式案例

在一个pod内启动两个容器，访问B容器的时候，都需要经过A容器，只有通过A处理后的数据才会发送给B容器。

在整个过程中，A容器就是B容器应用的代理服务器

范例：

```

#定制资源配置文件
[root@master1 ~]#wget
http://www.wangxiaochun.com:8888/kubernetes/yaml/envoy.yaml
[root@master1 ~]#cat envoy.yaml
admin:
  access_log_path: /tmp/admin_access.log
  address:
    socket_address: { address: 0.0.0.0, port_value: 9901 }

static_resources:
  listeners:
  - name: listener_0
    address:
      socket_address: { address: 0.0.0.0, port_value: 80 }
    filter_chains:
    - filters:

```

```

- name: envoy.http_connection_manager
  config:
    stat_prefix: ingress_http
    codec_type: AUTO
    route_config:
      name: local_route
      virtual_hosts:
        - name: local_service
          domains: ["*"]
          routes:
            - match: { prefix: "/" }
              route: { cluster: local_service }
    http_filters:
      - name: envoy.router

```

clusters:

```

- name: local_service
  connect_timeout: 0.25s
  type: STATIC
  lb_policy: ROUND_ROBIN
  load_assignment:
    cluster_name: local_service
    endpoints:
      - lb_endpoints:
          - endpoint:
              address:
                socket_address:
                  address: 127.0.0.1
                  port_value: 8080

```

[root@master1 ~]#cat pod-sidecar-test.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: sidecar-test
spec:
  containers:
    - name: proxy
      #image: envoyproxy/envoy-alpine:v1.14.1
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/envoy-alpine:v1.14.1
      command: ['sh', '-c', 'sleep 5 && envoy -c /etc/envoy/envoy.yaml']
      lifecycle:
        postStart:
          exec:
            command: ["/bin/sh", "-c", "wget -O /etc/envoy/envoy.yaml
http://www.wangxiaochun.com:8888/kubernetes/yaml/envoy.yaml"]
    - name: pod-test
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
      env:
        - name: HOST
          value: "127.0.0.1"
        - name: PORT
          value: "8080"

```

#启动文件

[root@master1 ~]#kubectl apply -f pod-sidecar-test.yaml

#检查效果

```
[root@master1 ~]#kubectl get pod sidecar-test -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS	GATES		
sidecar-test	2/2	Running	0	88s	172.16.3.46	node1.wang.org
<none>	<none>					

```
[root@master1 ~]#curl 172.16.3.46
```

```
kubernetes pod-test v0.1!! ClientIP: 127.0.0.1, ServerName: sidecar-test,  
ServerIP: 172.16.3.46!
```

```
[root@master1 ~]#curl -I 172.16.3.46
```

```
HTTP/1.1 200 OK
```

```
content-type: text/html; charset=utf-8
```

```
content-length: 97
```

```
server: envoy
```

```
date: Wed, 17 Mar 2021 03:32:13 GMT
```

```
x-envoy-upstream-service-time: 1
```

#结果显示: 这个结果是 x-envoy-upstream-service-time 响应的数据信息, 而不是真正的pod-test 容器

#获取大量的监控项

```
[root@master1 ~]#curl 172.16.3.46:9901/help
```

```
[root@master1 ~]#curl 172.16.3.46:9901/stats
```

```
[root@master1 ~]#curl 172.16.3.46:9901/stats/prometheus/
```

3.7.3.3 Ambassador 模式案例

#查看kubectl proxy的用法

```
[root@master1 ~]#kubectl proxy --help
```

```
.....
```

Usage:

```
kubectl proxy [--port=PORT] [--www=static-dir] [--www-prefix=prefix] [--api-  
prefix=prefix] [options]
```

#前台运行proxy, 默认监听127.0.0.1:8001

```
[root@master1 ~]#kubectl proxy --address=0.0.0.0
```

```
Starting to serve on [::]:8001
```

```
.....
```

#测试在master节点本机访问成功

```
[root@master1 ~]#curl -s 127.0.0.1:8001/api/v1/namespaces/default/pods
```

```
{  
  "kind": "PodList",  
  "apiVersion": "v1",  
  "metadata": {  
    "resourceVersion": "1258001"  
  },  
  "items": [  
    {  
      "metadata": {  
.....
```

#worker节点访问失败, 权限不足

```
[root@node1 ~]#curl -s kubeapi.wang.org:8001/api/v1/namespaces/default/pods  
Forbidden
```

```
[root@master1 ~]#cat pod-ambassador-container.yaml
apiVersion: v1
kind: Pod
metadata:
  name: ambassador-container-demo
spec:
  containers:
  - name: curl
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1
    command: ["sleep", "99999"]
  - name: ambassador
    image: bitnami/kubectl:latest
    command: ["/bin/sh", "-c", "kubectl proxy"]
    args:
      # 传递给 kubectl proxy 的选项，若不使用默认监听的tcp/8001端口，可以附加"--port=NUM"选项
      - --server="https://kubernetes.default.svc"
      - --certificate-
authority="/var/run/secrets/kubernetes.io/serviceaccount/ca.crt"
      - --token="$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)"
      - --accept-paths='^.*\*'

[root@master1 ~]#kubectl apply -f pod-ambassador-container.yaml

#查看成功运行
[root@master1 ~]#kubectl get pod ambassador-container-demo
NAME                                READY   STATUS    RESTARTS   AGE
ambassador-container-demo          2/2     Running   0           52s

#进入容器测试访问
[root@master1 ~]#kubectl exec -it ambassador-container-demo -c curl -- sh
root@ambassador-container-demo # curl 127.0.0.1:8001/api/
{
  "kind": "APIVersions",
  "versions": [
    "v1"
  ],
  "serverAddressByClientCIDRs": [
    {
      "clientCIDR": "0.0.0.0/0",
      "serverAddress": "10.0.0.200:6443"
    }
  ]
}
```

3.7.3.4 Adapter 模式案例

```
[root@master1 ~]#cat pod-adapter-container.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  default.conf: |
    server {
      listen      80;
```

```

server_name localhost;
location / {
    root    /usr/share/nginx/html;
    index  index.html index.htm;
}
error_page 500 502 503 504 /50x.html;
location = /50x.html {
    root    /usr/share/nginx/html;
}
location /nginx_status {
    stub_status;
    allow 127.0.0.1;
    deny all;
}
}
}
---
apiVersion: v1
kind: Pod
metadata:
  name: adapter-container-demo
spec:
  containers:
  - name: nginx
    image: nginx:alpine
    ports:
    - containerPort: 80
    volumeMounts:
    - mountPath: /etc/nginx/conf.d/
      name: nginx-conf
      readOnly: true
  - name: adapter
    image: nginx/nginx-prometheus-exporter:latest
    args: ["-nginx.scrape-uri", "http://localhost/nginx_status"]
    ports:
    # nginx-prometheus-exporter默认监听tcp/9113端口
    - name: exporter
      containerPort: 9113
  volumes:
  - name: nginx-conf
    configMap:
      name: nginx-conf
      items:
      - key: default.conf
        path: default.conf

#应用
[root@master1 ~]#kubectl apply -f pod-adapter-container.yaml

#查看容器的IP
[root@master1 ~]#kubectl get pod adapter-container-demo -o wide

```

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
adapter-container-demo	2/2	Running	0	54s	10.244.1.142		
node1.wang.org	<none>		<none>				

```

#访问adapter容器
[root@master1 ~]#curl 10.244.1.142:9113
<!DOCTYPE html>

```

```
<title>NGINX Exporter</title>
<h1>NGINX Exporter</h1>
<p><a href="/metrics">Metrics</a></p>
```

#查看/metrics

```
[root@master1 ~]#curl 10.244.1.142:9113/metrics
```

HELP nginx_connections_accepted Accepted client connections

TYPE nginx_connections_accepted counter

nginx_connections_accepted 1

HELP nginx_connections_active Active client connections

TYPE nginx_connections_active gauge

nginx_connections_active 1

HELP nginx_connections_handled Handled client connections

TYPE nginx_connections_handled counter

nginx_connections_handled 1

HELP nginx_connections_reading Connections where NGINX is reading the request header

TYPE nginx_connections_reading gauge

nginx_connections_reading 0

HELP nginx_connections_waiting Idle client connections

TYPE nginx_connections_waiting gauge

nginx_connections_waiting 0

HELP nginx_connections_writing Connections where NGINX is writing the response back to the client

TYPE nginx_connections_writing gauge

nginx_connections_writing 1

HELP nginx_http_requests_total Total http requests

TYPE nginx_http_requests_total counter

nginx_http_requests_total 2

HELP nginx_up Status of the last metric scrape

TYPE nginx_up gauge

nginx_up 1

HELP nginxexporter_build_info Exporter build information

TYPE nginxexporter_build_info gauge

nginxexporter_build_info{commit="7a03d0314425793cf4001f0d9b0b2cfd19563433",date="2021-12-21T19:24:34Z",version="0.10.0"} 1